

Catatan Ujian Akhir Semester
IF2224 Teori Bahasa Formal dan Otomata
Semester 4 Tahun 2025/2026



NARENDRA DHARMA WISTARA M.
13524044

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
10 Juni 2026

Daftar Isi

1	Informasi Ujian	1
2	Capaian Pembelajaran Mata Kuliah (CPMK)	1
2.1	Tabel Keterpenuhan CPMK	1
3	Daftar Materi yang Diujikan	1
4	Outline Keseluruhan Materi	1
4.1	PDA, termasuk konversi CFG	1
4.2	CYK	2
4.3	Mesin Turing	2
4.4	Compiler	2
4.5	Recursive Descent Parser	2
4.6	LL(1) Parser	2
4.7	Intermediate Code	3
4.8	Interpreter dan Virtual Machine	3
5	PDA	4
5.1	Outline Fundamental Konsep Materi	4
5.2	Penjelasan Konsep	5
5.2.1	PDA sebagai ϵ -NFA dengan Stack	5
5.2.2	Definisi Formal 7-Tuple	5
5.2.3	Fungsi Transisi dan Operasi Stack	6
5.2.4	Diagram Transisi dan Label $a, X/\gamma$	7
5.2.5	Instantaneous Description (q, w, γ)	7
5.2.6	Relasi $\vdash$ dan $\vdash^*$	7
5.2.7	Sifat Validitas ID	8
5.2.8	Acceptance by Final State	8
5.2.9	Acceptance by Empty Stack	9
5.2.10	Ekuivalensi Mode Penerimaan	9
5.2.11	Konversi CFG ke PDA	10
5.2.12	Mengapa $N(P_G) = L(G)$	10
5.2.13	Konversi PDA ke CFG	11
5.2.14	Pertumbuhan Ukuran Grammar pada Konversi PDA ke CFG	12
5.2.15	Derivasi Leftmost/Rightmost dan Parse Tree	13
5.2.16	Deterministic Pushdown Automata	14
5.2.17	Prefix Property dan Unambiguous CFG	14
5.2.18	Closure Properties CFL via PDA	15
5.3	Algoritma dan Rumus Penting	16
5.3.1	Fungsi Transisi PDA	16
5.3.2	Langkah ID PDA	16
5.3.3	Mode Penerimaan PDA	16
5.3.4	Pseudocode Simulasi PDA Nondeterministik	17
6	CYK	18
6.1	Outline Fundamental Konsep Materi	18
6.2	Penjelasan Konsep	19
6.2.1	Membership Problem untuk CFL	19
6.2.2	Kelemahan Pendekatan Naif	19
6.2.3	CYK sebagai Dynamic Programming	20
6.2.4	Dependensi terhadap Chomsky Normal Form	20
6.2.5	Clean-Up Grammar Menuju CNF	21

6.2.6	Struktur Tabel X_{ij}	22
6.2.7	Langkah Basis	22
6.2.8	Langkah Induksi	23
6.2.9	Acceptance Condition	23
6.2.10	Kompleksitas CYK	23
6.2.11	Contoh Simulasi dengan baaba	24
6.2.12	CYK dan Decidability CFL	25
6.3	Algoritma dan Rumus Penting	25
6.3.1	Definisi Sel X_{ij}	25
6.3.2	Basis dan Induksi CYK	25
6.3.3	Pseudocode CYK	26
7	Mesin Turing	27
7.1	Outline Fundamental Konsep Materi	27
7.2	Penjelasan Konsep	28
7.2.1	Motivasi Batas Komputasi	28
7.2.2	Church-Turing Thesis	29
7.2.3	Definisi Formal 7-Tuple	29
7.2.4	Fungsi Transisi	30
7.2.5	Instantaneous Description $\alpha q \beta$	30
7.2.6	Aturan Gerak Kanan dan Kiri	30
7.2.7	Trace Eksekusi untuk $0^n 1^n$	31
7.2.8	Acceptance by Final State	32
7.2.9	Acceptance by Halting	33
7.2.10	Mesin Turing sebagai Transducer	33
7.2.11	Mesin Turing sebagai Enumerator	34
7.2.12	Recursively Enumerable dan Recursive Language	34
7.2.13	Teknik Pemrograman Mesin Turing	35
7.2.14	Multitape dan Nondeterministic TM	36
7.2.15	Multi-Head, Two-Way, dan Off-Line TM	36
7.2.16	Restricted TM	36
7.2.17	Hubungan dengan Komputer Riil	37
7.2.18	Encoding Mesin Turing	37
7.2.19	Diagonalization Language	38
7.2.20	Universal Language	38
7.2.21	Post Correspondence Problem	38
7.2.22	Reduction	39
7.2.23	Rice's Theorem	39
7.3	Algoritma dan Rumus Penting	40
7.3.1	Fungsi Transisi Mesin Turing	40
7.3.2	Bahasa yang Diterima dan Dihentikan	40
7.3.3	Pseudocode Universal Turing Machine	40
8	Compiler	42
8.1	Outline Fundamental Konsep Materi	42
8.2	Penjelasan Konsep	43
8.2.1	Tujuan Desain Bahasa dan Compilability	43
8.2.2	Compiler, Interpreter, dan Virtual Machine	43
8.2.3	Language-Processing System	44
8.2.4	Bootstrapping dan Translator Tools	44
8.2.5	Front End dan Back End	45
8.2.6	Fase-Fase Compiler	46
8.2.7	Symbol Table	46

8.2.8	Error Handling dalam Pipeline Compiler	47
8.2.9	Lexeme, Pattern, dan Token	48
8.2.10	Input Buffering dan Sentinel	48
8.2.11	Regular Expression dan Transition Diagram	48
8.2.12	Lex/Flex dan Longest Prefix	49
8.2.13	Syntax Analysis sebagai Konteks CFG	49
8.2.14	Bottom-Up Parsing, LR Parser, dan Yacc	49
8.2.15	Semantic Analysis, SDD, dan SDT	50
8.2.16	Type Checking, Coercion, dan Overloading	50
8.2.17	Runtime Environment	51
8.2.18	Activation Record	51
8.2.19	Heap dan Garbage Collection	52
8.2.20	Basic Block dan Flow Graph	52
8.2.21	Register dan Address Descriptor	52
8.2.22	Instruction Selection, Register Allocation, dan Peephole Optimization	53
8.2.23	Data-Flow Analysis dan Gen-Kill	53
8.2.24	Reaching Definitions	54
8.2.25	Available Expressions	54
8.2.26	Live Variables	55
8.3	Algoritma dan Rumus Penting	55
8.3.1	Gen-Kill Transfer Function	55
8.3.2	Pseudocode Iterative Data-Flow Analysis	55
9	Recursive Descent Parser	57
9.1	Outline Fundamental Konsep Materi	57
9.2	Penjelasan Konsep	58
9.2.1	Top-Down Parsing	58
9.2.2	Definisi Recursive Descent Parser	59
9.2.3	Token, Lookahead, dan Pointer Input	59
9.2.4	Algoritma Prosedur Nonterminal	59
9.2.5	Brute-Force Parsing dan Recursive Parsing	60
9.2.6	Terminal Matching	60
9.2.7	Backtracking	61
9.2.8	Contoh Fungsi E dan T	61
9.2.9	Left Recursion	62
9.2.10	Eliminasi Left Recursion	62
9.2.11	Left Factoring	63
9.2.12	Predictive Parsing	64
9.2.13	Recursive Descent untuk Syntax-Directed Translation	64
9.3	Algoritma dan Rumus Penting	65
9.3.1	Skema Umum Fungsi Nonterminal	65
9.3.2	Eliminasi Left Recursion	65
9.3.3	Left Factoring	66
10	LL(1) Parser	67
10.1	Outline Fundamental Konsep Materi	67
10.2	Penjelasan Konsep	68
10.2.1	Definisi LL(1)	68
10.2.2	Hubungan dengan Top-Down Parsing	68
10.2.3	FIRST Set	69
10.2.4	FOLLOW Set	69
10.2.5	Tabel Parsing LL(1)	70
10.2.6	Kondisi Grammar LL(1)	71

10.2.7	Algoritma Nonrecursive Predictive Parsing	72
10.2.8	Transformasi Grammar untuk LL(1)	72
10.2.9	Error Handling dan Error Recovery	73
10.2.10	synch Entries dan Phrase-Level Recovery	74
10.3	Algoritma dan Rumus Penting	74
10.3.1	Rumus FIRST	74
10.3.2	Rumus FOLLOW	75
10.3.3	Algoritma Konstruksi Tabel LL(1)	75
10.3.4	Algoritma Nonrecursive Predictive Parser	75
10.3.5	Trace Parsing <code>int * int</code>	76
11	Intermediate Code	77
11.1	Outline Fundamental Konsep Materi	77
11.2	Penjelasan Konsep	78
11.2.1	Motivasi Intermediate Representation	78
11.2.2	Posisi IR dalam Pipeline Compiler	78
11.2.3	Syntax Tree dan DAG	79
11.2.4	Value-Number dan Hash Node pada DAG	79
11.2.5	Postfix Notation sebagai Intermediate Code	80
11.2.6	Three-Address Code	81
11.2.7	Quadruples, Triples, dan Indirect Triples	82
11.2.8	Translasi Ekspresi, Array, dan Pointer	82
11.2.9	Boolean dan Control Flow	83
11.2.10	Backpatching	83
11.2.11	Kaitan dengan Contoh Soal UAS	84
11.2.12	Optimasi dan Deteksi pada Intermediate Code	84
11.3	Algoritma dan Rumus Penting	85
11.3.1	Rumus Offset Array	85
11.3.2	Pseudocode Pembuatan Three-Address Code Ekspresi	86
11.3.3	Pseudocode Konstruksi Tabel Quadruple	86
11.3.4	Pseudocode Backpatching	86
12	Interpreter dan Virtual Machine	87
12.1	Outline Fundamental Konsep Materi	87
12.2	Penjelasan Konsep	88
12.2.1	Compiler, Interpreter, VM, Bytecode, dan JIT	88
12.2.2	HotSpot, Client Compiler, dan Server Compiler	89
12.2.3	Motivasi Model Hibrida Compiler dan VM	89
12.2.4	Posisi Interpreter dan VM dalam Language-Processing System	90
12.2.5	Stack-Based VM	90
12.2.6	Register-Based VM	90
12.2.7	Instruction Set dan Bytecode	91
12.2.8	Runtime Memory Model	91
12.2.9	Siklus Fetch-Decode-Execute	92
12.2.10	Optimasi Interpreter VM	92
12.2.11	Garbage Collection	93
12.2.12	Relasi VM dengan Intermediate Code dan Code Generation	93
12.2.13	Kaitan dengan Contoh UAS	94
12.3	Algoritma dan Rumus Penting	94
12.3.1	Pseudocode Interpreter Loop	94
12.3.2	Pseudocode Eksekusi Instruksi Stack	95
12.3.3	Pseudocode Mark-and-Sweep	95

1 Informasi Ujian

Jenis Ujian	UAS
Nama Mata Kuliah	IF2224 Teori Bahasa Formal dan Otomata
Tanggal Ujian	Kamis, 11 Juni 2026

Tabel 1: Informasi ujian

2 Capaian Pembelajaran Mata Kuliah (CPMK)

1. Menjelaskan konsep, notasi dan penerapan konsep teori automaton dan bahasa formal seperti deterministik, non-deterministik, dan penerapannya dalam compiler, hierarki automaton, dan teori bahasa formal terkait dengan batasan kompleksitas dari permasalahan praktik.
2. Merancang automaton (finite automaton, push down automaton, turing machine) untuk permasalahan tertentu

2.1 Tabel Keterpenuhan CPMK

CPMK	PDA	CYK	TM	Comp.	RD	LL(1)	IR	VM
CPMK 1	✓	✓	✓	✓	✓	✓	✓	✓
CPMK 2	✓		✓		✓	✓		

Tabel 2: Keterpenuhan CPMK per materi yang dibahas

3 Daftar Materi yang Diujikan

1. PDA, termasuk konversi CFG
2. CYK
3. Mesin Turing
4. Compiler
5. Recursive Descent Parser
6. LL(1) Parser
7. Intermediate Code
8. Interpreter dan Virtual Machine

4 Outline Keseluruhan Materi

Bagian ini menyajikan peta belajar ringkas untuk seluruh materi yang diujikan. Penjelasan mendalam tersedia pada section masing-masing materi yang sudah disusun.

4.1 PDA, termasuk konversi CFG

- Memahami PDA sebagai finite automaton nondeterministik yang dilengkapi stack.
- Menguasai definisi formal, fungsi transisi, instantaneous description, dan dua mode penerimaan.

- Menguasai konstruksi ekuivalensi CFG ke PDA dan PDA ke CFG.
- Berlatih derivasi leftmost/rightmost, parse tree, dan walkthrough konversi CFG/PDA bertipe UAS.
- Memahami peran DPDA, prefix property, closure properties CFL via PDA, dan batas determinisme pada CFL.

4.2 CYK

- Membership problem untuk CFL dan kebutuhan Chomsky Normal Form.
- Clean-up grammar menuju CNF: nullable, unit pairs, generating, dan reachable symbols.
- Tabel segitiga CYK, basis, langkah induksi, dan kondisi $S \in X_{1n}$.
- Kompleksitas waktu $O(n^3)$ dan ruang $O(n^2)$.

4.3 Mesin Turing

- Model komputasi umum, definisi formal, fungsi transisi, dan instantaneous description.
- Trace eksekusi TM pada input konkret dan strategi mark-scan untuk bahasa seimbang.
- Acceptance, halting, transducer, enumerator, recursive language, recursively enumerable language, teorema komplemen RE, dan undecidability.
- Encoding, universal language, reduction, PCP, dan Rice's Theorem.
- Varian TM serta ekuivalensinya dengan model dasar.

4.4 Compiler

- Sistem pemrosesan bahasa, fase front end, middle end, dan back end.
- Lexical analysis, parsing, bottom-up LR/Yacc, semantic analysis, symbol table, error handling, intermediate representation, runtime environment, code generation, dan optimization.
- Instruction selection, register allocation, peephole optimization, dan single accumulator code generation.

4.5 Recursive Descent Parser

- Top-down parsing sebagai leftmost derivation.
- Satu prosedur per nonterminal, backtracking, left recursion, left factoring, dan hubungan dengan SDT.

4.6 LL(1) Parser

- FIRST, FOLLOW, tabel parsing prediktif, dan kondisi grammar LL(1).
- Algoritma nonrecursive predictive parsing berbasis stack, **synch** entries, dan phrase-level error recovery.

4.7 Intermediate Code

- Tujuan IR sebagai jembatan front end dan back end.
- AST, DAG dengan value-number, postfix notation, three-address code, quadruples, triples, BZ/BR, control-flow translation, backpatching, dan optimasi IR.
- Constant folding, common subexpression elimination, dead code, unreachable code, dan optimasi loop.

4.8 Interpreter dan Virtual Machine

- Perbandingan compiler, interpreter, bytecode, VM, dan JIT.
- Stack-based VM, runtime memory model, garbage collection, HotSpot JIT, dan relasi VM dengan code generation.

5 PDA

Pushdown Automata (PDA) adalah model automaton untuk bahasa bebas konteks, yaitu kelas bahasa yang lebih kuat daripada regular language tetapi masih lebih terbatas daripada model komputasi umum seperti Mesin Turing. Fokus utama materi ini adalah memahami bagaimana stack memberi PDA memori tak terbatas secara terstruktur, bagaimana komputasi PDA dinyatakan secara formal, dan mengapa PDA ekuivalen dengan Context-Free Grammar (CFG).

Dalam konteks ujian, PDA perlu dipahami sebagai objek desain sekaligus objek pembuktian: mahasiswa harus dapat membaca atau membuat transisi PDA, menentukan mode penerimaan, menelusuri instantaneous description, serta menjelaskan konstruksi konversi antara CFG dan PDA.

5.1 Outline Fundamental Konsep Materi

1. **WAJIB** Konsep dasar Pushdown Automata
 - (a) **WAJIB** PDA sebagai ϵ -NFA dengan stack
 - (b) **WAJIB** Definisi formal 7-tuple
 - (c) **WAJIB** Fungsi transisi dan operasi stack
 - (d) **PAHAM** Contoh bahasa yang membutuhkan stack
2. **WAJIB** Representasi komputasi PDA
 - (a) **WAJIB** Diagram transisi dan label $a, X/\gamma$
 - (b) **WAJIB** Instantaneous Description (q, w, γ)
 - (c) **WAJIB** Relasi $\vdash$ dan $\vdash^*$
 - (d) **PAHAM** Sifat validitas ID
3. **WAJIB** Penerimaan bahasa oleh PDA
 - (a) **WAJIB** Acceptance by final state
 - (b) **WAJIB** Acceptance by empty stack
 - (c) **WAJIB** Ekuivalensi kedua mode penerimaan
4. **WAJIB** Ekuivalensi PDA dan CFG
 - (a) **WAJIB** Konversi CFG ke PDA
 - (b) **WAJIB** Konversi PDA ke CFG
 - (c) **PAHAM** Pertumbuhan ukuran grammar pada konversi
 - (d) **WAJIB** Derivasi leftmost/rightmost dan parse tree
5. **PAHAM** Deterministic Pushdown Automata
 - (a) **PAHAM** Syarat determinisme DPDA
 - (b) **PAHAM** Hierarki $Regular \subset L(DPDA) \subset CFL$
 - (c) **PAHAM** Prefix property dan unambiguous CFG
6. **PAHAM** Keterbatasan dan sifat tambahan
 - (a) **PAHAM** Perbedaan NPDA dan DPDA
 - (b) **PAHAM** Sifat closure DCFL
 - (c) **PAHAM** Closure CFL via product PDA-DFA
 - (d) **PAHAM** Inverse homomorphism untuk CFL
 - (e) **PAHAM** Strategi desain PDA dari pola bahasa

5.2 Penjelasan Konsep

5.2.1 PDA sebagai ϵ -NFA dengan Stack

Inti Konsep.

PDA adalah ϵ -NFA yang dilengkapi stack, sehingga automaton dapat menyimpan informasi dalam pola Last-In, First-Out. Stack membuat PDA mampu mengenali struktur bersarang atau berpasangan yang tidak dapat dikenali finite automaton biasa.

Finite automaton hanya mengingat informasi melalui state yang jumlahnya berhingga. Keterbatasan ini membuat finite automaton gagal mengenali bahasa yang membutuhkan pencocokan jumlah atau struktur bersarang, misalnya bahasa dengan jumlah simbol pembuka dan penutup yang harus cocok. PDA menambahkan **stack**, yaitu memori yang hanya dapat diakses dari puncaknya, sehingga informasi yang terakhir disimpan akan menjadi informasi pertama yang dapat diambil kembali.

Secara informal, satu langkah PDA menggabungkan tiga aksi: membaca satu simbol input atau tidak membaca input sama sekali melalui transisi ϵ , berpindah state, dan mengganti simbol pada puncak stack. Karena PDA boleh nondeterministic, pada satu konfigurasi yang sama PDA dapat memiliki beberapa pilihan transisi; sebuah input diterima jika ada setidaknya satu jalur komputasi yang berhasil memenuhi kondisi penerimaan.

Komentar 5.1: Catatan: Sumber Pengetahuan Umum

Contoh klasik yang menonjolkan kebutuhan stack adalah $L = \{a^n b^n \mid n \geq 0\}$. PDA dapat melakukan push satu simbol stack untuk setiap a , lalu melakukan pop satu simbol untuk setiap b . Bahasa ini bukan regular karena finite automaton tidak memiliki memori tak terbatas untuk menghitung jumlah a yang harus dicocokkan dengan jumlah b .

Konsep ini menjadi dasar CPMK 1 karena memperlihatkan batas finite automaton dan posisi PDA dalam hierarki automaton. Konsep ini juga menjadi dasar CPMK 2 karena desain PDA selalu dimulai dari keputusan informasi apa yang harus disimpan di stack.

5.2.2 Definisi Formal 7-Tuple

Inti Konsep.

Secara formal PDA dinyatakan sebagai tuple $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$. Tuple ini menentukan state, alfabet input, alfabet stack, fungsi transisi, state awal, simbol awal stack, dan state penerima.

Definisi formal diperlukan agar perilaku PDA tidak hanya dipahami melalui gambar, tetapi juga dapat dibuktikan secara matematis. Setiap komponen tuple membatasi apa yang boleh dibaca, apa yang boleh berada di stack, serta kapan sebuah string dinyatakan diterima.

Komponen	Definisi atau Peran
Q	Himpunan berhingga state, yaitu kendali terbatas PDA.
Σ	Alfabet input, yaitu simbol-simbol yang boleh muncul pada string masukan.
Γ	Alfabet stack, yaitu simbol-simbol yang boleh disimpan di stack.
δ	Fungsi transisi yang menentukan pilihan state berikutnya dan perubahan stack.
q_0	State awal tempat PDA mulai beroperasi.
Z_0	Simbol awal stack, biasanya berperan sebagai penanda dasar stack.
F	Himpunan state akhir atau accepting states untuk mode penerimaan final state.

Tabel 3: Komponen formal PDA

Jika PDA digunakan dengan acceptance by empty stack, komponen F tidak menentukan penerimaan dan kadang modelnya ditulis sebagai 6-tuple. Namun dalam catatan ini bentuk 7-tuple tetap digunakan karena memudahkan perbandingan dua mode penerimaan.

Definisi formal ini terhubung langsung dengan instantaneous description, karena ID hanya memotret tiga bagian yang berubah selama eksekusi: state saat ini, sisa input, dan isi stack.

5.2.3 Fungsi Transisi dan Operasi Stack

Inti Konsep.

Fungsi transisi PDA menentukan apa yang terjadi ketika PDA berada di state tertentu, melihat simbol input tertentu atau ϵ , dan melihat simbol tertentu di puncak stack. Hasil transisi adalah pilihan pasangan state baru dan string stack pengganti.

Fungsi transisi PDA ditulis:

$$\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow 2^{Q \times \Gamma^*}.$$

Jika $(p, \alpha) \in \delta(q, a, X)$, artinya PDA yang sedang berada di state q , membaca simbol input a atau ϵ , dan melihat X di puncak stack boleh berpindah ke state p serta mengganti X dengan string stack α .

Perubahan stack selalu terjadi pada puncak. Jika stack sebelumnya $X\beta$, maka setelah transisi stack menjadi $\alpha\beta$. Bagian β tidak disentuh karena berada di bawah puncak stack.

Bentuk Pengganti	Makna Operasi
$\alpha = \epsilon$	Pop: simbol puncak X dihapus.
$\alpha = X$	No-op terhadap stack: X diganti dengan dirinya sendiri.
$\alpha = Y$	Replace: X diganti dengan satu simbol stack baru Y .
$\alpha = YZ$	Push berantai: X diganti oleh dua simbol, dengan simbol paling kiri menjadi puncak stack.

Tabel 4: Makna operasi stack dari hasil transisi

Sumber menekankan aturan ****no mixing and matching****. Jika $\delta(q, a, X) = \{(p, YZ), (r, \epsilon)\}$, PDA harus memilih satu pasangan secara utuh. PDA tidak boleh mengambil state p dari pasangan pertama tetapi operasi stack ϵ dari pasangan kedua.

Fungsi transisi adalah inti desain PDA: untuk merancang PDA, kita menentukan simbol apa yang disimpan saat fase pertama input dibaca, kapan automaton berpindah fase, dan kapan stack harus dikosongkan atau dipertahankan.

5.2.4 Diagram Transisi dan Label $a, X/\gamma$

Inti Konsep.

Diagram transisi PDA menggambarkan state sebagai node dan transisi sebagai panah berlabel $a, X/\gamma$. Label tersebut berarti PDA membaca a , mensyaratkan X di puncak stack, lalu mengganti X dengan γ .

Diagram transisi membantu membaca perilaku PDA tanpa menuliskan seluruh fungsi δ dalam bentuk himpunan. Jika label panah adalah $a, X/\gamma$, transisi itu hanya dapat dipakai ketika dua syarat terpenuhi: simbol input berikutnya adalah a atau $a = \epsilon$, dan puncak stack adalah X .

Jika a adalah simbol input nyata, pointer input maju satu simbol. Jika $a = \epsilon$, PDA tidak mengonsumsi input sehingga transisi dapat terjadi spontan. Transisi ϵ sering dipakai untuk berpindah fase, misalnya dari fase push ke fase pop, atau untuk menebak produksi CFG pada konstruksi CFG ke PDA.

Diagram dan tuple formal saling melengkapi. Diagram cocok untuk desain dan simulasi cepat, sedangkan tuple dan fungsi δ cocok untuk pembuktian, konversi, dan penulisan rumus penerimaan.

5.2.5 Instantaneous Description (q, w, γ)

Inti Konsep.

Instantaneous Description (ID) adalah snapshot konfigurasi PDA pada suatu saat. ID berbentuk (q, w, γ) , dengan q state saat ini, w sisa input, dan γ isi stack dengan simbol puncak di posisi paling kiri.

ID diperlukan karena state saja tidak cukup untuk menjelaskan kondisi PDA. PDA juga mempunyai input yang belum dibaca dan stack yang dapat berubah panjang selama komputasi. Dengan ID, setiap langkah komputasi dapat dinyatakan secara formal.

Komponen ID	Makna
q	State PDA saat ini.
w	Sisa input yang belum dikonsumsi.
γ	Isi stack. Simbol paling kiri adalah puncak stack.

Tabel 5: Komponen instantaneous description PDA

Sebagai contoh, ID (q, abb, XZ_0) berarti PDA sedang berada di state q , belum membaca string abb , dan puncak stack adalah X dengan Z_0 di bawahnya. Jika transisi berikutnya mem-pop X , maka bagian bawah stack Z_0 tetap ada.

ID menghubungkan pembacaan intuitif diagram dengan bukti formal. Rumus penerimaan final state dan empty stack sama-sama ditulis menggunakan ID awal dan ID akhir.

5.2.6 Relasi $\vdash$ dan $\vdash^*$

Inti Konsep.

Relasi $\vdash$ menyatakan satu langkah komputasi PDA dari satu ID ke ID berikutnya. Relasi $\vdash^*$ menyatakan nol, satu, atau banyak langkah komputasi.

Jika $(p, \alpha) \in \delta(q, a, X)$, maka langkah formalnya ditulis:

$$(q, aw, X\beta) \vdash (p, w, \alpha\beta).$$

Rumus ini mengatakan bahwa PDA mengonsumsi simbol a dari awal sisa input aw , mengganti puncak stack X dengan α , dan membiarkan bagian bawah stack β tetap sama.

Jika transisi menggunakan input ϵ , bentuknya dapat dibaca sebagai:

$$(q, w, X\beta) \vdash (p, w, \alpha\beta),$$

karena tidak ada simbol input yang dikonsumsi. Sementara itu, $\vdash^*$ adalah penutupan refleksif-transitif dari $\vdash$, sehingga $(I \vdash^* I)$ selalu benar untuk nol langkah, dan $I_0 \vdash^* I_k$ benar jika ada rantai $I_0 \vdash I_1 \vdash \dots \vdash I_k$.

Relasi ini penting dalam pembuktian karena definisi bahasa yang diterima PDA tidak merujuk pada diagram, melainkan pada ada atau tidaknya rantai ID dari konfigurasi awal menuju konfigurasi penerima.

5.2.7 Sifat Validitas ID

Inti Konsep.

Sifat validitas ID menjelaskan bahwa komputasi PDA tetap legal jika bagian input yang tidak disentuh atau bagian bawah stack yang tidak terlihat diperlakukan secara konsisten. Sifat ini berguna dalam pembuktian induktif tentang komputasi PDA.

Karena PDA hanya membaca input dari kiri ke kanan dan hanya melihat puncak stack, bagian tertentu dari konfigurasi tidak memengaruhi langkah lokal. Sumber menyebut tiga sifat utama.

Teorema 5.2: Sifat Penambahan dan Pemotongan ID

Jika $(q, x, \alpha) \vdash^* (p, y, \beta)$, maka untuk string input tambahan w dan string stack tambahan γ berlaku

$$(q, xw, \alpha\gamma) \vdash^* (p, yw, \beta\gamma).$$

Sebaliknya, jika komputasi $(q, xw, \alpha) \vdash^* (p, yw, \beta)$ tidak pernah membaca ekor w , maka

$$(q, x, \alpha) \vdash^* (p, y, \beta).$$

Makna praktisnya adalah bagian bawah stack dapat dianggap sebagai konteks yang tetap selama transisi tidak mencapainya. Demikian pula, ekor input yang belum pernah dibaca tidak memengaruhi validitas langkah yang sudah terjadi.

Sifat ini berhubungan dengan pembuktian ekuivalensi PDA dan CFG. Pada konstruksi CFG ke PDA, pembuktian induktif memerlukan argumen bahwa sisa input dan tail sentential form dapat dibawa sepanjang komputasi tanpa mengubah legalitas langkah lokal.

5.2.8 Acceptance by Final State

Inti Konsep.

Acceptance by final state menerima string jika seluruh input habis dibaca dan PDA berada di salah satu state penerima. Isi stack akhir tidak memengaruhi penerimaan.

Untuk PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, bahasa yang diterima melalui final state adalah:

$$L(P) = \{w \mid (q_0, w, Z_0) \vdash^* (q, \epsilon, \alpha), q \in F\}.$$

Komponen α dapat berupa string stack apa pun. Artinya, selama input sudah habis dan state akhir tercapai, PDA menerima string tersebut walaupun stack masih berisi simbol.

Mode ini mirip finite automaton karena penerimaan ditentukan oleh state. Namun berbeda dari finite automaton, jalur menuju state akhir dapat bergantung pada isi stack selama komputasi.

Acceptance by final state sering nyaman untuk desain PDA yang memiliki state khusus “selesai”. Namun untuk pembuktian ekuivalensi dengan CFG, acceptance by empty stack sering lebih langsung karena stack dapat dipandang sebagai sentential form yang harus habis dicocokkan.

5.2.9 Acceptance by Empty Stack

Inti Konsep.

Acceptance by empty stack menerima string jika seluruh input habis dibaca dan stack menjadi kosong. State akhir tidak diperhatikan.

Bahasa yang diterima melalui empty stack adalah:

$$N(P) = \{w \mid (q_0, w, Z_0) \vdash^* (q, \epsilon, \epsilon)\}.$$

Pada definisi ini, state q boleh state apa pun. Syarat yang menentukan adalah input habis dan stack kosong.

Mode ini cocok dengan intuisi parsing. Jika stack menyimpan simbol grammar atau kewajiban yang masih harus dicocokkan, maka stack kosong berarti semua kewajiban derivasi sudah selesai dan semua input sudah cocok.

Perbedaan paling penting dengan final state adalah apa yang diabaikan. Final state mengabaikan isi stack akhir, sedangkan empty stack mengabaikan state akhir. Meskipun tampak berbeda, keduanya ekuivalen dalam kekuatan bahasa.

5.2.10 Ekuivalensi Mode Penerimaan

Inti Konsep.

PDA yang menerima dengan final state dan PDA yang menerima dengan empty stack memiliki kekuatan bahasa yang sama. Setiap PDA dari satu mode dapat dikonstruksi menjadi PDA mode lain dengan menambahkan marker stack dan state bantu.

Untuk mengubah empty stack ke final state, konstruksi menambahkan state awal baru p_0 , state final baru p_f , dan marker dasar baru X_0 . PDA baru mula-mula mendorong Z_0X_0 sehingga X_0 berada di dasar stack, lalu mensimulasikan PDA lama. Ketika simulasi lama berhasil mengosongkan stack aslinya dan marker dasar terlihat, PDA baru berpindah ke p_f .

Untuk mengubah final state ke empty stack, konstruksi menambahkan state awal baru, marker dasar baru, dan state pembersih. PDA baru mensimulasikan PDA lama. Dari setiap state penerima PDA lama, PDA baru boleh berpindah dengan transisi ϵ ke state pembersih. Di state pembersih, semua simbol stack di-pop sampai stack kosong.

Komentar 5.3: Catatan: Titik Rawan Pemahaman

Pada konstruksi final state ke empty stack, transisi menuju state pembersih harus dipakai setelah input yang relevan sudah dikonsumsi. Jika tidak hati-hati, PDA dapat mengosongkan stack terlalu awal. Dalam pembuktian formal, konstruksi diatur agar bahasa yang diterima tetap sama dengan bahasa PDA asal.

Ekuivalensi ini penting karena beberapa teorema lebih mudah dibuktikan dengan empty stack, sementara desain automaton sering lebih intuitif dengan final state.

5.2.11 Konversi CFG ke PDA

Inti Konsep.

Konversi CFG ke PDA membangun PDA yang mensimulasikan leftmost derivation grammar. Stack menyimpan simbol grammar yang masih harus dikembangkan atau dicocokkan dengan input.

Misalkan CFG $G = (V, T, R, S)$. PDA hasil konstruksi dapat ditulis secara ringkas sebagai:

$$P_G = (\{q\}, T, V \cup T, \delta, q, S),$$

dengan satu state q dan start stack symbol S . PDA menerima dengan empty stack.

Ada dua jenis transisi utama. Pertama, ****expand variable****:

$$\delta(q, \epsilon, A) = \{(q, \beta) \mid A \rightarrow \beta \in R\}.$$

Jika variabel A berada di puncak stack, PDA menebak produksi $A \rightarrow \beta$ dan mengganti A dengan β di stack.

Kedua, ****match terminal****:

$$\delta(q, a, a) = \{(q, \epsilon)\}$$

untuk setiap terminal $a \in T$. Jika puncak stack adalah terminal yang sama dengan input berikutnya, PDA membaca input tersebut dan menghapus terminal dari stack.

Konstruksi ini mengikuti leftmost derivation. Pada bentuk sentensial kiri $xA\alpha$, bagian x sudah dicocokkan dengan input, sedangkan $A\alpha$ masih tersimpan di stack. Saat PDA mengekspansi A , ia meniru langkah derivasi $xA\alpha \Rightarrow_{lm} x\beta\alpha$.

5.2.12 Mengapa $N(P_G) = L(G)$

Inti Konsep.

Kesetaraan $N(P_G) = L(G)$ berarti string yang dihasilkan CFG persis sama dengan string yang dapat diterima PDA hasil konstruksi melalui empty stack. Intinya, setiap leftmost derivation dapat disimulasikan oleh stack PDA, dan setiap simulasi PDA yang berhasil merepresentasikan derivasi grammar yang sah.

Jika $w \in L(G)$, ada leftmost derivation

$$S = \gamma_1 \Rightarrow_{lm} \gamma_2 \Rightarrow_{lm} \cdots \Rightarrow_{lm} \gamma_n = w.$$

Pada setiap langkah, PDA menggunakan transisi expand variable untuk mengganti variabel paling kiri di stack sesuai produksi yang dipakai derivasi. Ketika simbol terminal muncul di puncak stack, PDA menggunakan transisi match terminal untuk mencocokkannya dengan input.

Sebaliknya, jika PDA menerima w dengan empty stack, setiap transisi expand variable yang dipilih PDA berkaitan dengan produksi grammar yang sah, dan setiap transisi match terminal memastikan terminal pada stack benar-benar sama dengan input. Karena stack akhirnya kosong dan input habis, seluruh rangkaian pilihan PDA membentuk derivasi grammar untuk w .

Kesetaraan ini adalah salah satu hubungan utama antara automata dan grammar: CFG adalah cara generatif untuk mendeskripsikan CFL, sedangkan PDA adalah cara operasional untuk mengenali CFL.

Komentar 5.4: Contoh Soal UAS: CFG ke PDA

Salah satu pola latihan yang sering muncul adalah mengubah grammar menjadi PDA empty stack. Untuk grammar

$$S \rightarrow aAA, \quad A \rightarrow aS \mid bS \mid a,$$

konstruksi satu-state P_G memakai state q , start stack symbol S , dan acceptance by empty stack.

Transisi expand-variable:

$$\begin{aligned}\delta(q, \epsilon, S) &= \{(q, aAA)\}, \\ \delta(q, \epsilon, A) &= \{(q, aS), (q, bS), (q, a)\}.\end{aligned}$$

Transisi match-terminal:

$$\delta(q, a, a) = \{(q, \epsilon)\}, \quad \delta(q, b, b) = \{(q, \epsilon)\}.$$

Jadi daftar transisi lengkapnya terdiri dari tiga transisi ekspansi produksi dan dua transisi pencocokan terminal. PDA tidak membutuhkan state tambahan karena seluruh sentential form yang belum diproses disimpan pada stack.

Contoh verifikasi untuk string aaa :

$$\begin{aligned}(q, aaa, S) &\vdash (q, aaa, aAA) \\ &\vdash (q, aa, AA) \\ &\vdash (q, aa, aA) \\ &\vdash (q, a, A) \\ &\vdash (q, a, a) \\ &\vdash (q, \epsilon, \epsilon).\end{aligned}$$

Rantai ID ini sesuai derivasi leftmost $S \Rightarrow aAA \Rightarrow aaA \Rightarrow aaa$.

5.2.13 Konversi PDA ke CFG

Inti Konsep.

Konversi PDA ke CFG membangun grammar yang menghasilkan semua string yang dapat membuat PDA menghapus simbol stack tertentu dari satu state ke state lain. Variabel grammar berbentuk komposit $[pXq]$.

Misalkan PDA memiliki state dalam Q dan simbol stack dalam Γ . Untuk setiap $p, q \in Q$ dan $X \in \Gamma$, dibuat variabel komposit $[pXq]$. Maknanya adalah himpunan string yang dapat membawa PDA dari state p dengan X di puncak stack menuju state q setelah X terhapus sebagai efek bersih.

Aturan awal grammar adalah:

$$S \rightarrow [q_0Z_0p]$$

untuk setiap $p \in Q$. Produksi ini menyatakan bahwa string diterima jika dapat menghapus simbol stack awal Z_0 dari state awal q_0 menuju suatu state akhir proses pengosongan stack.

Untuk transisi $(r, Y_1Y_2 \cdots Y_k) \in \delta(q, a, X)$, grammar membuat produksi:

$$[qXr_k] \rightarrow a[rY_1r_1][r_1Y_2r_2] \cdots [r_{k-1}Y_kr_k],$$

untuk semua pilihan state perantara $r_1, \dots, r_k$. Jika $k = 0$, yaitu transisi langsung mem-pop X , produksinya menjadi:

$$[qXr] \rightarrow a.$$

Jika $a = \epsilon$, bagian terminal pada ruas kanan dapat diperlakukan sebagai kosong.

Konstruksi ini lebih sulit daripada CFG ke PDA karena grammar harus menebak semua state perantara tempat simbol-simbol hasil push akhirnya dihapus.

5.2.14 Pertumbuhan Ukuran Grammar pada Konversi PDA ke CFG

Inti Konsep.

Konversi PDA ke CFG dapat menghasilkan grammar yang sangat besar karena setiap transisi yang mendorong banyak simbol harus mempertimbangkan banyak kombinasi state perantara. Ukuran grammar dapat dikendalikan dengan membatasi jumlah simbol yang di-push per transisi.

Jumlah variabel komposit berformat $[pXq]$ bergantung pada kombinasi dua state dan satu simbol stack. Jika jumlah state n dan jumlah simbol stack juga diperhitungkan, jumlah variabel dapat tumbuh besar tetapi masih sistematis.

Masalah utama muncul ketika transisi mendorong string stack panjang $Y_1Y_2 \cdots Y_k$. Untuk membuat produksi grammar, konstruksi harus memilih state perantara $r_1, \dots, r_k$ dari himpunan state. Semakin besar k , semakin banyak produksi yang harus dibuat.

Sumber baru memperjelas ukuran pertumbuhan ini. Jika sebuah transisi PDA mem-push k simbol sekaligus, konstruksi grammar harus mempertimbangkan kombinasi state perantara, sehingga dapat menghasilkan orde $O(n^k)$ aturan produksi. Jika k ikut membesar mendekati jumlah state n , ukuran grammar dapat meledak menjadi $O(n^n)$.

Solusi konstruktifnya adalah memodifikasi PDA sebelum konversi. Untuk transisi yang mem-push string panjang, tambahkan maksimal $k - 2$ state bantu agar push panjang dipecah menjadi beberapa langkah pendek. Dengan menjaga setiap transisi hanya mem-push jumlah simbol terbatas, konstruksi CFG dapat dijaga tetap polinomial, dalam sumber disebut dapat ditekan ke orde $O(n^3)$ untuk bentuk yang sudah dinormalisasi.

Komentar 5.5: Contoh Soal UAS: PDA ke CFG

Untuk memahami konstruksi $[pXq]$, gunakan PDA empty stack sederhana untuk bahasa $\{0^n1^n \mid n \geq 1\}$ dengan state q_0 untuk fase push, q_1 untuk fase pop, start stack symbol Z_0 , dan simbol stack X . Transisi intinya:

$$\begin{aligned}\delta(q_0, 0, Z_0) &= \{(q_0, XZ_0)\}, \\ \delta(q_0, 0, X) &= \{(q_0, XX)\}, \\ \delta(q_0, 1, X) &= \{(q_1, \epsilon)\}, \\ \delta(q_1, 1, X) &= \{(q_1, \epsilon)\}, \\ \delta(q_1, \epsilon, Z_0) &= \{(q_1, \epsilon)\}.\end{aligned}$$

Variabel CFG yang dibangkitkan berbentuk $[pXq]$ untuk $p, q \in \{q_0, q_1\}$ dan $X \in \{Z_0, X\}$. Aturan start mengikuti pola umum:

$$S \rightarrow [q_0Z_0q_0] \mid [q_0Z_0q_1].$$

Karena penerimaan empty stack berakhir setelah Z_0 dihapus, produksi yang produktif biasanya menuju q_1 .

Untuk transisi pop langsung, dibuat produksi terminal:

$$[q_0Xq_1] \rightarrow 1, \quad [q_1Xq_1] \rightarrow 1, \quad [q_1Z_0q_1] \rightarrow \epsilon.$$

Untuk transisi push dua simbol $\delta(q_0, 0, Z_0) = (q_0, XZ_0)$, bentuk produksinya menebak state perantara r :

$$[q_0Z_0q] \rightarrow 0[q_0Xr][rZ_0q]$$

untuk setiap $r, q \in \{q_0, q_1\}$. Untuk transisi $\delta(q_0, 0, X) = (q_0, XX)$:

$$[q_0Xq] \rightarrow 0[q_0Xr][rXq].$$

Sumber NotebookLM juga mencatat pola pilihan ganda UAS 2425 yang perlu dikenali secara cepat. Jika PDA ditulis $P = (\{p, q\}, \{0, 1\}, \{X, Z\}, \delta, q, Z)$, maka aturan start memuat semua state tujuan:

Jika diberikan transisi push $\delta(p, 0, X) = \{(r, XX)\}$, produksi valid mengikuti bentuk pecah dua simbol:

Jika diberikan transisi pop $\delta(q, \epsilon, X) = \{(q, \epsilon)\}$, produksi langsungnya adalah:

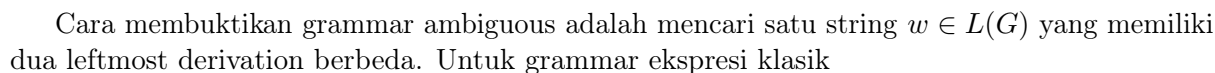
Pola ini adalah inti konversi PDA ke CFG: setiap simbol stack yang didorong harus diwakili oleh satu variabel komposit yang akhirnya menghapus simbol tersebut dari suatu state awal ke state akhir yang ditebak.

Inti Konsep.

Leftmost derivation selalu mengekspansi nonterminal paling kiri pada setiap langkah. Rightmost derivation selalu mengekspansi nonterminal paling kanan. Keduanya dapat menghasilkan string yang sama, tetapi urutan ekspansinya berbeda. Untuk grammar tidak ambiguous, parse tree yang dihasilkan tetap sama secara struktur.

$$S \rightarrow AB, \quad A \rightarrow 0A \mid 0, \quad B \rightarrow 1B \mid 1.$$
$$S \Rightarrow AB \Rightarrow 0AB \Rightarrow 00B \Rightarrow 001B \Rightarrow 0011.$$
$$S \Rightarrow AB \Rightarrow A1B \Rightarrow A11 \Rightarrow 0A11 \Rightarrow 0011.$$

Listing 1: Parse tree untuk 0011



13

string $id+id*id$ mempunyai dua struktur: $(id+id)*id$ dan $id+(id*id)$. Dua leftmost derivation-nya dapat dimulai sebagai $E \Rightarrow E * E \Rightarrow E + E * E \Rightarrow id + E * E \Rightarrow id + id * E \Rightarrow id + id * id$ atau $E \Rightarrow E + E \Rightarrow id + E \Rightarrow id + E * E \Rightarrow id + id * E \Rightarrow id + id * id$. Bukti ujian tidak cukup mengatakan “terlihat ambiguous”; tuliskan dua derivasi atau dua parse tree berbeda untuk string yang sama.

Subtopik ini terhubung langsung dengan konversi CFG ke PDA. PDA hasil konstruksi CFG meniru leftmost derivation, sehingga kemampuan menulis derivasi manual membantu membaca rangkaian ID PDA.

5.2.16 Deterministic Pushdown Automata

Inti Konsep.

DPDA adalah PDA yang tidak memiliki pilihan komputasi ganda pada konfigurasi yang sama. Determinisme ini membuat DPDA lebih mudah diimplementasikan sebagai parser deterministik, tetapi juga membuatnya lebih lemah daripada PDA nondeterministik penuh.

Sebuah DPDA membatasi fungsi transisi agar pada state, simbol input, dan puncak stack tertentu tidak ada lebih dari satu aksi yang mungkin. Selain itu, jika pada kombinasi (q, X) tersedia transisi ϵ , maka transisi yang membaca simbol input nyata dari konfigurasi yang sama tidak boleh bersaing dengannya. Jika tidak, automaton harus memilih antara membaca input sekarang atau bergerak spontan, yang melanggar determinisme.

Sumber memberikan perbandingan bahasa $L_{ww^R} = \{ww^R \mid w \in \{0,1\}^*\}$ dan $L_{wcw^R} = \{wcw^R \mid w \in \{0,1\}^*\}$. Bahasa pertama membutuhkan tebakan titik tengah karena tidak ada marker eksplisit. Bahasa kedua memiliki marker c , sehingga titik balik dari fase push ke fase pop dapat ditentukan secara deterministik.

DPDA menempati hierarki:

$$Regular \subset L(DPDA) \subset CFL.$$

Semua regular language dapat dikenali DPDA dengan mengabaikan stack dan meniru DFA. Namun tidak semua CFL dapat dikenali DPDA karena sebagian CFL memerlukan nondeterministic choice, misalnya untuk menebak titik tengah string. Sumber baru juga menegaskan bahwa bahasa yang diterima DPDA dengan final state merupakan subkelas sejati dari CFL dan setiap bahasa yang diterima DPDA memiliki CFG yang tidak ambigu.

5.2.17 Prefix Property dan Unambiguous CFG

Inti Konsep.

Prefix property menyatakan bahwa tidak ada string dalam bahasa yang menjadi prefix dari string lain dalam bahasa yang sama. Sifat ini penting untuk DPDA dengan acceptance by empty stack karena stack kosong memaksa keputusan penerimaan terjadi tepat saat input yang valid selesai.

Jika sebuah DPDA menerima dengan empty stack, begitu stack kosong dan input yang dibaca membentuk string valid, automaton tidak lagi memiliki informasi stack untuk melanjutkan pengenalan string yang lebih panjang dengan prefix yang sama. Karena itu bahasa yang diterima DPDA empty stack harus memiliki prefix property.

Sumber juga mengaitkan DPDA dengan unambiguous CFG. Intuisi dasarnya adalah determinisme automaton membatasi satu jalur komputasi penerimaan, sehingga ketika dikonversi ke grammar, bahasa yang dihasilkan memiliki struktur derivasi yang tidak ambigu untuk kelas yang sesuai. Namun hubungan ini perlu dibaca hati-hati: tidak semua unambiguous CFL deterministik, tetapi deterministic CFL merupakan subclass dari unambiguous CFL.

Komentar 5.6: Catatan: Sumber Pengetahuan Umum

Tambahan standar: deterministic context-free languages (DCFL) tertutup terhadap komplement, tetapi tidak tertutup terhadap union dan intersection umum. Ini berbeda dari CFL umum yang tidak tertutup terhadap komplement. Sifat ini berguna untuk memahami bahwa DPDA bukan sekadar PDA yang “lebih praktis”, melainkan mendefinisikan subclass bahasa dengan perilaku closure yang berbeda.

Konsep ini terhubung dengan parser deterministik seperti LL(1) dan LR: parser tersebut pada dasarnya menghindari tebakan nondeterministik dengan memastikan grammar memberi satu aksi parsing yang jelas untuk setiap konfigurasi.

5.2.18 Closure Properties CFL via PDA

Inti Konsep.

Beberapa sifat closure CFL dapat dibuktikan langsung memakai konstruksi PDA. Dua konstruksi penting adalah intersection antara CFL dan regular language melalui product PDA-DFA, serta inverse homomorphism dengan menerjemahkan setiap simbol input ke string asalnya.

Closure properties menjawab apakah hasil operasi terhadap bahasa masih berada dalam kelas yang sama. Untuk PDA, pertanyaan ini penting karena stack memberi memori tak terbatas yang terstruktur, tetapi tidak semua operasi bahasa aman untuk CFL.

Untuk intersection dengan regular language, misalkan L adalah CFL yang dikenali PDA P dan R adalah regular language yang dikenali DFA D . Bahasa $L \cap R$ dikenali oleh PDA baru yang mensimulasikan P dan D secara paralel:

1. State baru berbentuk pasangan (q_P, q_D) .
2. Jika PDA membaca simbol input nyata a , komponen DFA ikut berpindah dengan transisi pada a .
3. Jika PDA memakai transisi ϵ , komponen DFA tidak berubah.
4. String diterima jika simulasi PDA menerima dan komponen DFA berada pada state penerima.

Konstruksi ini berhasil karena DFA hanya menambah memori berhingga, sehingga dapat dimasukkan ke finite control PDA. Akibatnya, CFL tertutup terhadap intersection dengan regular language, walaupun CFL tidak tertutup terhadap intersection umum antara dua CFL.

Untuk inverse homomorphism, misalkan $h : \Sigma^* \rightarrow \Delta^*$ adalah homomorphism dan L adalah CFL atas alfabet Δ . Bahasa $h^{-1}(L) = \{w \mid h(w) \in L\}$ tetap CFL. PDA untuk $h^{-1}(L)$ membaca simbol a dari input asal, lalu mensimulasikan PDA untuk L seolah-olah menerima seluruh string $h(a)$.

Komentar 5.7: Catatan: Hubungan dengan Closure CFL

CFL tertutup terhadap union, concatenation, Kleene star, homomorphism, inverse homomorphism, dan intersection dengan regular language. Namun CFL tidak tertutup terhadap intersection umum dan complement umum. Product PDA-DFA menjelaskan mengapa intersection khusus dengan regular language masih aman: komponen regular hanya menambah state berhingga.

Closure via PDA menghubungkan materi PDA dengan hierarki bahasa formal. Jika soal meminta membuktikan bahasa tetap CFL setelah dibatasi pola regular, konstruksi product PDA-DFA adalah strategi utama.

5.3 Algoritma dan Rumus Penting

5.3.1 Fungsi Transisi PDA

Makna Rumus.

Rumus fungsi transisi menjelaskan domain informasi yang dilihat PDA dan bentuk pilihan aksi yang dapat diambil. Rumus ini digunakan saat mendefinisikan PDA secara formal atau menerjemahkan diagram transisi menjadi tuple.

$$\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow 2^{Q \times \Gamma^*} \quad (1)$$

dengan:

- Q : himpunan state.
- Σ : alfabet input.
- Γ : alfabet stack.
- ϵ : transisi tanpa membaca input.
- $2^{Q \times \Gamma^*}$: himpunan semua pilihan pasangan state baru dan string pengganti stack.

5.3.2 Langkah ID PDA

Makna Rumus.

Rumus langkah ID menyatakan efek satu transisi PDA terhadap sisa input dan stack. Rumus ini adalah dasar simulasi dan pembuktian penerimaan.

$$(q, aw, X\beta) \vdash (p, w, \alpha\beta) \quad \text{jika } (p, \alpha) \in \delta(q, a, X). \quad (2)$$

dengan:

- q : state sebelum transisi.
- p : state setelah transisi.
- a : simbol input yang dibaca, atau ϵ .
- w : sisa input setelah a .
- X : simbol puncak stack sebelum transisi.
- α : string pengganti untuk X .
- β : bagian stack di bawah X .

5.3.3 Mode Penerimaan PDA

Makna Rumus.

Dua rumus penerimaan membedakan apakah keberhasilan ditentukan oleh state akhir atau oleh stack kosong. Keduanya ekuivalen dalam kekuatan bahasa, tetapi sering berbeda kenyamanan saat desain atau pembuktian.

$$L(P) = \{w \mid (q_0, w, Z_0) \vdash^* (q, \epsilon, \alpha), q \in F\}, \quad (3)$$

$$N(P) = \{w \mid (q_0, w, Z_0) \vdash^* (q, \epsilon, \epsilon)\}. \quad (4)$$

dengan:

- $L(P)$: bahasa yang diterima dengan final state.
- $N(P)$: bahasa yang diterima dengan empty stack.
- α : isi stack akhir yang diabaikan pada final state.
- q : state akhir komputasi yang diabaikan pada empty stack.

5.3.4 Pseudocode Simulasi PDA Nondeterministik

Tujuan Algoritma.

Pseudocode ini menunjukkan cara berpikir operasional PDA: dari satu konfigurasi, coba semua transisi yang legal, lalu terima jika ada cabang yang mencapai kondisi penerimaan.

Listing 2: Pseudocode simulasi PDA nondeterministik

```

1 Input: PDA P, string w
2 Output: accept jika ada cabang komputasi yang menerima
3
4 1. Masukkan ID awal (q0, w, Z0) ke worklist.
5 2. Selama worklist tidak kosong:
6   a. Ambil satu ID (q, rest, stack).
7   b. Jika ID memenuhi mode penerimaan, return accept.
8   c. Untuk setiap transisi yang cocok dengan q, simbol input berikutnya,
9       dan top(stack):
10      i. Hitung sisa input baru.
11      ii. Hitung stack baru.
12      iii. Masukkan ID baru ke worklist.
13 3. Jika semua cabang habis tanpa menerima, return reject.
```

Pseudocode ini bukan algoritma keputusan umum tanpa batas tambahan, karena PDA nondeterministik dengan transisi ϵ dapat memiliki cabang yang berputar. Untuk keperluan teori bahasa, pseudocode ini terutama membantu memahami makna “ada jalur komputasi yang menerima”.

6 CYK

Algoritma CYK (Cocke-Younger-Kasami) adalah algoritma membership test untuk Context-Free Language (CFL). Tujuannya bukan menghasilkan grammar baru, melainkan memutuskan apakah sebuah string tertentu dapat diturunkan oleh sebuah CFG tertentu. Materi ini berdiri di atas konsep CFG, Chomsky Normal Form (CNF), parse tree biner, dan dynamic programming.

Untuk ujian, CYK perlu dikuasai sebagai prosedur tabel: mahasiswa harus dapat mengisi tabel dari substring panjang satu, menggabungkan sel melalui semua titik potong, membaca kondisi penerimaan, dan menjelaskan mengapa kompleksitasnya polinomial.

6.1 Outline Fundamental Konsep Materi

1. **WAJIB** Dasar membership problem CFL
 - (a) **WAJIB** Tujuan menguji $w \in L(G)$
 - (b) **PAHAM** Kelemahan enumerasi parse tree secara naif
 - (c) **WAJIB** CYK sebagai algoritma dynamic programming
 - (d) **WAJIB** Dependensi wajib terhadap Chomsky Normal Form
2. **WAJIB** Prasyarat CNF
 - (a) **WAJIB** Produksi bentuk $A \rightarrow BC$
 - (b) **WAJIB** Produksi terminal $A \rightarrow a$
 - (c) **PAHAM** Perlakuan start symbol dan string kosong
 - (d) **WAJIB** Clean-up grammar: nullable, unit pairs, generating, dan reachable symbols
 - (e) **WAJIB** Konversi CFG ke CNF sebelum CYK
 - (f) **PAHAM** Kompleksitas pertumbuhan grammar saat konversi
3. **WAJIB** Struktur tabel CYK
 - (a) **WAJIB** String input $w = a_1a_2 \cdots a_n$
 - (b) **WAJIB** Sel tabel X_{ij}
 - (c) **WAJIB** Makna $X_{ij} = \{A \mid A \Rightarrow_G^* a_i \cdots a_j\}$
 - (d) **PAHAM** Tabel segitiga dari panjang 1 sampai panjang n
4. **WAJIB** Prosedur algoritma CYK
 - (a) **WAJIB** Basis $X_{ii} = \{A \mid A \rightarrow a_i\}$
 - (b) **WAJIB** Induksi dengan semua split $i \leq k < j$
 - (c) **WAJIB** Tambahkan A jika ada $A \rightarrow BC$, $B \in X_{ik}$, dan $C \in X_{k+1,j}$
 - (d) **WAJIB** Acceptance condition $S \in X_{1n}$
5. **WAJIB** Kompleksitas
 - (a) **WAJIB** Jumlah sel $O(n^2)$
 - (b) **WAJIB** Split per sel sampai $O(n)$
 - (c) **WAJIB** Waktu $O(n^3)$ untuk grammar tetap
 - (d) **PAHAM** Ruang $O(n^2)$
 - (e) **PAHAM** Pengaruh ukuran grammar jika tidak dianggap konstan
6. **WAJIB** Contoh simulasi

- (a) **WAJIB** Grammar CNF contoh
 - (b) **WAJIB** Input baaba
 - (c) **WAJIB** Pengisian basis
 - (d) **WAJIB** Pengisian baris induksi
 - (e) **WAJIB** Pemeriksaan sel puncak
7. **PAHAM** Relasi teoretis
- (a) **PAHAM** CYK membuktikan membership CFL decidable
 - (b) **PAHAM** CFL termasuk recursive languages

6.2 Penjelasan Konsep

6.2.1 Membership Problem untuk CFL

Inti Konsep.

Membership problem menanyakan apakah sebuah string w termasuk dalam bahasa $L(G)$ yang dihasilkan oleh grammar G . Untuk CFL, CYK memberikan prosedur sistematis yang selalu berhenti dan menjawab ya atau tidak.

Dalam teori bahasa formal, pertanyaan “apakah $w \in L(G)$?” adalah pertanyaan keputusan. Inputnya adalah sebuah grammar bebas konteks G dan sebuah string w ; outputnya hanya dua kemungkinan: string diterima karena dapat diturunkan dari start symbol, atau ditolak karena tidak ada derivasi yang menghasilkan string tersebut.

Masalah ini penting karena parsing pada compiler pada dasarnya adalah membership testing terhadap grammar bahasa pemrograman. Jika token stream program berada dalam bahasa grammar, parser dapat membangun struktur sintaksis. Jika tidak, parser melaporkan syntax error.

CYK memecah masalah besar “apakah S menghasilkan seluruh string?” menjadi submasalah kecil “variabel apa saja yang menghasilkan substring tertentu?”. Submasalah ini disimpan dalam tabel agar hasil yang sama tidak dihitung berulang.

6.2.2 Kelemahan Pendekatan Naif

Inti Konsep.

Pendekatan naif mencoba semua kemungkinan parse tree biner untuk string sepanjang n . Jumlah kemungkinan pohon tumbuh eksponensial, sehingga cara ini tidak praktis untuk membership testing.

Jika grammar sudah berada dalam Chomsky Normal Form, parse tree untuk string terminal panjang n memiliki struktur biner: setiap node internal bercabang menjadi dua nonterminal, kecuali node yang langsung menghasilkan terminal. Struktur biner ini membantu analisis, tetapi tidak berarti semua pohon mudah dicoba satu per satu.

Pendekatan naif akan membangkitkan seluruh kemungkinan bentuk parse tree dan seluruh kemungkinan label variabel pada node internal, lalu memeriksa apakah salah satunya menghasilkan w . Banyak subtree yang sama dapat muncul berulang dalam percobaan berbeda. Inilah redundansi yang dihindari oleh dynamic programming CYK.

Sumber menyebut bahwa cara naif bersifat eksponensial terhadap panjang string. CYK memperbaikinya dengan menyimpan hasil setiap substring pada satu sel tabel, sehingga substring yang sama tidak perlu dianalisis ulang.

6.2.3 CYK sebagai Dynamic Programming

Inti Konsep.

CYK adalah algoritma dynamic programming yang mengisi tabel dari substring pendek menuju substring panjang. Setiap sel menyimpan semua nonterminal yang dapat menurunkan substring tertentu.

Dynamic programming cocok untuk CYK karena derivasi CNF memiliki sifat optimal substructure. Jika sebuah variabel A menurunkan substring panjang melalui produksi $A \rightarrow BC$, maka substring tersebut pasti dapat dipotong menjadi bagian kiri yang diturunkan oleh B dan bagian kanan yang diturunkan oleh C .

Dengan kata lain, keputusan untuk substring besar bergantung pada keputusan substring yang lebih kecil. CYK mengurutkan pengisian tabel berdasarkan panjang substring: panjang 1 terlebih dahulu, lalu panjang 2, dan seterusnya sampai panjang n .

Nama CYK berasal dari Cocke, Younger, dan Kasami, yang mengembangkan pendekatan ini secara independen. Dalam konteks mata kuliah, nama algoritma tidak sepenting kemampuan menjalankan recurrence tabelnya dengan benar.

6.2.4 Dependensi terhadap Chomsky Normal Form

Inti Konsep.

CYK membutuhkan grammar dalam Chomsky Normal Form karena recurrence-nya mengandalkan fakta bahwa produksi nonterminal berbentuk biner $A \rightarrow BC$ atau terminal tunggal $A \rightarrow a$. Tanpa bentuk ini, pembagian substring menjadi dua bagian tidak langsung sesuai dengan produksi grammar.

CNF membatasi bentuk produksi menjadi dua pola utama. Produksi $A \rightarrow a$ dipakai untuk mengisi sel basis karena substring panjang satu hanya berisi satu terminal. Produksi $A \rightarrow BC$ dipakai pada langkah induksi karena substring panjang lebih dari satu dapat dibelah menjadi dua bagian.

Bentuk Produksi	Peran dalam CYK
$A \rightarrow a$	Mengisi sel basis X_{ii} untuk terminal a_i .
$A \rightarrow BC$	Mengisi sel induksi X_{ij} dari kombinasi X_{ik} dan $X_{k+1,j}$.

Tabel 6: Peran bentuk produksi CNF dalam CYK

Komentar 6.1: Catatan: Sumber Pengetahuan Umum

Jika $\epsilon \in L(G)$, beberapa definisi CNF mengizinkan produksi khusus $S \rightarrow \epsilon$, dengan syarat start symbol tidak muncul di ruas kanan produksi lain. Untuk CYK pada string kosong, kasus ini biasanya ditangani terpisah sebelum tabel CYK biasa dijalankan.

Dependensi terhadap CNF menghubungkan materi CYK dengan materi penyederhanaan CFG dan konversi CNF. Sebelum CYK dapat dijalankan, grammar harus dibersihkan dari produksi ϵ , unit production, dan useless symbols, lalu dibentuk menjadi produksi terminal tunggal dan produksi biner.

6.2.5 Clean-Up Grammar Menuju CNF

Inti Konsep.

Clean-up grammar adalah rangkaian transformasi yang menyiapkan CFG agar dapat diubah ke Chomsky Normal Form tanpa mengubah bahasa pentingnya. Langkah ini melibatkan nullable symbols, unit pairs, generating symbols, dan reachable symbols.

CYK tidak dapat langsung dijalankan pada CFG arbitrer. Grammar perlu dibersihkan agar setiap produksi akhirnya cocok dengan pola CNF. Clean-up juga mencegah grammar memuat simbol yang tidak pernah menghasilkan terminal, tidak pernah tercapai dari start symbol, atau hanya menjadi rantai unit yang tidak membantu struktur parse tree biner.

Konsep	Peran dalam Konversi CNF
Nullable symbol	Nonterminal yang dapat menurunkan ϵ . Dipakai untuk menghapus produksi ϵ dengan menambahkan alternatif yang melewati simbol nullable.
Unit pair	Pasangan (A, B) dengan $A \Rightarrow^* B$ hanya melalui unit productions. Dipakai untuk mengganti rantai $A \rightarrow B$ dengan produksi non-unit milik B .
Generating symbol	Simbol yang dapat menurunkan string terminal. Simbol non-generating dihapus karena tidak dapat berkontribusi pada string hasil.
Reachable symbol	Simbol yang dapat muncul dari start symbol. Simbol unreachable dihapus karena tidak pernah dipakai dalam derivasi dari grammar.

Tabel 7: Konsep clean-up grammar menuju CNF

Urutan praktis yang aman untuk konversi CNF:

1. Buat start symbol baru $S_0 \rightarrow S$ agar start symbol asli tidak muncul di ruas kanan secara bermasalah.
2. Eliminasi produksi ϵ , kecuali kemungkinan khusus $S_0 \rightarrow \epsilon$ jika bahasa memang memuat string kosong.
3. Eliminasi unit productions dengan menghitung unit pairs.
4. Hapus useless symbols: pertama non-generating symbols, lalu unreachable symbols.
5. Ganti terminal pada produksi panjang dengan nonterminal bantu, misalnya $A \rightarrow aBC$ menjadi $A \rightarrow T_aBC$ dan $T_a \rightarrow a$.
6. Pecah produksi dengan ruas kanan lebih dari dua simbol menjadi produksi biner berantai.

Komentar 6.2: Catatan: Kompleksitas Konversi CNF

Konversi CNF dapat memperbesar ukuran grammar karena eliminasi ϵ menambahkan kombinasi produksi baru dan binarisasi menambah nonterminal bantu. Untuk soal manual, yang penting bukan menghitung batas kompleksitas formal secara rinci, melainkan menjaga bahasa tidak berubah dan memastikan hasil akhir hanya memiliki bentuk $A \rightarrow BC$, $A \rightarrow a$, dan mungkin $S_0 \rightarrow \epsilon$.

Tahap Konversi	Kompleksitas
Generating dan reachable symbol cleanup	$O(G)$, linear terhadap ukuran grammar karena setiap produksi dan simbol cukup dipindai sampai himpunan stabil.
Unit-pair computation	$O(n^2)$ terhadap jumlah variabel n , karena pasangan unit (A, B) antarnonterminal harus dihitung sebagai closure.
Ukuran grammar hasil CNF	$O(n^2)$ dalam kasus umum setelah unit pair dan binarisasi dikendalikan.

Tabel 8: Kompleksitas tahap penting konversi CNF

Blow-up $O(n^2)$ terutama muncul ketika eliminasi nullable symbols menambahkan variasi produksi dan ketika unit pairs menggandakan produksi non-unit milik variabel tujuan. Karena itu, pada pengerjaan ujian, jangan hanya menulis “ubah ke CNF”; tuliskan tahap clean-up yang dipakai agar grammar hasil tetap terkontrol.

Clean-up ini terhubung langsung dengan CYK: kesalahan kecil pada CNF akan membuat tabel X_{ij} salah, karena recurrence CYK hanya melihat produksi terminal tunggal dan produksi biner.

6.2.6 Struktur Tabel X_{ij}

Inti Konsep.

Sel X_{ij} berisi himpunan nonterminal yang dapat menurunkan substring dari posisi i sampai j . Tabel CYK berbentuk segitiga karena hanya substring dengan $i \leq j$ yang dihitung.

Misalkan input adalah

$$w = a_1 a_2 \cdots a_n.$$

Untuk setiap rentang posisi i sampai j , CYK mendefinisikan:

$$X_{ij} = \{A \mid A \Rightarrow_G^* a_i a_{i+1} \cdots a_j\}.$$

Artinya, X_{ij} bukan berisi string, melainkan variabel grammar yang mampu menghasilkan substring tersebut.

Tabel diisi berdasarkan panjang substring. Diagonal bawah berisi substring panjang 1: $X_{11}, X_{22}, \dots, X_{nn}$. Di atasnya adalah substring panjang 2: $X_{12}, X_{23}, \dots$. Puncak tabel adalah X_{1n} , yaitu sel untuk seluruh string.

Cara membaca tabel ini sangat penting: jika suatu nonterminal ada di sebuah sel, itu adalah bukti bahwa nonterminal tersebut dapat menghasilkan substring pada rentang sel itu. Jika tidak ada, berarti sejauh grammar CNF yang diberikan, nonterminal itu tidak dapat menghasilkan substring tersebut.

6.2.7 Langkah Basis

Inti Konsep.

Langkah basis mengisi sel untuk substring panjang satu. Sebuah nonterminal A masuk ke X_{ii} jika grammar memiliki produksi langsung $A \rightarrow a_i$.

Rumus basis CYK adalah:

$$X_{ii} = \{A \mid A \rightarrow a_i \in R\}.$$

Langkah ini memakai produksi terminal CNF. Karena substring panjang satu hanya terdiri atas satu terminal, tidak ada titik potong yang perlu dicoba.

Contoh: jika terminal pada posisi i adalah a dan grammar memiliki produksi $A \rightarrow a$ serta $C \rightarrow a$, maka $X_{ii} = \{A, C\}$. Jika hanya ada $B \rightarrow b$ untuk terminal b , maka sel untuk posisi berisi b adalah $\{B\}$.

Langkah basis adalah fondasi tabel. Kesalahan pada baris ini akan menyebar ke seluruh sel di atasnya, karena setiap sel induksi bergantung pada sel yang lebih pendek.

6.2.8 Langkah Induksi

Inti Konsep.

Langkah induksi mengisi sel substring panjang dua atau lebih dengan mencoba semua titik potong. Nonterminal A dimasukkan ke X_{ij} jika ada produksi $A \rightarrow BC$ yang cocok dengan satu pembagian substring.

Untuk menghitung X_{ij} dengan $i < j$, CYK mencoba semua k dengan $i \leq k < j$. Titik k membelah substring menjadi:

$$a_i \cdots a_k \quad \text{dan} \quad a_{k+1} \cdots a_j.$$

Jika ada produksi $A \rightarrow BC$, dengan $B \in X_{ik}$ dan $C \in X_{k+1,j}$, maka A masuk ke X_{ij} .

Secara ringkas:

$$X_{ij} = \{A \mid \exists k, i \leq k < j, \exists A \rightarrow BC, B \in X_{ik}, C \in X_{k+1,j}\}.$$

Intuisi langkah ini mengikuti parse tree biner. Jika A menghasilkan substring panjang, anak kiri B menghasilkan bagian kiri substring dan anak kanan C menghasilkan bagian kanan substring. Semua kemungkinan titik potong harus dicoba karena grammar tidak memberi tahu posisi split yang benar sejak awal.

6.2.9 Acceptance Condition

Inti Konsep.

String w diterima oleh grammar G jika start symbol S berada di sel puncak X_{1n} . Sel ini merepresentasikan seluruh input dari posisi pertama sampai posisi terakhir.

Setelah semua sel terisi, CYK tidak perlu mencari parse tree secara eksplisit untuk menjawab membership. Cukup periksa apakah:

$$S \in X_{1n}.$$

Jika ya, ada derivasi $S \Rightarrow_G^* w$, sehingga $w \in L(G)$. Jika tidak, tidak ada derivasi dari start symbol yang menghasilkan seluruh string.

Penting untuk tidak keliru membaca sel lain. Misalnya S muncul di X_{12} hanya berarti start symbol dapat menghasilkan substring posisi 1 sampai 2, bukan seluruh string. Untuk membership seluruh input, hanya X_{1n} yang menentukan jawaban akhir.

6.2.10 Kompleksitas CYK

Inti Konsep.

Untuk grammar tetap, CYK memakai ruang $O(n^2)$ dan waktu $O(n^3)$ terhadap panjang string n . Kompleksitas kubik muncul dari $O(n^2)$ sel dan sampai $O(n)$ titik potong per sel.

Jumlah substring dari string panjang n adalah $O(n^2)$, sehingga tabel CYK memiliki $O(n^2)$ sel. Setiap sel menyimpan himpunan nonterminal. Jika grammar dianggap tetap, ukuran himpunan nonterminal dan jumlah produksi dianggap konstan terhadap n .

Untuk setiap sel X_{ij} , algoritma mencoba semua titik potong k . Banyaknya titik potong paling banyak $n - 1$, sehingga biaya per sel adalah $O(n)$ untuk grammar tetap. Maka total waktu:

$$O(n^2) \cdot O(n) = O(n^3).$$

Ruang yang dibutuhkan adalah $O(n^2)$ sel tabel.

Komentar 6.3: Catatan: Sumber Pengetahuan Umum

Jika ukuran grammar tidak dianggap konstan, waktu juga dipengaruhi oleh jumlah produksi dan jumlah nonterminal. Implementasi praktis biasanya mengindeks produksi berbentuk $A \rightarrow BC$ berdasarkan pasangan (B, C) agar pencarian parent A lebih cepat.

Kompleksitas ini menjelaskan mengapa CYK jauh lebih baik daripada enumerasi parse tree eksponensial. Walaupun $O(n^3)$ tidak selalu tercepat untuk parser produksi nyata, CYK sangat penting secara teoretis karena sederhana dan membuktikan decidability membership CFL.

6.2.11 Contoh Simulasi dengan baaba

Inti Konsep.

Contoh baaba menunjukkan cara CYK mengisi tabel dari terminal individual menuju seluruh string. Karena S muncul di sel puncak X_{15} , string tersebut diterima oleh grammar.

Gunakan grammar CNF:

$$\begin{aligned} S &\rightarrow AB \mid BC, \\ A &\rightarrow BA \mid a, \\ B &\rightarrow CC \mid b, \\ C &\rightarrow AB \mid a. \end{aligned}$$

Input adalah $w = \text{baaba}$, sehingga $n = 5$.

Langkah basis:

$$\begin{aligned} X_{11} &= \{B\} && \text{untuk } b, \\ X_{22} &= \{A, C\} && \text{untuk } a, \\ X_{33} &= \{A, C\} && \text{untuk } a, \\ X_{44} &= \{B\} && \text{untuk } b, \\ X_{55} &= \{A, C\} && \text{untuk } a. \end{aligned}$$

Contoh pengisian induksi untuk X_{12} , substring ba : pasangan dari X_{11} dan X_{22} adalah B dengan A atau C , sehingga pasangan yang mungkin adalah BA dan BC . Grammar memiliki $A \rightarrow BA$ dan $S \rightarrow BC$, maka:

$$X_{12} = \{S, A\}.$$

Sumber menyebut bahwa setelah seluruh tabel dihitung, sel puncak untuk rentang seluruh string adalah:

$$X_{15} = \{S, A, C\}.$$

Karena $S \in X_{15}$, string baaba diterima.

Komentar 6.4: Catatan: Strategi Ujian

Saat mengerjakan soal CYK, jangan hanya menulis hasil akhir. Tunjukkan baris basis, minimal beberapa sel induksi penting, dan alasan penerimaan dari $S \in X_{1n}$. Untuk jawaban manual, tabel yang rapi biasanya lebih bernilai daripada paragraf panjang.

6.2.12 CYK dan Decidability CFL

Inti Konsep.

CYK membuktikan bahwa membership problem untuk CFL decidable karena algoritma ini selalu berhenti dan memberi jawaban benar untuk setiap string input. Dengan demikian, setiap CFL adalah recursive language untuk masalah keanggotaan stringnya.

Sebuah masalah disebut decidable jika ada algoritma yang selalu berhenti dengan jawaban benar. CYK memenuhi syarat ini untuk pertanyaan apakah $w \in L(G)$, selama grammar dapat dipersiapkan ke CNF dan string input berhingga.

Relasi ini penting dalam hierarki bahasa. Regular language memiliki membership test lebih sederhana melalui DFA. CFL lebih kompleks, tetapi tetap decidable melalui algoritma seperti CYK. Ini berbeda dari kelas bahasa yang dikenali Mesin Turing secara umum, di mana membership dapat menjadi undecidable jika mesin tidak selalu halt.

Dengan demikian, CYK tidak hanya algoritma parsing, tetapi juga bukti konstruktif bahwa CFL berada dalam kelas bahasa yang membership-nya dapat diputuskan secara algoritmik.

6.3 Algoritma dan Rumus Penting

6.3.1 Definisi Sel X_{ij}

Makna Rumus.

Rumus ini mendefinisikan isi setiap sel tabel CYK. Sel menyimpan nonterminal yang dapat menghasilkan substring tertentu, bukan menyimpan substring itu sendiri.

$$X_{ij} = \{A \mid A \Rightarrow_G^* a_i a_{i+1} \cdots a_j\} \quad (5)$$

dengan:

- X_{ij} : sel tabel untuk substring dari indeks i sampai j .
- A : nonterminal grammar.
- $\Rightarrow_G^*$: derivasi nol atau banyak langkah dalam grammar G .
- $a_i \cdots a_j$: substring input yang sedang dianalisis.

6.3.2 Basis dan Induksi CYK

Makna Rumus.

Rumus basis mengisi terminal individual, sedangkan rumus induksi menggabungkan dua substring lebih pendek melalui produksi biner CNF.

$$X_{ii} = \{A \mid A \rightarrow a_i \in R\}, \quad (6)$$

$$X_{ij} = \{A \mid \exists k, i \leq k < j, \exists A \rightarrow BC \in R, B \in X_{ik}, C \in X_{k+1,j}\}. \quad (7)$$

dengan:

- R : himpunan produksi grammar.
- k : titik potong substring.
- B dan C : nonterminal anak kiri dan kanan dalam produksi CNF.

6.3.3 Pseudocode CYK

Tujuan Algoritma.

Pseudocode ini merangkum urutan pengisian tabel CYK dari substring panjang satu sampai panjang n , lalu memeriksa apakah start symbol muncul pada sel puncak.

Listing 3: Pseudocode algoritma CYK

```
1 Input: CFG  $G=(V,T,R,S)$  dalam CNF, string  $w=a_1 \dots a_n$ 
2 Output: accept jika  $w$  berada dalam  $L(G)$ 
3
4 1. Untuk  $i = 1$  sampai  $n$ :
5      $X[i,i] = \{ A \mid \text{produksi } A \rightarrow a_i \text{ ada di } R \}$ 
6
7 2. Untuk panjang = 2 sampai  $n$ :
8     Untuk  $i = 1$  sampai  $n - \text{panjang} + 1$ :
9          $j = i + \text{panjang} - 1$ 
10         $X[i,j] = \{\}$ 
11        Untuk  $k = i$  sampai  $j - 1$ :
12            Untuk setiap produksi  $A \rightarrow B C$  di  $R$ :
13                Jika  $B$  ada di  $X[i,k]$  dan  $C$  ada di  $X[k+1,j]$ :
14                    Tambahkan  $A$  ke  $X[i,j]$ 
15
16 3. Jika  $S$  ada di  $X[1,n]$ , return accept.
17 4. Jika tidak, return reject.
```

Kondisi keberhasilan algoritma adalah $S \in X_{1n}$. Jika string kosong diuji, perlakuannya bergantung pada apakah grammar mengizinkan produksi start khusus $S \rightarrow \epsilon$.

7 Mesin Turing

Mesin Turing adalah model matematika untuk komputasi umum. Jika finite automaton hanya memiliki memori berhingga dan PDA memiliki stack, Mesin Turing memiliki pita baca-tulis tak terbatas yang membuatnya cukup kuat untuk memodelkan algoritma umum. Materi ini penting karena menghubungkan teori automata dengan pertanyaan batas komputasi: persoalan mana yang dapat diselesaikan algoritma, mana yang hanya dapat dikenali, dan mana yang tidak dapat diputuskan sama sekali.

Fokus belajar untuk ujian adalah definisi formal Mesin Turing, cara membaca instantaneous description, perbedaan bahasa recursive dan recursively enumerable, ekuivalensi beberapa varian Mesin Turing, serta teknik pembuktian undecidability seperti diagonalization, universal language, reduction, dan Rice's Theorem.

7.1 Outline Fundamental Konsep Materi

1. **WAJIB** Motivasi dan batas komputasi
 - (a) **WAJIB** Masalah yang dapat dan tidak dapat diselesaikan algoritma
 - (b) **PAHAM** Argumen paradoks program penguji
 - (c) **WAJIB** Church-Turing Thesis
2. **WAJIB** Definisi formal Mesin Turing
 - (a) **WAJIB** 7-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$
 - (b) **WAJIB** Fungsi transisi $\delta(q, Z) = (p, Y, D)$
 - (c) **PAHAM** Diagram transisi X/YD
3. **WAJIB** Instantaneous Description
 - (a) **WAJIB** Format $\alpha q \beta$
 - (b) **WAJIB** Aturan gerak kanan
 - (c) **WAJIB** Aturan gerak kiri
 - (d) **WAJIB** Relasi $\vdash$ dan $\vdash^*$
 - (e) **WAJIB** Trace eksekusi pada input konkret
4. **WAJIB** Penerimaan dan halting
 - (a) **WAJIB** Acceptance by final state $L(M)$
 - (b) **WAJIB** Acceptance by halting $H(M)$
 - (c) **PAHAM** Mesin Turing sebagai transducer
 - (d) **PAHAM** Mesin Turing sebagai enumerator
 - (e) **WAJIB** Recursively enumerable language
 - (f) **WAJIB** Recursive language
 - (g) **WAJIB** Teorema komplemen RE
 - (h) **PAHAM** Recognizer dan decider
5. **PAHAM** Teknik pemrograman Mesin Turing
 - (a) **PAHAM** Storage in state
 - (b) **PAHAM** Multiple tracks
 - (c) **PAHAM** Marker dan shifting-over

- (d) **PAHAM** Subroutines
- 6. **PAHAM** Varian dan ekuivalensi Mesin Turing
 - (a) **PAHAM** Multitape TM
 - (b) **PAHAM** Nondeterministic TM
 - (c) **PAHAM** Multi-head TM dan off-line TM
 - (d) **PAHAM** Semi-infinite tape
 - (e) **PAHAM** Two-stack dan counter machine
- 7. **PAHAM** Hubungan dengan komputer riil
 - (a) **PAHAM** Komputer mensimulasikan Mesin Turing
 - (b) **PAHAM** Mesin Turing mensimulasikan komputer
- 8. **WAJIB** Undecidability dan reduksi
 - (a) **WAJIB** Encoding Mesin Turing
 - (b) **WAJIB** Diagonalization language
 - (c) **WAJIB** Universal language
 - (d) **WAJIB** Reduction
 - (e) **PAHAM** Post Correspondence Problem
 - (f) **WAJIB** Rice's Theorem

7.2 Penjelasan Konsep

7.2.1 Motivasi Batas Komputasi

Inti Konsep.

Mesin Turing dipakai untuk memformalkan konsep algoritma dan membuktikan bahwa ada masalah yang tidak dapat diselesaikan oleh algoritma apa pun. Model ini cukup sederhana untuk dianalisis, tetapi cukup kuat untuk mewakili komputasi umum.

Finite automaton dan PDA menjawab pertanyaan tentang kelas bahasa tertentu, tetapi belum cukup untuk membahas semua program komputer. Mesin Turing memperluas model automata dengan pita baca-tulis tak terbatas, sehingga dapat menyimpan dan memodifikasi informasi selama komputasi berlangsung.

Motivasi utamanya bukan hanya membuat mesin yang lebih kuat, melainkan menentukan batas komputasi. Sumber memberi contoh program penguji “hello, world”. Andaikan ada program H yang dapat menentukan apakah program lain akan mencetak “hello, world”. Program ini dapat dimodifikasi menjadi program paradoks H_2 yang mencetak “hello, world” justru ketika inputnya tidak mencetaknya, dan tidak mencetak ketika inputnya mencetak. Ketika H_2 dijalankan pada dirinya sendiri, timbul kontradiksi. Argumen seperti ini menunjukkan bahwa ada properti program yang mustahil diputuskan secara universal.

Konsep ini menjadi jembatan dari automata ke computability theory. Pada bagian sebelumnya, pertanyaan membership untuk CFL decidable melalui CYK. Pada Mesin Turing, kita mulai melihat bahwa tidak semua pertanyaan tentang program atau bahasa dapat diputuskan.

7.2.2 Church-Turing Thesis

Inti Konsep.

Church-Turing Thesis menyatakan bahwa setiap komputasi efektif yang dapat dilakukan oleh metode algoritmik apa pun dapat dilakukan oleh Mesin Turing. Ini bukan teorema matematis formal, melainkan tesis yang diterima luas tentang makna komputasi efektif.

A.M. Turing memperkenalkan Mesin Turing pada tahun 1936 sebagai model untuk menangkap gagasan “segala kemungkinan komputasi”. Tesis ini tidak dapat dibuktikan seperti teorema biasa karena frasa “metode algoritmik apa pun” bersifat informal. Namun banyak model komputasi yang tampak berbeda, seperti lambda calculus, recursive functions, dan mesin register, ternyata ekuivalen dalam daya komputasinya dengan Mesin Turing.

Makna praktisnya adalah jika sebuah persoalan terbukti tidak dapat diputuskan oleh Mesin Turing, maka persoalan itu dianggap tidak dapat diselesaikan oleh program komputer biasa. Karena itu, Mesin Turing menjadi model standar untuk membahas decidability dan undecidability.

Komentar 7.1: Catatan: Sumber Pengetahuan Umum

Versi modern yang sering dipakai dalam analisis kompleksitas adalah Extended Church-Turing Thesis, yang secara kasar menyatakan bahwa model komputasi realistis dapat disimulasikan oleh Mesin Turing dengan overhead polinomial. Ini berbeda dari Church-Turing Thesis dasar, yang hanya membahas apa yang dapat dikomputasi, bukan efisiensinya.

7.2.3 Definisi Formal 7-Tuple

Inti Konsep.

Mesin Turing didefinisikan sebagai $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$. Tuple ini menentukan kendali berhingga, alfabet input, alfabet pita, fungsi transisi, state awal, simbol blank, dan state penerima.

Definisi formal diperlukan agar perhitungan Mesin Turing dapat dinyatakan tanpa ambiguitas. Mesin memiliki **finite control** berupa state, tetapi memori utamanya adalah pita yang secara konseptual tak terbatas.

Komponen	Definisi atau Peran
Q	Himpunan berhingga state.
Σ	Alfabet input yang muncul pada pita saat awal, tidak memuat blank.
Γ	Alfabet pita, mencakup Σ dan simbol tambahan yang boleh ditulis mesin.
δ	Fungsi transisi yang menentukan state baru, simbol tulis, dan arah gerak head.
q_0	State awal.
B	Simbol blank, anggota Γ tetapi bukan anggota Σ .
F	Himpunan final states atau accepting states.

Tabel 9: Komponen formal Mesin Turing

Input awal diletakkan pada pita, dan semua sel di luar input berisi blank. Head membaca satu sel pada satu waktu, menulis simbol baru, lalu bergerak satu sel ke kiri atau ke kanan.

7.2.4 Fungsi Transisi

Inti Konsep.

Fungsi transisi Mesin Turing menentukan aksi berdasarkan state saat ini dan simbol pita yang sedang dibaca. Dalam satu langkah, mesin menulis simbol baru, berpindah state, dan menggerakkan head.

Bentuk transisi dasar adalah:

$$\delta(q, Z) = (p, Y, D).$$

Artinya, jika mesin berada di state q dan head membaca simbol Z , mesin akan masuk ke state p , menulis simbol Y menggantikan Z , lalu bergerak ke arah D . Pada model sumber, D adalah L untuk kiri atau R untuk kanan; head bergerak tepat satu kotak setiap langkah.

Diagram transisi menuliskan panah dari state q ke state p dengan label Z/YD . Label ini berarti “baca Z , tulis Y , bergerak ke arah D ”. Diagram memudahkan desain mesin, sedangkan fungsi δ memudahkan pembuktian formal.

Jika $\delta(q, Z)$ tidak terdefinisi, mesin tidak dapat bergerak dari konfigurasi itu. Kondisi ini menjadi dasar acceptance by halting.

7.2.5 Instantaneous Description $\alpha q \beta$

Inti Konsep.

Instantaneous Description (ID) Mesin Turing menulis konfigurasi sebagai $\alpha q \beta$. State q disisipkan tepat sebelum simbol yang sedang dibaca head, sehingga isi pita dan posisi head dapat direpresentasikan dalam satu string.

Karena pita tak terbatas, ID tidak menuliskan seluruh blank tak terbatas. ID hanya menuliskan bagian relevan di antara simbol nonblank paling kiri dan paling kanan, ditambah state saat ini. Pada ID $\alpha q \beta$, string $\alpha \beta$ adalah isi pita relevan, dan simbol pertama dari β adalah simbol yang sedang dipindai.

Contoh: ID $abqcd$ berarti isi pita relevan adalah $abcd$, mesin sedang berada di state q , dan head sedang membaca simbol c . Jika β kosong, head berada di blank tepat di kanan isi nonblank yang ditulis.

ID adalah alat utama untuk menulis penerimaan formal, karena bahasa $L(M)$ dan $H(M)$ didefinisikan sebagai keberadaan rangkaian ID dari konfigurasi awal menuju konfigurasi penerima atau berhenti.

7.2.6 Aturan Gerak Kanan dan Kiri

Inti Konsep.

Aturan gerak ID menjelaskan bagaimana satu transisi mengubah isi pita, posisi state dalam string ID, dan arah head. Gerak kanan memindahkan state ke kanan simbol yang baru ditulis, sedangkan gerak kiri memindahkan state ke sebelum simbol di kiri.

Jika $\delta(q, Z) = (p, Y, R)$, maka:

$$\alpha q Z \beta \vdash \alpha Y p \beta.$$

Simbol Z diganti menjadi Y , lalu head bergerak ke kanan, sehingga state p ditulis setelah Y .

Jika $\delta(q, Z) = (p, Y, L)$ dan ada simbol X di kiri head, maka:

$$\alpha X q Z \beta \vdash \alpha p X Y \beta.$$

State p berpindah ke sebelum X , karena setelah bergerak kiri head membaca X .

Jika head bergerak ke kiri dari posisi paling kiri yang sedang ditulis, sumber menyatakan bentuk:

$$qZ\beta \vdash pBY\beta.$$

Blank B muncul sebagai sel baru di kiri.

Relasi $\vdash$ menyatakan satu langkah, sedangkan $\vdash^*$ menyatakan nol, satu, atau banyak langkah. Sama seperti pada PDA, notasi ini memungkinkan pembuktian dengan rantai konfigurasi.

7.2.7 Trace Eksekusi untuk 0^n1^n

Inti Konsep.

Trace eksekusi menunjukkan perubahan konfigurasi Mesin Turing secara konkret dari input awal sampai accept atau reject. Untuk bahasa 0^n1^n , pola desain yang umum adalah menandai satu 0 paling kiri yang belum dipakai, mencari satu 1 pasangannya, lalu mengulang sampai semua simbol ditandai.

Soal Mesin Turing sering tidak cukup dijawab dengan ide umum. Jika diminta melakukan trace, tuliskan state, isi tape, posisi head, dan aksi transisi pada setiap langkah penting. Strategi mark-scan untuk $L = \{0^n1^n \mid n \geq 1\}$ adalah:

1. Scan dari kiri dan tandai 0 pertama yang belum dipakai menjadi X .
2. Scan ke kanan melewati 0 dan Y sampai menemukan 1 pertama yang belum dipakai, lalu tandai menjadi Y .
3. Kembali ke kiri sampai melewati marker X atau blank, lalu cari 0 berikutnya.
4. Jika tidak ada 0 tersisa, scan kanan untuk memastikan semua 1 sudah menjadi Y .
5. Accept jika hanya ada X dan Y tersisa; reject jika bentuk input melanggar urutan 0^*1^* atau jumlahnya tidak seimbang.

Langkah	State	Isi Tape	Posisi Head	Aksi
0	q_0	0011	simbol pertama 0	Mulai; pilih 0 paling kiri yang belum ditandai.
1	q_1	X011	simbol kedua 0	Tulis X , bergerak kanan mencari 1 pasangan.
2	q_1	X011	simbol pertama 1	Lewati 0 yang masih belum dipasangkan.
3	q_2	X0Y1	simbol 0	Tulis Y pada 1 pertama, bergerak kiri kembali.
4	q_2	X0Y1	simbol X	Bergerak kiri melewati 0 sampai marker kiri.
5	q_0	X0Y1	simbol kedua 0	Kembali ke kanan untuk mencari 0 berikutnya.
6	q_1	XXY1	simbol Y	Tandai 0 kedua menjadi X , lalu scan kanan.
7	q_1	XXY1	simbol terakhir 1	Lewati Y yang sudah menjadi pasangan.
8	q_2	XXYY	simbol Y kedua	Tandai 1 terakhir menjadi Y , kembali ke kiri.
9	q_3	XXYY	simbol pertama Y	Tidak ada 0 tersisa; verifikasi sisa kanan hanya Y .
10	q_{acc}	XXYY	blank kanan	Semua simbol ditandai seimbang; accept.

Tabel 10: Trace ringkas TM mark-scan untuk input 0011

Komentar 7.2: Catatan: Pola Soal UAS dan Transducer Unary

Sumber soal/latihan NotebookLM menekankan bahwa trace TM sering ditulis sebagai konfigurasi $\alpha q \beta$, misalnya state disisipkan tepat sebelum simbol yang dibaca. Untuk soal transducer aritmetika, integer n biasanya direpresentasikan unary: $n = 3$ ditulis sebagai 000 atau 111, sesuai konvensi soal. Untuk fungsi $f(n) = 2n$, strategi standar adalah menyediakan area output, lalu untuk setiap simbol input unary yang belum diproses, tandai simbol tersebut dan tulis dua simbol unary pada area output. Multi-tape TM membuat pola ini jauh lebih mudah karena satu tape dapat menyimpan input dan tape lain menyimpan output, tetapi daya komputasinya tetap ekuivalen dengan single-tape TM.

7.2.8 Acceptance by Final State

Inti Konsep.

Acceptance by final state menerima string jika komputasi dari ID awal mencapai ID yang memuat state penerima. Mesin boleh saja memiliki isi pita apa pun saat menerima.

Bahasa yang diterima oleh Mesin Turing M adalah:

$$L(M) = \{w \mid q_0 w \vdash^* \alpha p \beta, p \in F\}.$$

ID awal ditulis q_0w , dengan input w berada di pita dan head membaca simbol pertama input.

Jika mesin mencapai state $p \in F$, string diterima. Jika string bukan anggota bahasa, mesin tidak harus berhenti; ia boleh berhenti di konfigurasi non-accepting atau berjalan selamanya, tergantung mesin.

Inilah perbedaan penting antara recognizer dan decider. Recognizer cukup menerima semua string anggota bahasa, tetapi tidak harus memberi jawaban penolakan untuk semua string bukan anggota.

7.2.9 Acceptance by Halting

Inti Konsep.

Acceptance by halting menerima string jika komputasi mencapai ID yang tidak memiliki transisi lanjutan. Mode ini ekuivalen dengan acceptance by final state dalam kelas bahasa yang dapat dikenali.

Bahasa yang diterima dengan halting adalah:

$$H(M) = \{w \mid q_0w \vdash^* I, \text{ dan tidak ada langkah dari ID } I\}.$$

Artinya, mesin berhenti karena fungsi transisi tidak terdefinisi pada konfigurasi tersebut.

Sumber menyebutkan bahwa kelas bahasa dari acceptance by final state dan acceptance by halting ekuivalen. Intuisi pembuktiannya: final state dapat dibuat tanpa transisi keluar sehingga masuk final state berarti halt; sebaliknya, semua kondisi halt dapat dialihkan ke state penerima khusus.

Ekuivalensi ini berguna karena beberapa pembuktian lebih mudah dinyatakan dengan “mencapai final state”, sedangkan pembahasan algoritma sering lebih natural memakai “halt”.

7.2.10 Mesin Turing sebagai Transducer

Inti Konsep.

Sebagai transducer, Mesin Turing tidak hanya menerima atau menolak input, tetapi menghitung sebuah fungsi. Input diletakkan pada pita, mesin berhenti, lalu isi pita tertentu dibaca sebagai output.

Recognizer menjawab pertanyaan ya atau tidak tentang membership bahasa. Namun program nyata sering menghitung nilai, misalnya menambah dua bilangan, mengubah string, atau menghasilkan representasi baru. Untuk fungsi seperti ini, Mesin Turing dipandang sebagai **transducer**, yaitu mesin yang mentransformasi input menjadi output.

Secara umum, jika fungsi f terdefinisi pada input w , maka mesin mulai dari konfigurasi awal berisi w , melakukan komputasi, berhenti, dan meninggalkan $f(w)$ pada pita dalam format encoding yang disepakati. Jika $f(w)$ tidak terdefinisi, mesin boleh tidak halt.

Peran TM	Makna
Recognizer	Menentukan apakah input termasuk bahasa dengan menerima anggota bahasa.
Decider	Menentukan membership dengan selalu halt untuk semua input.
Transducer	Menghitung output dari input, bukan hanya memberi jawaban accept/reject.

Tabel 11: Peran Mesin Turing dalam bahasa dan fungsi

Transducer menghubungkan teori Mesin Turing dengan konsep algoritma umum. Ketika kita mengatakan fungsi dapat dihitung, maknanya adalah ada Mesin Turing transducer yang menghitung fungsi tersebut.

7.2.11 Mesin Turing sebagai Enumerator

Inti Konsep.

Enumerator adalah Mesin Turing yang mencetak string-string suatu bahasa satu per satu. Sebuah bahasa recursively enumerable jika dan hanya jika ada enumerator yang menghasilkan semua string dalam bahasa tersebut.

Recognizer mengenali bahasa melalui input tunggal. Enumerator mengambil sudut pandang berbeda: mesin berjalan tanpa harus diberi satu input tertentu, lalu mencetak daftar string yang menjadi anggota bahasa. Urutan cetak boleh tidak teratur dan boleh mengandung pengulangan, selama setiap string anggota akhirnya muncul.

Jika ada recognizer untuk bahasa L , enumerator untuk L dapat dibuat dengan dovetailing:

1. Enumerasi semua string Σ^* dalam urutan kanonik, misalnya panjang naik lalu leksikografis.
2. Jalankan recognizer secara paralel terbatas pada semakin banyak string dan semakin banyak langkah.
3. Setiap kali sebuah simulasi menerima, cetak string terkait.

Sebaliknya, jika ada enumerator untuk L , recognizer untuk L dapat menjalankan enumerator dan menerima ketika string input muncul pada output enumerator. Jika input bukan anggota, recognizer dapat berjalan selamanya.

Enumerator memperjelas istilah recursively enumerable: bahasa RE adalah bahasa yang anggotanya dapat didaftar secara efektif, walaupun mungkin tidak ada algoritma yang selalu berhenti untuk memastikan non-anggota.

7.2.12 Recursively Enumerable dan Recursive Language

Inti Konsep.

Recursively enumerable language adalah bahasa yang dapat dikenali Mesin Turing, tetapi mesin bisa loop untuk input yang bukan anggota. Recursive language adalah bahasa yang dapat diputuskan Mesin Turing yang selalu halt untuk semua input.

Sebuah bahasa **recursively enumerable (RE)** memiliki TM yang menerima setiap string anggota. Untuk string bukan anggota, mesin boleh menolak dengan berhenti atau berjalan tanpa akhir. Karena itu RE berhubungan dengan recognizer.

Sebuah bahasa **recursive** memiliki TM yang selalu berhenti untuk setiap input. Jika input anggota bahasa, mesin accept; jika bukan anggota, mesin reject. Mesin seperti ini disebut decider dan merupakan formalisasi dari algoritma keputusan.

Kelas	Jika $w \in L$	Jika $w \notin L$
Recursively enumerable	Pasti accept suatu saat.	Boleh reject atau loop selamanya.
Recursive	Pasti accept suatu saat.	Pasti reject suatu saat.

Tabel 12: Perbedaan RE dan recursive language

Semua recursive language adalah RE, tetapi tidak semua RE recursive. Universal language adalah contoh penting bahasa yang RE tetapi undecidable.

Teorema 7.3: Teorema Komplemen RE

Jika $L \in RE$ dan $\bar{L} \in RE$, maka L recursive.

$$L \in RE \wedge \bar{L} \in RE \Rightarrow L \in Recursive.$$

Kontraposisifnya: jika $L \in RE$ tetapi $L \notin Recursive$, maka $\bar{L} \notin RE$.

Ide pembuktiannya adalah menjalankan recognizer untuk L dan recognizer untuk $\bar{L}$ secara dovetailing. Untuk input apa pun, tepat salah satu recognizer akhirnya menerima; ketika salah satunya menerima, decider dapat berhenti dengan jawaban accept atau reject. Teorema ini penting untuk argumen undecidability: jika sebuah bahasa RE terbukti tidak recursive, maka komplemennya tidak mungkin RE.

Komentar 7.4: Catatan: Hubungan dengan L_d dan L_u

Untuk bahasa diagonal L_d dan universal language L_u , sumber standar memakai relasi komplemen untuk menunjukkan batas RE. L_u adalah RE karena universal TM dapat mensimulasikan M pada w dan menerima jika simulasi menerima, tetapi L_u tidak recursive. Bahasa diagonal L_d tidak RE; relasi ini konsisten dengan fakta bahwa komplemen bahasa RE undecidable tidak harus RE.

7.2.13 Teknik Pemrograman Mesin Turing

Inti Konsep.

Teknik pemrograman TM adalah pola desain yang membuat konstruksi mesin lebih mudah tanpa menambah daya komputasi. Storage in state, multiple tracks, dan subroutine membantu menulis mesin tingkat tinggi secara ringkas.

****Storage in state**** menyimpan informasi kecil dalam state. Misalnya state dapat ditulis sebagai $[q, A, B, C]$, dengan q sebagai kontrol utama dan A, B, C sebagai data terbatas yang diingat. Karena jumlah state tetap berhingga, teknik ini hanya cocok untuk informasi berhingga.

****Multiple tracks**** memperlakukan satu sel pita sebagai tuple beberapa komponen, misalnya $[X, Y, Z]$. Ini memungkinkan mesin menandai suatu posisi pada track atas tanpa menghapus simbol asli pada track bawah. Secara formal, ini hanya memperbesar alfabet pita Γ .

****Marker dan shifting-over**** adalah teknik untuk mengubah isi tape secara terkontrol. Marker seperti X, Y , atau simbol khusus lain dipakai untuk mencatat progress tanpa melupakan posisi penting, misalnya simbol yang sudah dipasangkan pada bahasa $0^n 1^n$ atau batas antarbagian string. Shifting-over dipakai ketika mesin perlu menyisipkan simbol di tengah tape: simbol-simbol di kanan posisi sisip digeser satu sel ke kanan, lalu simbol baru ditulis pada posisi kosong. Teknik ini lazim pada konstruksi untuk bahasa seperti $\{ww^R\}$ atau transducer aritmetika yang perlu membangun output di pita yang sama.

****Subroutines**** adalah kumpulan state yang menjalankan tugas tertentu dan dipakai berulang. Sumber menekankan bahwa subroutine memiliki start state dan return state; untuk pemanggilan dari tempat berbeda, dapat dibuat salinan state agar arah kembali tidak ambigu.

Teknik ini penting untuk pembuktian ekuivalensi model. Alih-alih menulis semua transisi kecil, kita boleh mendeskripsikan konstruksi TM pada level prosedural selama jelas dapat diterjemahkan menjadi transisi formal.

7.2.14 Multitape dan Nondeterministic TM

Inti Konsep.

Multitape TM dan nondeterministic TM mempermudah desain komputasi, tetapi tidak menambah kelas bahasa yang dapat dikenali. Keduanya dapat disimulasikan oleh deterministic single-tape TM.

****Multitape TM**** memiliki k pita dan k head baca-tulis. Satu transisi membaca k simbol sekaligus, menulis k simbol, dan menggerakkan tiap head. Model ini lebih nyaman untuk algoritma, tetapi dapat disimulasikan oleh satu pita dengan $2k$ track: sebagian track menyimpan isi pita, sebagian track menyimpan marker posisi head. Sumber menyebut simulasi ini berbiaya waktu $O(n^2)$.

****Nondeterministic TM (NTM)**** memiliki beberapa pilihan transisi dari satu konfigurasi. Input diterima jika ada satu cabang komputasi yang menerima. Deterministic TM dapat mensimulasikan NTM dengan menelusuri pohon konfigurasi secara breadth-first menggunakan antrean ID, sehingga tidak terjebak pada satu cabang yang loop. Simulasi ini dapat lambat secara eksponensial, tetapi daya pengenalan bahasanya tetap sama.

Kesimpulannya, varian ini memperjelas perbedaan antara kekuatan komputasi dan efisiensi. Dari sisi bahasa yang dapat dikenali, model-model ini ekuivalen; dari sisi waktu, overhead simulasi dapat signifikan.

7.2.15 Multi-Head, Two-Way, dan Off-Line TM

Inti Konsep.

Multi-head, two-way infinite tape, dan off-line TM adalah variasi Mesin Turing yang mengubah organisasi pita dan head. Variasi ini mempermudah pemodelan algoritma tertentu, tetapi tetap ekuivalen dengan Mesin Turing standar dalam daya komputasi.

Sumber baru menambahkan beberapa variasi operasional. ****Two-way infinite tape**** memiliki pita tak terbatas ke kiri dan kanan, bukan hanya semi-infinite dengan batas kiri. Model ini tampak lebih longgar karena head boleh bergerak melewati posisi awal ke sisi kiri, tetapi perilakunya dapat disimulasikan oleh pita standar dengan teknik pelipatan pita atau multi-track.

****Multi-head TM**** memakai beberapa head baca-tulis pada satu pita. Tiap head dapat membaca posisi berbeda dan bergerak secara independen. Model ini membuat beberapa algoritma lebih ringkas karena mesin dapat menandai dan membandingkan beberapa posisi sekaligus, tetapi semua posisi head tetap dapat dikodekan pada satu pita standar.

****Off-line TM**** memisahkan pita input dan pita kerja. Pita input bersifat read-only dan dibatasi marker kiri dan kanan, sedangkan working tape dipakai untuk menulis hasil antara. Model ini dekat dengan cara algoritma dianalisis dalam praktik: input tidak perlu diubah, sementara memori kerja dipakai untuk komputasi.

Kesamaan ketiganya adalah menambah kenyamanan desain, bukan menambah kelas bahasa yang dapat diputuskan atau dikenali. Ini memperkuat Church-Turing Thesis: detail fisik pita dan jumlah head tidak mengubah batas komputabilitas.

7.2.16 Restricted TM

Inti Konsep.

Restricted TM adalah model yang fiturnya tampak lebih terbatas, tetapi masih dapat mensimulasikan Mesin Turing biasa. Ini menunjukkan bahwa daya komputasi TM cukup robust terhadap variasi representasi memori.

****Semi-infinite tape**** hanya tak terbatas ke kanan. Pita dua arah dapat disimulasikan dengan melipat pita: posisi nonnegatif disimpan di satu track, posisi negatif di track lain. Marker khusus menandai titik awal agar mesin tidak bergerak keluar batas kiri.

****Multistack machine**** memakai beberapa stack sebagai memori. Dua stack cukup untuk mensimulasikan pita TM: satu stack menyimpan isi di kiri head, stack lain menyimpan isi di posisi head dan kanan head. Gerak head dapat diterjemahkan sebagai pop dari satu stack dan push ke stack lain.

****Counter machine**** mengganti stack dengan counter bilangan bulat nonnegatif. Mesin hanya dapat menguji apakah counter nol atau bukan nol. Sumber menyebut 3 counter dapat mensimulasikan multistack, dan 3 counter dapat dikodekan menjadi 2 counter dengan representasi faktorisasi prima seperti $2^i 3^j 5^k$.

Model-model ini membantu membuktikan bahwa komputasi tidak bergantung pada detail fisik pita, melainkan pada kemampuan menyimpan dan memanipulasi informasi tak terbatas secara efektif.

7.2.17 Hubungan dengan Komputer Riil

Inti Konsep.

Komputer riil dan Mesin Turing saling mensimulasikan secara konseptual jika komputer diberi memori eksternal tak terbatas. Karena itu, Mesin Turing menjadi model formal yang layak untuk membahas program nyata.

Komputer nyata dapat mensimulasikan Mesin Turing dengan menyimpan pita pada memori atau media eksternal. Keterbatasan fisik RAM atau disk diabaikan secara teoritis dengan asumsi persediaan memori kosong selalu dapat ditambah.

Sebaliknya, Mesin Turing multitape dapat mensimulasikan komputer riil dengan merepresentasikan memori, instruction counter, register, dan instruksi pada pita-pitanya. Operasi seperti pencarian alamat, pemindahan data, penjumlahan, dan shifting dapat dijalankan sebagai prosedur TM.

Sumber menyebut bahwa simulasi program komputer dengan n instruksi dapat dilakukan dalam batas waktu polinomial, misalnya $O(n^3)$ dalam konstruksi yang dibahas. Ini mendukung pandangan bahwa TM dan komputer riil ekuivalen untuk pembahasan decidability dan polynomial-time secara konseptual.

7.2.18 Encoding Mesin Turing

Inti Konsep.

Encoding Mesin Turing mengubah spesifikasi mesin menjadi string biner, sehingga sebuah TM dapat menerima deskripsi TM lain sebagai input. Ide ini memungkinkan universal TM, diagonalization, dan pembuktian undecidability.

Untuk membahas program sebagai data, setiap state, simbol, arah, dan transisi harus dikodekan. Sumber menggambarkan konvensi kode biner dengan deretan 0 dan 1, misalnya transisi dikodekan sebagai pola yang memuat indeks state, simbol baca, state tujuan, simbol tulis, dan arah gerak, lalu seluruh transisi digabung dengan separator.

Dengan encoding, semua Mesin Turing dapat dinumerasi sebagai $M_1, M_2, \dots$, dan semua string biner dapat dinumerasi sebagai $w_1, w_2, \dots$. Enumerasi ini menjadi dasar diagonalization language L_d , yang membandingkan mesin ke- i dengan input ke- i .

Encoding juga menjelaskan mengapa interpreter atau operating system dapat dipahami secara formal: program hanyalah data yang dikodekan dan dibaca oleh mesin universal.

7.2.19 Diagonalization Language

Inti Konsep.

Diagonalization language L_d berisi string w_i yang tidak diterima oleh mesin M_i pada input dirinya sendiri. Bahasa ini membuktikan bahwa ada bahasa yang tidak recursively enumerable.

Definisi sumber dapat ditulis:

$$L_d = \{w_i \mid w_i \notin L(M_i)\}.$$

Bayangkan tabel tak hingga dengan baris mesin M_i dan kolom string w_j . Entri tabel menyatakan apakah M_i menerima w_j . Bahasa L_d dibentuk dengan mengambil diagonal tabel, yaitu kasus M_i pada w_i , lalu membalik status penerimaannya.

Jika ada mesin D yang mengenali L_d , maka D sendiri muncul sebagai salah satu M_k dalam enumerasi. Pertanyaan apakah $w_k \in L(D)$ akan memunculkan kontradiksi: D menerima w_k jika dan hanya jika M_k tidak menerima w_k , padahal $D = M_k$.

Kesimpulannya, L_d bukan RE. Ini lebih kuat daripada sekadar undecidable: tidak ada TM yang bahkan dapat mengenali semua anggotanya dengan benar.

7.2.20 Universal Language

Inti Konsep.

Universal language L_u berisi pasangan encoding (M, w) sedemikian sehingga Mesin Turing M menerima input w . Bahasa ini RE karena dapat disimulasikan oleh Universal Turing Machine, tetapi tidak recursive.

Definisinya:

$$L_u = \{(M, w) \mid M \text{ menerima } w\}.$$

Universal Turing Machine U membaca encoding M dan input w , lalu mensimulasikan langkah-langkah M pada w . Jika M menerima, U juga menerima. Jika M loop, U ikut loop.

Karena simulasi ini mungkin tidak pernah berhenti pada input yang bukan anggota, L_u adalah RE. Namun L_u undecidable: tidak ada decider yang selalu berhenti dan menentukan apakah mesin arbitrer menerima input arbitrer. Ini adalah bentuk umum dari halting/acceptance problem.

Universal language menjadi sumber utama reduksi. Banyak problem lain dibuktikan undecidable dengan menunjukkan bahwa jika problem itu decidable, maka L_u juga decidable, kontradiksi.

7.2.21 Post Correspondence Problem

Inti Konsep.

Post Correspondence Problem (PCP) adalah problem undecidable tentang mencocokkan deretan kartu sehingga string atas dan string bawah menjadi sama. PCP penting karena sering dipakai sebagai sumber reduksi untuk membuktikan undecidability pada grammar dan bahasa formal.

Sebuah instansi PCP berisi himpunan kartu, masing-masing berbentuk pasangan string (x_i, y_i) . Pertanyaannya: apakah ada urutan indeks $i_1, i_2, \dots, i_k$ dengan $k \geq 1$ sehingga

$$x_{i_1}x_{i_2}\cdots x_{i_k} = y_{i_1}y_{i_2}\cdots y_{i_k}?$$

Kartu boleh dipakai berulang dan urutan solusi tidak harus memakai semua kartu.

Contoh struktur pencarian PCP:

1. Pilih satu kartu sebagai awal urutan.
2. Bandingkan prefix string atas dan bawah yang terbentuk.
3. Tambahkan kartu berikutnya untuk mencoba menutup selisih prefix.
4. Jika kedua string sama persis setelah sejumlah kartu, instansi memiliki solusi.

PCP undecidable: tidak ada algoritma umum yang selalu berhenti dan menentukan apakah instansi arbitrer memiliki solusi. Namun PCP semi-decidable karena semua urutan kartu berhingga dapat dienumerasi; jika solusi ada, pencarian sistematis akhirnya menemukannya.

Komentar 7.5: Catatan: Relevansi PCP

PCP sering muncul bukan karena konstruksi PDA atau CYK, tetapi karena ia menjadi jembatan reduksi dari Mesin Turing ke problem tentang string, grammar, dan derivasi. Jika soal undecidability menyinggung CFG atau ambiguity, PCP dapat menjadi sumber reduksi yang lebih natural daripada langsung memakai universal language.

PCP menguatkan pola berpikir reduction. Untuk membuktikan problem target undecidable, kita dapat menunjukkan bahwa solusi problem target akan memungkinkan kita menyelesaikan PCP, padahal PCP sudah diketahui undecidable.

7.2.22 Reduction

Inti Konsep.

Reduction adalah teknik membuktikan undecidability dengan mengubah instansi problem yang sudah diketahui undecidable menjadi instansi problem target. Jika problem target dapat diputuskan, maka problem asal juga dapat diputuskan; karena problem asal mustahil, problem target juga mustahil.

Misalkan P_1 sudah diketahui undecidable dan kita ingin membuktikan P_2 undecidable. Kita membangun algoritma transformasi dari input P_1 menjadi input P_2 sehingga jawaban P_2 dapat dipakai untuk menjawab P_1 .

Struktur buktinya:

1. Asumsikan ada decider untuk P_2 .
2. Tunjukkan cara memakai decider P_2 untuk membuat decider P_1 .
3. Karena P_1 sudah undecidable, asumsi decider P_2 salah.
4. Maka P_2 undecidable.

Reduksi perlu arah yang benar. Untuk membuktikan problem baru sulit, problem lama yang sudah sulit harus direduksi ke problem baru, bukan sebaliknya.

7.2.23 Rice's Theorem

Inti Konsep.

Rice's Theorem menyatakan bahwa setiap properti nontrivial tentang bahasa yang dikenali Mesin Turing bersifat undecidable. Teorema ini membuat banyak pertanyaan semantik tentang program otomatis tidak dapat diputuskan.

Properti disebut **nontrivial** jika ada setidaknya satu RE language yang memiliki properti itu dan setidaknya satu RE language yang tidak memilikinya. Properti yang dimaksud adalah properti bahasa yang dikenali, bukan sekadar properti sintaks encoding mesin.

Contoh pertanyaan yang termasuk cakupan Rice's Theorem:

- Apakah bahasa yang dikenali mesin ini kosong?
- Apakah bahasa yang dikenali mesin ini finite?
- Apakah bahasa yang dikenali mesin ini regular?
- Apakah bahasa yang dikenali mesin ini context-free?

Teorema ini menjelaskan mengapa analisis program umum sangat terbatas. Jika sebuah pertanyaan bergantung pada perilaku bahasa/program secara nontrivial, maka tidak ada algoritma universal yang selalu menjawab benar untuk semua Mesin Turing.

7.3 Algoritma dan Rumus Penting

7.3.1 Fungsi Transisi Mesin Turing

Makna Rumus.

Rumus ini menyatakan satu langkah atomik Mesin Turing: membaca state dan simbol pita, lalu menentukan state baru, simbol tulis, dan arah gerak head.

$$\delta(q, Z) = (p, Y, D) \quad (8)$$

dengan:

- q : state saat ini.
- Z : simbol pita yang sedang dibaca.
- p : state berikutnya.
- Y : simbol yang ditulis menggantikan Z .
- $D \in \{L, R\}$: arah gerak head.

7.3.2 Bahasa yang Diterima dan Dihentikan

Makna Rumus.

Rumus $L(M)$ menyatakan penerimaan melalui state final, sedangkan $H(M)$ menyatakan penerimaan melalui konfigurasi halt. Keduanya mendefinisikan kelas bahasa yang ekuivalen.

$$L(M) = \{w \mid q_0 w \vdash^* \alpha p \beta, p \in F\}, \quad (9)$$

$$H(M) = \{w \mid q_0 w \vdash^* I, \text{ dan tidak ada langkah dari } I\}. \quad (10)$$

dengan:

- $q_0 w$: ID awal dengan input w .
- $\alpha p \beta$: ID yang memuat state penerima p .
- I : ID halt yang tidak memiliki transisi lanjutan.

7.3.3 Pseudocode Universal Turing Machine

Tujuan Algoritma.

Pseudocode ini menggambarkan ide Universal Turing Machine: membaca deskripsi mesin lain dan mensimulasikan langkahnya pada input tertentu.

Listing 4: Pseudocode Universal Turing Machine

```
1 Input: encoding  $\langle M, w \rangle$ 
2 Output: accept jika  $M$  menerima  $w$ 
3
4 1. Decode deskripsi  $M$  menjadi daftar state, alfabet, dan transisi.
5 2. Inisialisasi konfigurasi simulasi sebagai ID awal  $M$  pada input  $w$ .
6 3. Selama konfigurasi simulasi belum accepting:
7   a. Baca state saat ini dan simbol yang sedang dipindai.
8   b. Cari transisi  $M$  yang sesuai.
9   c. Jika tidak ada transisi, berhenti tanpa menerima.
10  d. Terapkan transisi itu pada konfigurasi simulasi.
11 4. Jika state penerima  $M$  tercapai, accept.
```

Jika mesin M berjalan selamanya pada w , universal machine juga berjalan selamanya. Karena itu L_u dapat dikenali tetapi tidak dapat diputuskan.

8 Compiler

Compiler adalah program penerjemah yang mengubah program sumber menjadi program target yang ekuivalen, biasanya kode mesin atau assembly. Materi ini menunjukkan penerapan teori bahasa formal dalam sistem nyata: regular expression dan finite automata dipakai pada lexical analysis, CFG dipakai pada syntax analysis, dan struktur semantik program dipakai untuk menghasilkan intermediate representation, optimasi, serta kode target.

Fokus section ini adalah arsitektur compiler secara menyeluruh. Detail algoritma parsing top-down, Recursive Descent, dan LL(1) dibahas pada section terpisah; di sini parsing ditempatkan sebagai salah satu fase dalam pipeline compiler.

8.1 Outline Fundamental Konsep Materi

1. **WAJIB** Sistem pemrosesan bahasa
 - (a) **PAHAM** Tujuan desain bahasa dan compilability
 - (b) **WAJIB** Compiler, interpreter, VM, bytecode, dan JIT
 - (c) **PAHAM** Preprocessor, compiler, assembler, linker, dan loader
 - (d) **PAHAM** Bootstrapping dan translator tools
 - (e) **PAHAM** Cross-compiling
2. **WAJIB** Struktur compiler
 - (a) **WAJIB** Analysis/front end
 - (b) **WAJIB** Synthesis/back end
 - (c) **WAJIB** Fase lexical, syntax, semantic, IR, optimization, dan code generation
 - (d) **WAJIB** Symbol table
 - (e) **WAJIB** Error handling compiler
 - (f) **PAHAM** Passes
3. **WAJIB** Lexical analysis
 - (a) **WAJIB** Lexeme, pattern, dan token
 - (b) **PAHAM** Input buffering dan sentinel
 - (c) **WAJIB** Regular expression dan transition diagram
 - (d) **PAHAM** Lex/Flex dan longest prefix
4. **WAJIB** Syntax dan semantic analysis
 - (a) **WAJIB** CFG, terminal, nonterminal, start symbol, dan production
 - (b) **WAJIB** Parse tree atau syntax tree
 - (c) **PAHAM** Bottom-up parsing, LR parser, dan Yacc
 - (d) **WAJIB** SDD, SDT, synthesized dan inherited attributes
 - (e) **WAJIB** Type checking, coercion, dan overloading
5. **WAJIB** Runtime environment
 - (a) **WAJIB** Code, static data, stack, dan heap
 - (b) **WAJIB** Activation record
 - (c) **PAHAM** Garbage collection dan reachability
6. **WAJIB** Code generation

- (a) **WAJIB** Basic block dan flow graph
 - (b) **WAJIB** Register descriptor dan address descriptor
 - (c) **PAHAM** Instruction selection dan register allocation
 - (d) **PAHAM** Peephole optimization
 - (e) **PAHAM** Single accumulator code generation
7. **WAJIB** Machine-independent optimization
- (a) **WAJIB** Data-flow analysis
 - (b) **WAJIB** Gen-kill equation
 - (c) **WAJIB** Reaching definitions
 - (d) **WAJIB** Available expressions
 - (e) **WAJIB** Live variables

8.2 Penjelasan Konsep

8.2.1 Tujuan Desain Bahasa dan Compilability

Inti Konsep.

Desain bahasa pemrograman memengaruhi seberapa mudah bahasa dianalisis dan dikompilasi. Simplicity, orthogonality, machine independence, dan compilability membantu compiler menerjemahkan program secara konsisten dan efisien.

Compiler tidak bekerja di ruang kosong; bentuk bahasa sumber menentukan kompleksitas lexical analysis, syntax analysis, semantic analysis, dan code generation. Bahasa yang terlalu banyak pengecualian membuat grammar, type checker, dan optimizer lebih sulit dibangun.

Tujuan Desain	Dampak pada Compiler
Simplicity	Aturan bahasa yang sederhana mengurangi kasus khusus pada parser dan semantic analyzer.
Orthogonality	Fitur dapat dikombinasikan secara konsisten, sehingga compiler tidak perlu banyak pengecualian semantik.
Machine independence	Program tidak terlalu bergantung pada detail hardware, sehingga front end dan IR lebih portabel.
Compilability	Konstruksi bahasa dapat diterjemahkan secara mekanis ke IR dan kode target dengan biaya masuk akal.

Tabel 13: Tujuan desain bahasa yang memengaruhi compiler

Konsep ini menghubungkan teori bahasa formal dengan rekayasa bahasa. Grammar yang rapi dan aturan semantik yang konsisten membuat compiler lebih mudah dibangun, diuji, dan dioptimasi.

8.2.2 Compiler, Interpreter, dan Virtual Machine

Inti Konsep.

Compiler menerjemahkan program sumber menjadi program target sebelum eksekusi, sedangkan interpreter mengeksekusi operasi program sumber secara langsung. Virtual machine menggabungkan keduanya melalui bytecode yang portabel dan dapat dipercepat dengan JIT compiler.

Compiler membaca program dalam **source language** dan menghasilkan program ekuivalen dalam **target language**. Target sering berupa assembly atau kode mesin. Keunggulan utamanya adalah performa eksekusi, karena program target sudah disiapkan sebelum dijalankan.

Interpreter tidak menghasilkan program target utuh. Ia membaca program sumber dan langsung melakukan operasi yang diminta program. Karena eksekusi mengikuti program secara langsung, interpreter sering memberi diagnostik error yang lebih dekat dengan lokasi kesalahan, tetapi biasanya lebih lambat daripada kode hasil kompilasi.

Model hibrida menggunakan **bytecode**, yaitu representasi antara yang lebih rendah daripada source code tetapi belum terikat penuh pada hardware tertentu. **Virtual machine (VM)** mengeksekusi bytecode sebagai software interpreter. Pada sistem modern, **Just-In-Time (JIT) compiler** dapat menerjemahkan bytecode menjadi kode mesin saat runtime untuk mengurangi penalti performa interpretasi.

Konsep ini menghubungkan compiler dengan materi Interpreter dan Virtual Machine. Section ini hanya memberi gambaran pipeline; detail VM akan dibahas pada materi tersendiri.

8.2.3 Language-Processing System

Inti Konsep.

Compiler biasanya bukan satu-satunya program dalam sistem pemrosesan bahasa. Program sumber melewati preprocessor, compiler, assembler, linker, dan loader sebelum menjadi proses yang berjalan.

Sistem pemrosesan bahasa membentuk pipeline dari source code menuju program eksekutabel. Setiap komponen menyelesaikan bentuk penerjemahan yang berbeda.

Komponen	Peran
Preprocessor	Mengumpulkan modul terpisah dan mengekspansi macro atau short-hand.
Compiler	Menerjemahkan source program hasil preprocessing menjadi assembly atau representasi target lain.
Assembler	Mengubah assembly menjadi relocatable machine code.
Linker	Menggabungkan file objek dan library, serta menyelesaikan referensi alamat eksternal.
Loader	Memuat program eksekutabel ke memori agar dapat dijalankan.

Tabel 14: Komponen sistem pemrosesan bahasa

Pipeline ini penting karena banyak error atau konsep sistem tidak berada murni dalam compiler. Misalnya undefined reference biasanya muncul saat linking, bukan saat parsing. Sebaliknya, syntax error ditemukan saat compiler menganalisis token dan grammar.

8.2.4 Bootstrapping dan Translator Tools

Inti Konsep.

Bootstrapping dan translator tools menjelaskan bagaimana compiler dapat dibangun dengan bantuan compiler lain atau generator otomatis. Konsep ini penting karena compiler nyata jarang ditulis seluruhnya dari nol dalam satu bahasa target.

Bootstrapping adalah teknik membangun compiler untuk suatu bahasa dengan memanfaatkan compiler atau interpreter yang sudah tersedia. Dalam praktik, compiler awal dapat ditulis dalam bahasa lain, lalu setelah cukup matang, compiler dapat dipakai untuk mengompilasi

versi berikutnya dari dirinya sendiri. Ini menunjukkan bahwa penerjemahan bahasa adalah proses bertahap yang dapat disusun dari beberapa translator.

Sumber baru juga menyebut beberapa **translator tools**, yaitu perangkat bantu yang menghasilkan bagian compiler dari spesifikasi formal:

Tool	Peran
Lex/Flex	Menghasilkan lexical analyzer dari spesifikasi regular expression.
Yacc/Bison	Menghasilkan parser dari spesifikasi grammar dan semantic actions.
ANTLR	Framework populer untuk membuat recognizer, compiler, dan translator berbasis grammar description.
ELI	Lingkungan pengembangan compiler yang membantu analisis struktural, analisis tipe/nama, dan generasi kode.
GENTLE	Toolkit untuk membangun compiler dengan dukungan AST, pattern matching, dan traversal.

Tabel 15: Contoh translator tools dalam pengembangan compiler

Hubungannya dengan teori bahasa formal jelas: bagian lexical analyzer berasal dari regular language dan automata hingga DFA, sedangkan parser berasal dari CFG dan algoritma parsing seperti RD atau LL(1).

Cross-compiling adalah membangun compiler yang berjalan pada mesin host tetapi menghasilkan kode untuk mesin target yang berbeda. Teknik ini penting ketika target belum memiliki compiler matang, misalnya pada sistem embedded atau arsitektur baru.

8.2.5 Front End dan Back End

Inti Konsep.

Front end menganalisis program sumber dan menghasilkan intermediate representation, sedangkan back end mensintesis program target dari representasi tersebut. Pemisahan ini membuat compiler modular dan dapat digunakan untuk banyak bahasa sumber atau banyak target mesin.

Analysis atau **front end** memecah source program, memeriksa struktur grammar, memeriksa konsistensi semantik, dan membangun intermediate representation (IR). Front end juga mengisi symbol table dengan informasi identifier seperti tipe, scope, dan lokasi.

Synthesis atau **back end** memakai IR dan symbol table untuk menghasilkan kode target. Back end melakukan pemilihan instruksi, alokasi register, penentuan lokasi memori, dan penyusunan urutan instruksi.

Pemisahan ini memungkinkan desain m front end dan n back end. Secara ideal, setiap bahasa sumber hanya perlu diterjemahkan ke IR, lalu IR dapat diterjemahkan ke berbagai arsitektur target.

Komentar 8.1: Catatan: Sumber Pengetahuan Umum

Banyak compiler modern juga memakai istilah middle end untuk fase optimasi yang bekerja pada IR dan relatif independen dari bahasa sumber maupun mesin target. Istilah ini tidak mengubah pipeline konseptual front end dan back end, tetapi membantu memahami mengapa optimasi sering dipisahkan sebagai lapisan tersendiri.

8.2.6 Fase-Fase Compiler

Inti Konsep.

Compiler dibagi menjadi fase-fase logis: lexical analysis, syntax analysis, semantic analysis, intermediate code generation, optimization, dan code generation. Setiap fase mengubah representasi program menjadi bentuk yang lebih terstruktur atau lebih dekat ke mesin target.

Fase-fase compiler membentuk alur bertahap. Input awal adalah karakter source code. Output akhir adalah program target.

Fase	Peran Utama
Lexical Analysis	Mengubah character stream menjadi token stream.
Syntax Analysis	Memeriksa struktur token terhadap CFG dan membangun syntax tree.
Semantic Analysis	Memeriksa makna program, misalnya tipe dan scope.
Intermediate Code Generation	Menghasilkan IR seperti three-address code.
Machine-Independent Optimization	Memperbaiki IR tanpa bergantung pada mesin target.
Code Generation	Memetakan IR ke instruksi mesin target, register, dan memori.

Tabel 16: Fase logis compiler

Walaupun fase-fase ini logis, implementasi nyata dapat menggabungkan beberapa fase ke dalam satu **pass**, yaitu satu siklus membaca input dan menghasilkan output antara. Misalnya front-end pass dapat mencakup scanning, parsing, semantic analysis, dan pembuatan IR.

8.2.7 Symbol Table

Inti Konsep.

Symbol table adalah struktur data yang menyimpan identifier dan atributnya. Hampir semua fase compiler membutuhkan symbol table untuk memahami nama, tipe, scope, parameter, dan lokasi penyimpanan.

Saat scanner menemukan identifier, compiler perlu mengetahui apakah identifier itu variabel, fungsi, parameter, field, atau nama lain. Symbol table menyimpan entri untuk setiap nama penting beserta atributnya.

Atribut yang umum disimpan meliputi tipe data, scope, jumlah argumen fungsi, jenis simbol, ukuran memori, dan lokasi penyimpanan. Semantic analyzer memakai informasi ini untuk type checking dan scope checking. Code generator memakai informasi lokasi untuk menghasilkan load dan store yang benar.

Karena symbol table diakses berulang oleh banyak fase, operasinya harus cepat. Implementasi dapat memakai hash table, tree, atau struktur berlapis untuk mendukung nested scope.

Sumber terbaru memberi struktur symbol table yang lebih rinci. Dalam compiler besar, satu “symbol table” konseptual dapat dipecah menjadi beberapa tabel khusus agar atribut yang berbeda tidak bercampur.

Tabel	Peran
Identifier table	Menyimpan nama identifier dan pointer ke atribut semantiknya.
Array table	Menyimpan informasi array seperti dimensi, batas indeks, dan ukuran elemen.
Block table	Menyimpan informasi scope atau block, termasuk hubungan parent-child antar-block.
Real/string table	Menyimpan literal real atau string agar dapat dirujuk ulang tanpa duplikasi.
Display table	Membantu akses variabel non-lokal pada bahasa dengan nested scope.

Tabel 17: Fragmentasi symbol table pada compiler

Fragmentasi ini tidak mengubah fungsi dasar symbol table, tetapi membuat implementasi lebih terorganisasi. Hubungannya dengan runtime environment terlihat pada display table dan block table, karena keduanya menentukan cara variabel pada scope luar diakses saat program berjalan.

8.2.8 Error Handling dalam Pipeline Compiler

Inti Konsep.

Error handling adalah mekanisme compiler untuk mendeteksi, melaporkan, dan sebisa mungkin melanjutkan analisis setelah menemukan kesalahan. Tujuannya bukan memperbaiki program secara sempurna, tetapi mencegah satu error menghasilkan crash atau rentetan pesan palsu.

Compiler yang baik tidak boleh bereaksi dengan tiga cara buruk: crash, infinite loop, atau menghasilkan target program yang tampak valid tetapi korup. Karena itu, error handler biasanya dipakai lintas fase: scanner melaporkan token ilegal, parser melaporkan struktur sintaks salah, semantic analyzer melaporkan type atau scope error, dan code generator tidak boleh menghasilkan kode jika informasi semantik belum aman.

Strategi Recovery	Peran dalam Compiler
Panic mode	Melewati input sampai delimiter atau token sinkronisasi ditemukan.
Unit deletion	Mengabaikan unit sintaks utuh seperti statement atau block yang rusak.
Terminal repair	Menyisipkan token yang hilang atau menghapus token berlebih secara lokal.
Context-sensitive repair	Memakai informasi semantik seperti tipe, scope, atau deklarasi untuk mengurangi error lanjutan.
Spelling repair	Mencoba mengenali salah ketik keyword atau identifier secara heuristik.

Tabel 18: Strategi error recovery dalam compiler

Dummy identifier adalah contoh context-sensitive repair. Jika identifier x belum dideklarasikan, compiler dapat memasukkan x ke symbol table dengan tipe dummy agar penggunaan berikutnya tidak selalu menghasilkan error baru. Pesan error utama tetap dilaporkan, tetapi analisis lanjutan menjadi lebih stabil.

8.2.9 Lexeme, Pattern, dan Token

Inti Konsep.

Lexical analysis mengelompokkan karakter menjadi lexeme dan mengirim token ke parser. Lexeme adalah string aktual di source code, pattern adalah aturan bentuknya, dan token adalah simbol abstrak yang dipakai parser.

Tiga istilah ini harus dibedakan.

Istilah	Definisi
Pattern	Aturan yang mendeskripsikan bentuk lexeme, sering ditulis dengan regular expression.
Lexeme	Urutan karakter aktual dalam program sumber yang cocok dengan pattern.
Token	Pasangan abstrak $\langle token-name, attribute-value \rangle$ yang dikirim ke parser.

Tabel 19: Istilah dasar lexical analysis

Contoh: pada kode `'count = count + 1'`, lexeme `'count'` dapat cocok dengan pattern identifier dan dikirim sebagai token `'ID'` dengan attribute pointer ke entri symbol table. Parser tidak perlu melihat karakter `'c'`, `'o'`, `'u'`, `'n'`, `'t'`; parser cukup melihat token abstrak.

Lexical analysis berhubungan langsung dengan teori automata regular: pattern token dapat dispesifikasikan dengan regular expression dan diimplementasikan dengan finite automata.

8.2.10 Input Buffering dan Sentinel

Inti Konsep.

Input buffering mempercepat lexical analysis dengan membaca karakter dalam blok dan mendukung lookahead. Sentinel mengurangi biaya pengecekan batas buffer pada setiap karakter.

Lexer sering harus melihat beberapa karakter ke depan untuk menentukan batas lexeme. Misalnya, saat membaca `'<'`, lexer perlu melihat karakter berikutnya untuk membedakan `'<'` dan `'<='`. Jika setiap karakter dibaca langsung dari file, biaya I/O menjadi besar.

Teknik `**buffer pairs**` memakai dua buffer yang bergantian diisi dari file. Dua pointer penting adalah `**lexemeBegin**`, yang menunjuk awal lexeme saat ini, dan `**forward**`, yang bergerak mencari akhir lexeme.

`**Sentinel**` seperti `'eof'` ditempatkan di akhir buffer. Dengan sentinel, lexer dapat mendeteksi akhir buffer tanpa melakukan pengecekan batas eksplisit pada setiap karakter. Ini adalah optimasi kecil tetapi penting pada scanner yang berjalan untuk seluruh source code.

8.2.11 Regular Expression dan Transition Diagram

Inti Konsep.

Regular expression mendeskripsikan pattern token secara deklaratif, sedangkan transition diagram mengimplementasikan proses pengenalan token sebagai automaton. Keduanya menghubungkan lexical analysis dengan finite automata.

Regular expression dibangun dari operasi union, concatenation, dan Kleene closure. Token seperti identifier, integer literal, operator, dan whitespace dapat dispesifikasikan dengan regular expression.

Transition diagram adalah representasi grafis dari automaton yang mengenali pattern. Node adalah state, panah adalah transisi berdasarkan karakter input, dan accepting state menandakan lexeme berhasil dikenali.

Dalam implementasi sistematis, regular expression dapat dikonversi ke NFA, lalu ke DFA, lalu diminimasi jika diperlukan. Materi compiler biasanya menekankan bahwa scanner dapat dihasilkan otomatis dari spesifikasi token karena token berada pada kelas regular language.

8.2.12 Lex/Flex dan Longest Prefix

Inti Konsep.

Lex atau Flex adalah generator lexical analyzer yang mengubah spesifikasi regular expression menjadi program scanner. Konflik token diselesaikan dengan aturan longest prefix, lalu prioritas aturan yang muncul lebih awal.

Lex menerima daftar regular expression dan aksi program. Outputnya adalah program scanner, sering berupa 'lex.yy.c', yang membaca input dan menghasilkan token.

Masalah muncul ketika beberapa pattern dapat cocok dengan awalan input yang sama. Contoh: 'if' dapat cocok sebagai keyword, tetapi 'ifx' harus menjadi identifier. Lex memilih lexeme terpanjang yang dapat dicocokkan. Jika ada dua pattern dengan panjang cocok sama, aturan yang ditulis lebih awal dipilih.

Aturan ini membuat spesifikasi token harus disusun hati-hati. Keyword biasanya ditulis sebelum identifier agar lexeme 'if' dikenali sebagai keyword, bukan identifier, ketika panjangnya sama.

8.2.13 Syntax Analysis sebagai Konteks CFG

Inti Konsep.

Syntax analysis memeriksa apakah token stream mengikuti struktur grammar bahasa. CFG digunakan karena struktur program bersifat hierarkis, misalnya statement berisi expression dan expression berisi subexpression.

Parser menerima token dari scanner dan membangun parse tree atau syntax tree. Grammar yang dipakai adalah Context-Free Grammar dengan komponen terminal, nonterminal, start symbol, dan productions.

Terminal adalah token yang muncul di daun parse tree. Nonterminal adalah kategori sintaksis seperti 'stmt', 'expr', atau 'decl'. Start symbol adalah akar grammar, sedangkan production menjelaskan cara nonterminal diturunkan menjadi rangkaian terminal dan nonterminal.

Section ini tidak membahas algoritma parser secara detail. Recursive Descent dan LL(1) akan dibahas pada materi tersendiri. Di sini yang penting adalah posisi syntax analysis dalam compiler: ia mengubah token stream menjadi struktur sintaksis yang dapat dianalisis semantik.

8.2.14 Bottom-Up Parsing, LR Parser, dan Yacc

Inti Konsep.

Bottom-up parsing membangun parse tree dari token menuju start symbol dengan operasi shift dan reduce. LR parser adalah keluarga bottom-up parser deterministik yang banyak dipakai oleh generator parser seperti Yacc atau Bison.

Top-down parser mencoba menurunkan start symbol menjadi input. Bottom-up parser melakukan arah sebaliknya: ia membaca token, mengenali potongan yang cocok dengan ruas kanan produksi, lalu mereduksinya menjadi nonterminal sampai tersisa start symbol.

Aksi	Makna
Shift	Memindahkan token lookahead dari input ke stack parser.
Reduce	Mengganti rangkaian simbol di puncak stack yang cocok dengan body produksi menjadi head produksi.

Tabel 20: Aksi dasar shift-reduce parsing

LR parser membaca input dari kiri ke kanan dan membangun rightmost derivation secara terbalik. Keluarga LR lebih kuat daripada LL(1) untuk banyak grammar praktis, tetapi tabel dan konstruksinya lebih kompleks.

Yacc/Bison menerima grammar dengan semantic actions dan menghasilkan parser bottom-up. Jika grammar memiliki konflik, tool melaporkan ****shift/reduce conflict**** atau ****reduce/reduce conflict****. Ini berbeda dari konflik LL(1), tetapi maknanya sama-sama menunjukkan bahwa parser tidak memiliki satu aksi deterministik yang jelas pada konfigurasi tertentu.

Materi ini menjadi konteks bagi Recursive Descent dan LL(1). Ujian yang fokus pada top-down parsing biasanya tidak meminta konstruksi tabel LR lengkap, tetapi memahami shift-reduce membantu membaca posisi Yacc dalam compiler toolchain.

8.2.15 Semantic Analysis, SDD, dan SDT

Inti Konsep.

Semantic analysis memeriksa makna program yang tidak cukup ditentukan oleh syntax. SDD dan SDT memberi cara mengaitkan aturan grammar dengan komputasi atribut atau aksi semantik.

Syntax dapat memastikan struktur $x + y$ valid sebagai expression, tetapi belum memastikan tipe x dan y cocok untuk operator $+$. Itulah tugas semantic analysis.

****Syntax-Directed Definition (SDD)**** adalah notasi deklaratif yang menempelkan atribut dan aturan semantik pada grammar. ****Synthesized attribute**** dihitung dari anak ke parent, sedangkan ****inherited attribute**** mengalir dari parent atau sibling ke node lain.

****Syntax-Directed Translation Scheme (SDT)**** menempatkan semantic actions langsung pada body produksi. Aksi ini dieksekusi pada momen tertentu selama parsing. SDT lebih operasional daripada SDD karena memperhatikan urutan eksekusi.

Hasil semantic analysis dapat berupa annotated syntax tree, type information, atau IR awal yang siap dioptimasi.

8.2.16 Type Checking, Coercion, dan Overloading

Inti Konsep.

Type checking memastikan operasi diterapkan pada tipe operand yang sah. Coercion menyisipkan konversi implisit, sedangkan overloading resolution memilih makna operator atau fungsi berdasarkan tipe argumen.

Type checking mendeteksi kesalahan seperti menerapkan operator boolean pada operand numerik, memanggil fungsi dengan jumlah argumen salah, atau mengembalikan nilai yang tidak cocok dengan tipe deklarasi fungsi.

****Coercion**** adalah konversi implisit yang ditambahkan compiler jika bahasa mengizinkan. Contoh sumber: jika integer dijumlahkan dengan float, compiler dapat menyisipkan operasi `inttofloat` agar operasi dilakukan pada tipe yang konsisten.

****Overloading**** terjadi ketika satu simbol memiliki beberapa makna. Operator $+$ dapat berarti penjumlahan numerik atau penggabungan string, bergantung pada tipe operand. Compiler menyelesaikan overloading melalui informasi tipe dan konteks pemanggilan.

Analisis ini bergantung pada symbol table dan syntax tree. Tanpa informasi identifier dan struktur program, compiler tidak dapat memutuskan makna semantik yang benar.

8.2.17 Runtime Environment

Inti Konsep.

Runtime environment adalah model memori dan mekanisme eksekusi yang diasumsikan compiler saat menghasilkan kode. Area utamanya meliputi code, static data, stack, dan heap.

Compiler tidak hanya menerjemahkan ekspresi menjadi instruksi. Ia juga harus menempatkan data dan mengatur pemanggilan fungsi agar program dapat berjalan.

Area	Peran
Code	Menyimpan instruksi program target.
Static Data	Menyimpan data global atau data berukuran tetap.
Stack	Menyimpan activation record untuk call dan return fungsi.
Heap	Menyimpan objek atau data dinamis yang umur hidupnya tidak mengikuti satu pemanggilan fungsi.

Tabel 21: Area runtime environment

Pemahaman runtime environment penting untuk menjelaskan variabel lokal, parameter fungsi, return value, alokasi objek, dan garbage collection.

8.2.18 Activation Record

Inti Konsep.

Activation record atau stack frame adalah blok data pada runtime stack yang mewakili satu pemanggilan fungsi. Isinya mendukung parameter, variabel lokal, nilai return, link ke caller, dan status mesin.

Setiap pemanggilan fungsi membuat activation record baru. Ketika fungsi selesai, activation record di-pop dari stack. Model ini cocok untuk fungsi yang dipanggil secara bertumpuk, termasuk rekursi.

Komponen	Peran
Return value	Tempat nilai hasil fungsi dikembalikan.
Actual parameters	Argumen yang dikirim oleh caller.
Control link	Pointer ke activation record caller.
Access link	Pointer untuk mengakses variabel non-lokal pada nested scope.
Saved machine status	Register atau informasi mesin yang perlu dipulihkan.
Local data dan temporaries	Variabel lokal dan nilai sementara hasil komputasi.

Tabel 22: Isi umum activation record

Code generator harus mengetahui layout activation record agar dapat menghasilkan alamat yang benar untuk parameter dan variabel lokal.

8.2.19 Heap dan Garbage Collection

Inti Konsep.

Heap menyimpan objek dinamis yang umur hidupnya dapat melampaui fungsi pembuatnya. Garbage collection membebaskan objek yang tidak lagi reachable dari root set.

Tidak semua data cocok disimpan di stack. Objek yang dibuat dengan mekanisme seperti ‘new’ dapat tetap hidup setelah fungsi pembuatnya selesai. Data seperti ini ditempatkan di heap.

Masalah heap adalah deallocation. Jika objek yang tidak lagi dipakai tidak dibebaskan, terjadi memory leak. Pada bahasa dengan automatic memory management, garbage collector mencari objek yang masih dapat dijangkau dari **root set**, misalnya register, variabel global, dan variabel pada stack.

Sumber menyebut algoritma **mark-and-sweep**. Fase mark menandai objek reachable. Fase sweep menyapu heap dan membebaskan objek yang tidak ditandai.

8.2.20 Basic Block dan Flow Graph

Inti Konsep.

Basic block adalah urutan instruksi linear dengan satu titik masuk dan satu titik keluar. Flow graph merepresentasikan hubungan kontrol antar-basic block.

Code generator dan optimizer lebih mudah bekerja pada basic block daripada instruksi mentah satu per satu. Basic block tidak memiliki cabang masuk di tengah dan tidak memiliki cabang keluar sebelum instruksi terakhir.

Leader biasanya mencakup instruksi pertama, target dari jump, dan instruksi tepat setelah jump. Setiap leader memulai basic block baru.

Flow graph memakai node untuk basic block dan edge untuk kemungkinan aliran kontrol. Jika block dapat melompat atau jatuh lanjut ke block lain, ada edge dari block pertama ke block kedua. Flow graph menjadi dasar analisis loop, liveness, dan data-flow analysis.

8.2.21 Register dan Address Descriptor

Inti Konsep.

Register descriptor mencatat nilai apa yang sedang berada di setiap register, sedangkan address descriptor mencatat lokasi terkini dari setiap variabel. Keduanya membantu code generator menghindari load dan store yang tidak perlu.

Register adalah sumber daya cepat tetapi terbatas. Code generator harus memutuskan nilai mana yang disimpan di register dan kapan nilai harus ditulis kembali ke memori.

Register descriptor menjawab pertanyaan: register R saat ini memuat nilai variabel apa? **Address descriptor** menjawab pertanyaan: nilai variabel x saat ini tersedia di lokasi mana, misalnya register, stack slot, atau memori statik?

Dengan descriptor, compiler dapat menghindari instruksi berulang. Jika nilai sudah ada di register, tidak perlu melakukan load dari memori. Jika nilai di register belum disimpan tetapi akan dibutuhkan nanti, compiler perlu store sebelum register dipakai ulang.

8.2.22 Instruction Selection, Register Allocation, dan Peephole Optimization

Inti Konsep.

Code generation tidak hanya menerjemahkan IR secara literal, tetapi memilih instruksi target, menentukan lokasi nilai di register atau memori, dan membersihkan pola instruksi redundan. Tiga teknik kunci adalah instruction selection, register allocation, dan peephole optimization.

Instruction selection memetakan pola IR ke instruksi target yang tersedia. Misalnya 3AC $x = y + z$ pada mesin single accumulator dapat diterjemahkan menjadi:

```
LDA y
ADD z
STO x.
```

Jika hasil sementara langsung dipakai oleh instruksi berikutnya, code generator dapat menghindari penyimpanan temporary ke memori dan langsung mempertahankan nilai di accumulator.

Contoh sumber NotebookLM untuk ekspresi

$$Z := (X + Y) * 2 - X$$

menunjukkan alasan optimasi accumulator penting:

Belum Dioptimasi	Dioptimasi
LDA X ADD Y STO T1 LDA T1 MUL 2 STO T2 LDA T2 SUB X STO Z	LDA X ADD Y MUL 2 SUB X STO Z

Tabel 23: Optimasi code generation single accumulator

Register allocation menentukan nilai mana yang disimpan dalam register terbatas. Model umum memakai interference graph: node adalah variabel atau temporary, edge berarti dua nilai hidup pada waktu yang sama sehingga tidak boleh memakai register yang sama. Jika graf dapat diwarnai dengan k warna, program dapat memakai k register. Jika tidak, sebagian nilai harus di-spill ke memori.

Peephole optimization memeriksa jendela kecil, biasanya 3–5 instruksi, lalu menghapus atau mengganti pola lokal yang jelas redundan:

- MOV R0, R0 dihapus karena tidak mengubah state.
- STO x diikuti LDA x dapat dihapus jika accumulator masih memuat nilai yang sama.
- Operasi identitas seperti tambah nol atau kali satu dapat disederhanakan.

Teknik ini menghubungkan bagian Compiler dengan Intermediate Code. IR menyediakan operasi sederhana; code generator memilih urutan instruksi target yang benar dan cukup efisien.

8.2.23 Data-Flow Analysis dan Gen-Kill

Inti Konsep.

Data-flow analysis menghitung fakta program pada titik-titik kontrol menggunakan persamaan yang diulang sampai konvergen. Gen-kill menjelaskan fakta yang dibuat dan fakta yang dihapus oleh sebuah basic block.

Optimasi global membutuhkan informasi lintas basic block. Misalnya, untuk menghapus ekspresi berulang, compiler perlu tahu apakah ekspresi tersebut sudah dihitung di semua jalur sebelumnya dan operandnya belum berubah.

Sumber menuliskan bentuk umum fungsi transfer gen-kill:

$$OUT[B] = Gen[B] \cup (IN[B] - Kill[B]).$$

Gen adalah fakta yang dihasilkan oleh block. **Kill** adalah fakta lama yang tidak lagi valid karena block menimpa atau mengubah sesuatu. Persamaan seperti ini diterapkan berulang pada flow graph sampai nilai setiap set stabil.

Data-flow analysis adalah fondasi banyak optimasi machine-independent karena bekerja pada IR dan flow graph, bukan pada detail instruksi hardware tertentu.

8.2.24 Reaching Definitions

Inti Konsep.

Reaching definitions menentukan definisi variabel mana yang mungkin mencapai suatu titik program tanpa ditimpa di sepanjang jalur. Analisis ini bergerak maju dan memakai union pada join point.

Sebuah definisi variabel adalah assignment yang memberi nilai pada variabel. Definisi d mencapai titik p jika ada jalur dari d ke p tanpa definisi lain yang menimpa variabel yang sama.

Analisis ini **forward** karena informasi mengalir dari awal program ke akhir program. Pada percabangan yang bergabung, meet operator yang dipakai adalah union: jika definisi dapat datang dari salah satu jalur, definisi itu dianggap mungkin mencapai titik tersebut.

Reaching definitions berguna untuk optimasi seperti constant propagation dan analisis use-definition chain.

8.2.25 Available Expressions

Inti Konsep.

Available expressions menentukan ekspresi mana yang sudah pasti dihitung sebelumnya dan operandnya belum berubah pada semua jalur menuju titik program. Analisis ini forward dan memakai intersection pada join point.

Ekspresi seperti $x + y$ available pada titik tertentu jika setiap jalur menuju titik itu sudah menghitung $x + y$ dan tidak ada assignment ke x atau y setelah perhitungan tersebut. Karena harus benar pada semua jalur, meet operator-nya adalah intersection.

Informasi ini dipakai untuk global common subexpression elimination. Jika ekspresi sudah available, compiler dapat memakai hasil lama daripada menghitung ulang.

Perbedaan utama dengan reaching definitions adalah sifat kepastiannya. Reaching definitions bertanya “mungkin sampai?”, sedangkan available expressions bertanya “pasti tersedia di semua jalur?”.

8.2.26 Live Variables

Inti Konsep.

Live variables menentukan apakah nilai variabel saat ini masih akan digunakan di masa depan. Analisis ini bergerak mundur dari akhir program dan membantu dead-code elimination serta register allocation.

Variabel disebut live pada titik tertentu jika ada jalur dari titik itu ke penggunaan variabel sebelum variabel tersebut ditimpa. Jika nilai variabel tidak akan digunakan lagi, variabel itu dead.

Analisis ini **backward** karena informasi penggunaan masa depan harus dipropagasikan ke titik sebelumnya. Jika sebuah variabel live di salah satu jalur keluar, maka pada titik sebelum percabangan variabel itu harus dianggap live.

Live-variable analysis membantu compiler membebaskan register lebih awal. Jika nilai dalam register sudah dead, register tersebut dapat dipakai untuk nilai lain tanpa store.

8.3 Algoritma dan Rumus Penting

8.3.1 Gen-Kill Transfer Function

Makna Rumus.

Rumus gen-kill menyatakan fakta keluar dari sebuah block sebagai fakta yang dihasilkan block ditambah fakta masuk yang tidak dibunuh oleh block.

$$OUT[B] = Gen[B] \cup (IN[B] - Kill[B]) \quad (11)$$

dengan:

- $IN[B]$: fakta data-flow sebelum masuk block B .
- $OUT[B]$: fakta data-flow setelah keluar dari block B .
- $Gen[B]$: fakta yang dibuat oleh block B .
- $Kill[B]$: fakta lama yang dibuat tidak valid oleh block B .

8.3.2 Pseudocode Iterative Data-Flow Analysis

Tujuan Algoritma.

Algoritma iteratif menghitung persamaan data-flow pada flow graph sampai tidak ada perubahan lagi. Pola ini digunakan untuk reaching definitions, available expressions, live variables, dan analisis sejenis.

Listing 5: Pseudocode iterative data-flow analysis

```
1 Input: flow graph CFG, fungsi transfer tiap basic block
2 Output: IN dan OUT stabil untuk setiap block
3
4 1. Inisialisasi IN[B] dan OUT[B] untuk semua block B.
5 2. repeat
6 3.     changed = false
7 4.     for setiap block B:
8 5.         old = OUT[B]
9 6.         IN[B] = meet(OUT[P] untuk setiap predecessor P dari B)
10 7.         OUT[B] = transfer_B(IN[B])
11 8.         if OUT[B] != old:
```

```
12 9.         changed = true
13 10. until changed == false
```

Untuk analisis backward seperti live variables, arah predecessor/successor dibalik dan persamaan menghitung informasi dari titik keluar menuju titik masuk.

9 Recursive Descent Parser

Recursive Descent Parser adalah implementasi langsung dari top-down parsing menggunakan sekumpulan fungsi rekursif. Setiap nonterminal pada grammar diterjemahkan menjadi satu fungsi, dan body produksi diterjemahkan menjadi urutan pemanggilan fungsi nonterminal atau pencocokan token terminal. Karena bentuknya dekat dengan kode program, RD Parser sangat cocok untuk memahami hubungan antara CFG dan implementasi parser.

Materi ini penting sebagai jembatan dari teori grammar menuju compiler nyata. Fokus ujian biasanya mencakup cara kerja fungsi parser, backtracking, keterbatasan akibat left recursion, transformasi grammar, dan hubungan RD Parser dengan predictive parsing serta syntax-directed translation.

9.1 Outline Fundamental Konsep Materi

1. **WAJIB** Dasar top-down parsing
 - (a) **WAJIB** Parse tree dibangun dari root ke leaves
 - (b) **WAJIB** Input dibaca kiri ke kanan
 - (c) **WAJIB** Relasi dengan leftmost derivation
2. **WAJIB** Definisi recursive descent
 - (a) **WAJIB** Satu prosedur untuk setiap nonterminal
 - (b) **WAJIB** Prosedur start symbol sebagai titik awal
 - (c) **WAJIB** Terminal dicocokkan dengan lookahead
3. **WAJIB** Terminologi implementasi
 - (a) **WAJIB** Token
 - (b) **WAJIB** Lookahead
 - (c) **WAJIB** Pointer input **next**
 - (d) **WAJIB** Fungsi pencocokan terminal
4. **WAJIB** Algoritma dasar
 - (a) **WAJIB** Pilih produksi $A \rightarrow X_1 X_2 \cdots X_k$
 - (b) **WAJIB** Jika X_i nonterminal, panggil prosedurnya
 - (c) **WAJIB** Jika X_i terminal, cocokkan dengan input
 - (d) **WAJIB** Jika gagal, laporkan error atau backtrack
5. **WAJIB** Backtracking
 - (a) **WAJIB** Simpan posisi input sebelum mencoba produksi
 - (b) **WAJIB** Reset pointer saat produksi gagal
 - (c) **PAHAM** Inefisiensi eksplorasi cabang
 - (d) **PAHAM** Keterbatasan implementasi boolean sederhana
6. **PAHAM** Brute-force parsing
 - (a) **PAHAM** Ekspansi nonterminal secara coba-coba
 - (b) **PAHAM** Backup posisi stack dan input
 - (c) **PAHAM** Perbandingan dengan recursive parsing

7. **WAJIB** Masalah left recursion
 - (a) **WAJIB** Rekursi kiri langsung
 - (b) **PAHAM** Rekursi kiri tidak langsung
 - (c) **PAHAM** General left recursion melalui rantai nonterminal
 - (d) **WAJIB** Infinite loop sebelum input dikonsumsi
8. **WAJIB** Transformasi grammar
 - (a) **WAJIB** Eliminasi left recursion
 - (b) **WAJIB** Bentuk umum eliminasi left recursion
 - (c) **PAHAM** Algoritma eliminasi left recursion tidak langsung
 - (d) **WAJIB** Left factoring
9. **PAHAM** Predictive parsing
 - (a) **PAHAM** Recursive descent deterministik
 - (b) **PAHAM** Tanpa backtracking
 - (c) **PAHAM** Pemilihan produksi dengan lookahead
 - (d) **PAHAM** Dependensi FIRST dan FOLLOW
10. **PAHAM** Recursive descent untuk SDT
 - (a) **PAHAM** Inherited attributes sebagai parameter fungsi
 - (b) **PAHAM** Synthesized attributes sebagai return value
 - (c) **PAHAM** Semantic actions dalam prosedur
 - (d) **PAHAM** On-the-fly code generation

9.2 Penjelasan Konsep

9.2.1 Top-Down Parsing

Inti Konsep.

Top-down parsing membangun parse tree dari start symbol sebagai root menuju terminal sebagai leaves. Proses ini ekuivalen dengan mencari leftmost derivation untuk token input.

Parser menerima token stream dari lexical analyzer dan menentukan apakah token tersebut sesuai dengan grammar bahasa. Pada top-down parsing, konstruksi dimulai dari start symbol. Parser memilih produksi untuk mengekspansi nonterminal, lalu terus menurunkan sampai terminal yang dihasilkan cocok dengan token input.

Input dipindai dari kiri ke kanan. Karena parser selalu berusaha mencocokkan terminal paling kiri yang belum dibaca, strategi top-down berhubungan erat dengan **leftmost derivation**, yaitu derivasi yang selalu menurunkan nonterminal paling kiri terlebih dahulu.

Relasi ini penting karena menjelaskan mengapa fungsi recursive descent biasanya mengikuti struktur grammar secara langsung. Setiap kali fungsi nonterminal dipanggil, parser sedang mencoba membangun bagian parse tree dari nonterminal tersebut.

9.2.2 Definisi Recursive Descent Parser

Inti Konsep.

Recursive Descent Parser adalah top-down parser yang direalisasikan sebagai kumpulan fungsi rekursif. Setiap nonterminal grammar memiliki satu fungsi, dan pemanggilan fungsi mengikuti struktur produksi grammar.

Misalkan grammar memiliki nonterminal E , T , dan F . RD Parser akan memiliki fungsi seperti $E()$, $T()$, dan $F()$. Fungsi untuk start symbol dipanggil pertama. Jika fungsi tersebut sukses dan seluruh input habis dikonsumsi, parsing berhasil.

Jika body produksi berisi nonterminal, fungsi parser memanggil fungsi nonterminal tersebut. Jika body produksi berisi terminal, fungsi parser mencocokkan terminal tersebut dengan token lookahead. Dengan cara ini, grammar seolah diterjemahkan langsung menjadi kode.

Kelebihan RD Parser adalah mudah dibaca dan mudah ditulis manual untuk grammar sederhana. Kelemahannya, grammar harus disiapkan agar tidak menyebabkan backtracking berlebihan atau infinite loop.

9.2.3 Token, Lookahead, dan Pointer Input

Inti Konsep.

RD Parser bekerja pada token stream, bukan character stream. Token berikutnya yang akan dibaca disebut lookahead dan sering diakses melalui pointer global seperti `next`.

Token adalah unit sintaksis yang dihasilkan scanner, misalnya `INT`, `OPEN`, `CLOSE`, `PLUS`, dan `TIMES`. Parser tidak perlu mengetahui karakter asli yang membentuk token; parser hanya peduli urutan token.

`**Lookahead**` adalah token yang sedang dilihat parser untuk memutuskan apakah terminal tertentu cocok atau produksi tertentu harus dipilih. Dalam contoh sumber, lookahead direpresentasikan oleh pointer `next`. Jika terminal cocok, pointer dimajukan ke token berikutnya.

Istilah	Peran
Token	Simbol terminal abstrak yang diterima parser dari lexer.
Lookahead	Token berikutnya yang sedang diamati untuk membuat keputusan parsing.
<code>next</code>	Pointer input yang menunjuk token yang belum dikonsumsi.
<code>term(tok)</code>	Fungsi pembantu untuk mencocokkan terminal dan memajukan pointer.

Tabel 24: Komponen implementasi RD Parser

Komponen ini membuat parser dapat ditulis sebagai fungsi boolean: fungsi mengembalikan `true` jika bagian grammar berhasil dikenali, dan `false` jika gagal.

9.2.4 Algoritma Prosedur Nonterminal

Inti Konsep.

Fungsi untuk nonterminal A mencoba produksi $A \rightarrow X_1 X_2 \cdots X_k$. Setiap simbol pada body produksi diproses berurutan: nonterminal dipanggil sebagai fungsi, terminal dicocokkan dengan lookahead.

Untuk sebuah produksi $A \rightarrow X_1 X_2 \cdots X_k$, algoritma konseptualnya adalah:

1. Proses X_1 sampai X_k dari kiri ke kanan.

2. Jika X_i adalah nonterminal, panggil fungsi $X_i()$.
3. Jika X_i adalah terminal, bandingkan dengan token lookahead.
4. Jika cocok, konsumsi token dan lanjut.
5. Jika tidak cocok, produksi gagal.

Jika nonterminal A memiliki beberapa alternatif produksi, parser harus menentukan alternatif mana yang dipakai. RD Parser umum dapat mencoba alternatif satu per satu dengan backtracking. Predictive RD Parser memilih alternatif secara deterministik dari lookahead.

Algoritma ini adalah implementasi langsung dari leftmost derivation: fungsi yang sedang berjalan mewakili nonterminal paling kiri yang sedang diturunkan.

9.2.5 Brute-Force Parsing dan Recursive Parsing

Inti Konsep.

Brute-force parsing mencoba ekspansi nonterminal dari kiri secara coba-coba dan mundur ketika pilihan gagal. Recursive descent lebih terstruktur karena nonterminal direpresentasikan sebagai prosedur yang mengembalikan sukses atau gagal.

Sumber baru membedakan brute-force parsing dan recursive parsing. Pada **brute-force parsing**, parser memperluas nonterminal paling kiri dengan mencoba alternatif produksi. Jika string yang dibentuk tidak cocok dengan input, parser harus melakukan backup terhadap posisi input dan riwayat stack, lalu mencoba alternatif lain. Cara ini benar secara konseptual untuk grammar sederhana, tetapi boros waktu dan memori karena banyak cabang gagal harus disimpan atau dikunjungi ulang.

Pada **recursive descent parsing**, struktur grammar dipetakan ke prosedur. Setiap nonterminal menjadi fungsi yang mengontrol alur parsing dengan pemanggilan fungsi lain dan pencocokan terminal. Fungsi mengembalikan nilai benar atau salah, sehingga proses coba-coba menjadi lebih mudah diorganisasi dalam kode.

Keduanya tetap termasuk top-down parsing. Karena itu, keduanya bermasalah terhadap left recursion: jika grammar memiliki produksi $A \rightarrow A\alpha$, parser akan menurunkan sisi kiri tanpa mengonsumsi terminal dan dapat loop tanpa akhir.

9.2.6 Terminal Matching

Inti Konsep.

Terminal matching adalah operasi memeriksa apakah lookahead sama dengan terminal yang diharapkan. Jika cocok, parser mengonsumsi token; jika tidak cocok, parsing cabang tersebut gagal.

Sumber memberi fungsi pembantu:

Listing 6: Fungsi terminal matching

```
1 bool term(TOKEN tok) {
2     return *next++ == tok;
3 }
```

Fungsi ini ringkas, tetapi perlu dibaca hati-hati. Ia membandingkan token yang ditunjuk `next` dengan `tok`, lalu memajukan pointer. Dalam implementasi yang lebih aman, pointer biasanya hanya dimajukan jika token cocok; jika tidak cocok, posisi input harus dikembalikan atau kegagalan ditangani oleh mekanisme backtracking.

Komentar 9.1: Catatan: Sumber Pengetahuan Umum

Dalam parser produksi nyata, fungsi terminal matching biasanya juga menghasilkan pesan error yang informatif, misalnya token apa yang diharapkan dan token apa yang ditemukan. Contoh sederhana dari sumber lebih menekankan ide parsing daripada kualitas diagnostik error.

9.2.7 Backtracking

Inti Konsep.

Backtracking memungkinkan RD Parser mencoba alternatif produksi lain setelah suatu pilihan gagal. Parser menyimpan posisi input sebelum mencoba produksi, lalu mengembalikannya jika produksi tersebut gagal.

Backtracking diperlukan ketika parser belum dapat menentukan alternatif produksi yang benar dari lookahead saat ini. Sebelum mencoba produksi, parser menyimpan pointer input:

```
TOKEN *save = next;
```

Jika produksi gagal, parser melakukan:

```
next = save;
```

sehingga alternatif berikutnya dicoba dari posisi input yang sama.

Tanpa reset pointer, kegagalan cabang pertama dapat merusak parsing cabang berikutnya karena sebagian token sudah terlanjur dikonsumsi. Karena itu, save dan reset adalah inti RD Parser nondeterministik.

Kelemahan backtracking adalah inefisiensi. Parser dapat mencoba banyak cabang yang akhirnya gagal, bahkan mengulangi pekerjaan yang sama. Untuk grammar bahasa pemrograman, compiler biasanya menghindari backtracking dengan grammar predictive atau parser generator.

9.2.8 Contoh Fungsi E dan T

Inti Konsep.

Contoh fungsi E dan T menunjukkan cara grammar diterjemahkan menjadi fungsi boolean. Alternatif produksi dicoba dengan operator OR dan setiap produksi dicoba dari posisi input yang sama.

Misalkan grammar:

$$\begin{aligned} E &\rightarrow T \mid T + E, \\ T &\rightarrow \text{int} \mid \text{int} * T \mid (E). \end{aligned}$$

Implementasi RD Parser dengan backtracking dapat ditulis:

Listing 7: Recursive descent parser dengan backtracking

```
1 bool E1() { return T(); }
2 bool E2() { return T() && term(PLUS) && E(); }
3
4 bool E() {
5     TOKEN *save = next;
6     return (next = save, E1())
7         || (next = save, E2());
8 }
9
10 bool T1() { return term(INT); }
11 bool T2() { return term(INT) && term(TIMES) && T(); }
```

```

12 bool T3() { return term(OPEN) && E() && term(CLOSE); }
13
14 bool T() {
15     TOKEN *save = next;
16     return (next = save, T1())
17         || (next = save, T2())
18         || (next = save, T3());
19 }

```

Pola ini mudah dipahami, tetapi tidak selalu efisien. Misalnya $T1()$ dan $T2()$ sama-sama diawali INT; parser dapat mencoba $T1()$ lebih dulu, sukses sebagian, lalu baru menyadari struktur lebih panjang mungkin dibutuhkan. Masalah seperti ini memotivasi left factoring dan predictive parsing.

9.2.9 Left Recursion

Inti Konsep.

Left recursion terjadi ketika nonterminal dapat menurunkan dirinya sendiri sebagai simbol paling kiri. Pada RD Parser, ini menyebabkan fungsi memanggil dirinya sendiri tanpa mengonsumsi token, sehingga terjadi infinite loop.

Produksi langsung seperti

$$A \rightarrow A\alpha$$

berbahaya untuk RD Parser. Fungsi $A()$ akan mencoba mengenali A dengan memanggil $A()$ lagi sebelum ada terminal yang dikonsumsi. Akibatnya, rekursi tidak pernah bergerak maju pada input.

Contoh klasik:

$$E \rightarrow E + T \mid T.$$

Jika diterjemahkan langsung, fungsi $E()$ akan memanggil $E()$ untuk alternatif pertama, lalu memanggil $E()$ lagi, dan seterusnya.

Komentar 9.2: Catatan: Sumber Pengetahuan Umum

Left recursion dapat langsung atau tidak langsung. Rekursi kiri tidak langsung terjadi misalnya ketika $A \Rightarrow^+ A\alpha$ melalui beberapa nonterminal, bukan melalui produksi $A \rightarrow A\alpha$ secara langsung. RD Parser tetap bermasalah karena pada akhirnya ada rantai fungsi yang kembali ke fungsi awal tanpa konsumsi input.

Karena itu, grammar untuk RD Parser harus bebas dari left recursion.

9.2.10 Eliminasi Left Recursion

Inti Konsep.

Eliminasi left recursion mengubah rekursi kiri menjadi rekursi kanan. Bentuk $A \rightarrow A\alpha \mid \beta$ diganti menjadi $A \rightarrow \beta A'$ dan $A' \rightarrow \alpha A' \mid \epsilon$.

Untuk bentuk sederhana:

$$A \rightarrow A\alpha \mid \beta,$$

dengan β tidak diawali A , transformasinya:

$$\begin{aligned}
 A &\rightarrow \beta A', \\
 A' &\rightarrow \alpha A' \mid \epsilon.
 \end{aligned}$$

Bahasa yang dihasilkan tetap sama, tetapi struktur derivasi menjadi cocok untuk top-down parsing.

Bentuk umum:

$$A \rightarrow A\alpha_1 \mid \cdots \mid A\alpha_n \mid \beta_1 \mid \cdots \mid \beta_m$$

diubah menjadi:

$$\begin{aligned} A &\rightarrow \beta_1 A' \mid \cdots \mid \beta_m A', \\ A' &\rightarrow \alpha_1 A' \mid \cdots \mid \alpha_n A' \mid \epsilon. \end{aligned}$$

Contoh:

$$E \rightarrow E + T \mid T$$

menjadi:

$$\begin{aligned} E &\rightarrow TE', \\ E' &\rightarrow +TE' \mid \epsilon. \end{aligned}$$

Transformasi ini sering muncul dalam soal karena mahasiswa harus mampu mengubah grammar sebelum menulis RD Parser.

Komentar 9.3: Catatan: General Left Recursion

Untuk grammar dengan banyak nonterminal $A_1, A_2, \dots, A_n$, rekursi kiri tidak langsung dapat dieliminasi dengan urutan sistematis. Algoritma formalnya:

1. Urutkan nonterminal menjadi $A_1, A_2, \dots, A_n$.
2. Untuk $i = 1$ sampai n :
 - (a) Untuk $j = 1$ sampai $i - 1$, ganti setiap produksi $A_i \rightarrow A_j \gamma$ dengan produksi $A_i \rightarrow \delta_1 \gamma \mid \cdots \mid \delta_k \gamma$, dengan $A_j \rightarrow \delta_1 \mid \cdots \mid \delta_k$ adalah semua alternatif A_j saat itu.
 - (b) Setelah semua awalan A_j dengan $j < i$ diganti, hilangkan rekursi kiri langsung pada A_i memakai transformasi $A_i \rightarrow \beta A'_i$ dan $A'_i \rightarrow \alpha A'_i \mid \epsilon$.

Proses ini mengubah rantai seperti $A \Rightarrow B\alpha \Rightarrow A\beta\alpha$ menjadi bentuk yang tidak memanggil nonterminal awal sebelum konsumsi input.

Contoh dua nonterminal:

$$A_1 \rightarrow A_2 a \mid b, \quad A_2 \rightarrow A_1 c \mid d.$$

Saat memproses A_2 , produksi $A_2 \rightarrow A_1 c$ diganti memakai alternatif A_1 :

$$A_2 \rightarrow A_2 a c \mid b c \mid d.$$

Sekarang rekursi kiri pada A_2 sudah langsung, sehingga dapat dihilangkan:

$$\begin{aligned} A_2 &\rightarrow b c A'_2 \mid d A'_2, \\ A'_2 &\rightarrow a c A'_2 \mid \epsilon. \end{aligned}$$

Grammar hasil tidak lagi membuat recursive descent memanggil A_2 kembali sebelum ada terminal yang dikonsumsi.

9.2.11 Left Factoring

Inti Konsep.

Left factoring menunda keputusan produksi ketika beberapa alternatif memiliki prefix yang sama. Transformasi ini membuat parser dapat memilih produksi berdasarkan lookahead tanpa mencoba cabang yang sama berulang.

Jika grammar memiliki:

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2,$$

parser tidak dapat memilih alternatif hanya dengan melihat prefix α , karena keduanya diawali sama. Left factoring mengubahnya menjadi:

$$\begin{aligned} A &\rightarrow \alpha A', \\ A' &\rightarrow \beta_1 \mid \beta_2. \end{aligned}$$

Transformasi ini tidak mengubah bahasa yang dihasilkan, tetapi mengubah struktur grammar agar keputusan parsing ditunda sampai token yang membedakan alternatif muncul.

Left factoring sangat penting untuk predictive parsing, karena parser deterministik tidak boleh mencoba satu alternatif lalu mundur.

9.2.12 Predictive Parsing

Inti Konsep.

Predictive parsing adalah varian RD Parser yang deterministik dan tidak memakai backtracking. Produksi dipilih dari lookahead dengan bantuan FIRST dan FOLLOW.

Pada predictive parsing, fungsi nonterminal tidak mencoba semua alternatif dengan OR. Fungsi memakai struktur **if-else** atau **switch** berdasarkan token lookahead.

Jika ada alternatif $A \rightarrow \alpha \mid \beta$, parser dapat memilih α ketika lookahead berada di $FIRST(\alpha)$, dan memilih β ketika lookahead berada di $FIRST(\beta)$. Agar tidak ambigu, FIRST dari alternatif-alternatif harus saling lepas.

Jika ada produksi nullable seperti $A \rightarrow \epsilon$, FOLLOW digunakan untuk menentukan kapan A boleh dikosongkan. Jika lookahead berada di $FOLLOW(A)$, parser dapat memilih produksi ϵ karena secara sintaksis nonterminal A boleh selesai pada titik itu.

Predictive parsing menjadi pengantar langsung ke LL(1) Parser, yang akan dibahas pada section berikutnya.

9.2.13 Recursive Descent untuk Syntax-Directed Translation

Inti Konsep.

RD Parser dapat diperluas menjadi syntax-directed translator dengan membawa atribut melalui parameter dan return value. Inherited attributes menjadi argumen fungsi, sedangkan synthesized attributes menjadi nilai kembalian.

Pada grammar beratribut-L, evaluasi atribut cocok dengan traversal top-down. ****Inherited attribute**** dapat diteruskan dari fungsi parent ke fungsi child sebagai parameter. ****Synthesized attribute**** dapat dikembalikan oleh fungsi child sebagai return value.

Semantic actions dapat disisipkan di antara pemanggilan fungsi parsing. Misalnya setelah mengenali ekspresi, parser dapat langsung mencetak instruksi intermediate code atau mengisi node AST.

Sumber menekankan ****on-the-fly code generation****. Jika output berupa rangkaian kode panjang, parser tidak harus menggabungkan string besar dari seluruh subtree. Ia dapat langsung memancarkan fragmen kode saat semantic action dieksekusi. Ini mengurangi beban memori dan cocok untuk translator sederhana.

Konsep ini menghubungkan RD Parser dengan compiler: parser bukan hanya alat validasi syntax, tetapi juga titik masuk untuk membangun representasi semantik program.

9.3 Algoritma dan Rumus Penting

9.3.1 Skema Umum Fungsi Nonterminal

Tujuan Algoritma.

Skema ini menunjukkan pola umum RD Parser: coba produksi, proses body dari kiri ke kanan, dan kembalikan sukses jika seluruh body cocok.

Listing 8: Skema umum fungsi nonterminal recursive descent

```
1 function parseA():
2     save = next
3
4     for setiap produksi A -> X1 X2 ... Xk:
5         next = save
6         ok = true
7
8         for i = 1 sampai k:
9             if Xi adalah terminal:
10                ok = ok and match(Xi)
11            else:
12                ok = ok and parseXi()
13
14            if not ok:
15                break
16
17        if ok:
18            return true
19
20    next = save
21    return false
```

Skema ini menggambarkan RD Parser dengan backtracking. Pada predictive parser, loop alternatif diganti dengan pemilihan produksi berdasarkan lookahead.

9.3.2 Eliminasi Left Recursion

Makna Rumus.

Rumus ini mengubah grammar yang tidak aman untuk RD Parser menjadi grammar rekursif kanan yang menghasilkan bahasa sama.

$$A \rightarrow A\alpha \mid \beta \quad (12)$$

$$A \rightarrow \beta A' \quad (13)$$

$$A' \rightarrow \alpha A' \mid \epsilon \quad (14)$$

dengan:

- A : nonterminal yang left-recursive.
- α : suffix setelah pemanggilan rekursif kiri.
- β : alternatif yang tidak diawali A .
- A' : nonterminal baru yang menyimpan pengulangan suffix.

9.3.3 Left Factoring

Makna Rumus.

Rumus left factoring mengekstrak prefix bersama agar keputusan alternatif dapat ditunda sampai bagian yang berbeda terlihat oleh parser.

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \quad (15)$$

$$A \rightarrow \alpha A' \quad (16)$$

$$A' \rightarrow \beta_1 \mid \beta_2 \quad (17)$$

Transformasi ini menjadi prasyarat penting untuk membuat grammar lebih cocok bagi predictive recursive descent parser.

10 LL(1) Parser

LL(1) Parser adalah predictive top-down parser yang memilih produksi grammar secara deterministik dengan satu token lookahead. Materi ini melanjutkan Recursive Descent Parser: jika RD Parser umum masih dapat memakai backtracking, LL(1) menghilangkan backtracking dengan bantuan himpunan FIRST, FOLLOW, dan tabel parsing.

Fokus utama untuk ujian adalah memahami arti LL(1), menghitung FIRST dan FOLLOW, membangun tabel parsing, mengenali konflik grammar, serta menjalankan algoritma parser berbasis stack. Secara praktis, LL(1) memperlihatkan bagaimana teori CFG diubah menjadi prosedur parsing yang mekanis dan efisien.

10.1 Outline Fundamental Konsep Materi

1. **WAJIB** Definisi LL(1)
 - (a) **WAJIB** Left-to-right scan
 - (b) **WAJIB** Leftmost derivation
 - (c) **WAJIB** Satu simbol lookahead
2. **WAJIB** Hubungan dengan top-down parsing
 - (a) **WAJIB** Predictive parsing
 - (b) **WAJIB** Recursive descent tanpa backtracking
 - (c) **WAJIB** Pemilihan produksi unik
3. **WAJIB** FIRST set
 - (a) **WAJIB** FIRST untuk terminal
 - (b) **WAJIB** FIRST untuk nonterminal
 - (c) **WAJIB** FIRST untuk string simbol grammar
 - (d) **WAJIB** Peran ϵ
4. **WAJIB** FOLLOW set
 - (a) **WAJIB** Definisi FOLLOW
 - (b) **WAJIB** Endmarker $\$$
 - (c) **WAJIB** Propagasi FOLLOW melalui produksi
5. **WAJIB** Tabel parsing LL(1)
 - (a) **WAJIB** Sel $M[A, a]$
 - (b) **WAJIB** Aturan pengisian dari FIRST
 - (c) **WAJIB** Aturan pengisian dari FOLLOW untuk produksi nullable
 - (d) **WAJIB** Sel kosong sebagai error
6. **WAJIB** Kondisi grammar LL(1)
 - (a) **WAJIB** Tidak ada multiply defined entries
 - (b) **WAJIB** Konflik akibat ambiguitas
 - (c) **WAJIB** Konflik akibat left recursion
 - (d) **WAJIB** Konflik akibat belum left-factored
7. **WAJIB** Algoritma nonrecursive predictive parsing

- (a) **WAJIB** Stack dan input buffer
 - (b) **WAJIB** Aksi expand
 - (c) **WAJIB** Aksi match
 - (d) **WAJIB** Aksi accept
 - (e) **WAJIB** Aksi error
8. **PAHAM** Transformasi grammar untuk LL(1)
- (a) **PAHAM** Eliminasi left recursion
 - (b) **PAHAM** Left factoring
9. **PAHAM** Error handling dan keterbatasan sumber
- (a) **PAHAM** Panic mode dan **synch** entries
 - (b) **PAHAM** Phrase-level recovery

10.2 Penjelasan Konsep

10.2.1 Definisi LL(1)

Inti Konsep.

LL(1) adalah kelas grammar dan parser top-down yang membaca input dari kiri ke kanan, merekonstruksi leftmost derivation, dan memakai satu token lookahead untuk memilih produksi. Angka 1 berarti keputusan parsing dibuat dari tepat satu simbol input berikutnya.

Motivasi LL(1) adalah menghilangkan tebakan dalam parsing top-down. Pada RD Parser biasa, parser dapat mencoba satu produksi, gagal, lalu kembali untuk mencoba produksi lain. LL(1) menuntut grammar disusun sedemikian rupa sehingga dari pasangan nonterminal di puncak stack dan token lookahead, hanya ada satu produksi yang mungkin.

Nama LL(1) mempunyai tiga bagian:

Bagian	Makna
L pertama	<i>Left-to-right scan</i> , yaitu input token dibaca dari kiri ke kanan.
L kedua	<i>Leftmost derivation</i> , yaitu parser merekonstruksi derivasi yang selalu menurunkan nonterminal paling kiri.
1	Satu token lookahead digunakan untuk memilih produksi.

Tabel 25: Makna notasi LL(1)

Konsep ini berkaitan langsung dengan CPMK karena mahasiswa harus mampu menjelaskan notasi parsing formal dan merancang mekanisme automaton/parser untuk grammar tertentu.

10.2.2 Hubungan dengan Top-Down Parsing

Inti Konsep.

LL(1) adalah bentuk predictive top-down parsing. Parser tetap membangun parse tree dari root ke leaves, tetapi produksi dipilih lewat tabel sehingga tidak perlu backtracking.

Top-down parser memulai dari start symbol, lalu menurunkan nonterminal sampai terminal yang dihasilkan cocok dengan input. Dalam recursive descent biasa, pemilihan produksi dapat berupa trial-and-error. LL(1) membuat pemilihan itu deterministik.

Hubungan antarkonsepnya adalah sebagai berikut:

Konsep	Peran dalam LL(1)
Top-down parsing	Strategi umum: parse tree dibangun dari start symbol menuju token input.
Recursive descent	Implementasi top-down dengan fungsi rekursif; dapat memakai backtracking.
Predictive parsing	Top-down parsing deterministik; produksi dipilih dari lookahead.
LL(1) table-driven parser	Implementasi predictive parsing nonrekursif dengan stack dan tabel.

Tabel 26: Relasi LL(1) dengan keluarga top-down parser

Jika RD Parser menerjemahkan nonterminal menjadi fungsi, LL(1) table-driven parser menerjemahkan keputusan produksi menjadi lookup tabel $M[A, a]$. Keduanya tetap merepresentasikan leftmost derivation.

10.2.3 FIRST Set

Inti Konsep.

$FIRST(\alpha)$ adalah himpunan terminal yang dapat muncul sebagai simbol pertama dari string yang diturunkan dari α . Jika α dapat menurunkan string kosong, maka ϵ juga masuk $FIRST(\alpha)$.

FIRST diperlukan karena parser harus tahu token apa yang mungkin muncul pertama kali jika suatu produksi dipakai. Misalnya jika produksi $A \rightarrow \alpha$ memiliki $id \in FIRST(\alpha)$, maka ketika lookahead adalah id , produksi tersebut mungkin relevan.

Definisi formalnya:

$$FIRST(\alpha) = \{a \in T \mid \alpha \Rightarrow^* aw\} \cup \{\epsilon \mid \alpha \Rightarrow^* \epsilon\}.$$

dengan:

- α : string simbol grammar, dapat berupa terminal, nonterminal, atau gabungannya.
- T : himpunan terminal.
- a : terminal pertama yang mungkin dihasilkan.
- ϵ : string kosong.

Aturan komputasi FIRST:

1. Jika a adalah terminal, maka $FIRST(a) = \{a\}$.
2. Jika ada produksi $A \rightarrow \epsilon$, tambahkan ϵ ke $FIRST(A)$.
3. Untuk produksi $A \rightarrow Y_1 Y_2 \cdots Y_k$, tambahkan $FIRST(Y_1) - \{\epsilon\}$ ke $FIRST(A)$.
4. Jika $\epsilon \in FIRST(Y_1)$, lanjutkan dengan menambahkan $FIRST(Y_2) - \{\epsilon\}$, dan seterusnya.
5. Jika semua Y_i dapat menurunkan ϵ , tambahkan ϵ ke $FIRST(A)$.
6. Ulangi sampai tidak ada himpunan FIRST yang berubah.

FIRST adalah dasar pengisian tabel LL(1): produksi ditempatkan pada kolom terminal yang dapat muncul pertama dari body produksi.

10.2.4 FOLLOW Set

Inti Konsep.

$FOLLOW(A)$ adalah himpunan terminal yang dapat muncul tepat setelah nonterminal A dalam suatu sentential form. $FOLLOW$ dipakai terutama ketika produksi dapat menghasilkan ϵ .

$FOLLOW$ diperlukan karena produksi nullable tidak menghasilkan terminal pertama. Jika $A \rightarrow \epsilon$, parser perlu tahu kapan aman mengosongkan A . Jawabannya diberikan oleh $FOLLOW(A)$: jika lookahead adalah simbol yang boleh muncul setelah A , maka A boleh dihapus.

Definisi formal:

$$FOLLOW(A) = \{a \in T \mid S \Rightarrow^* \alpha A a \beta\}.$$

Simbol $\$$ adalah **endmarker**, yaitu penanda akhir input sekaligus dasar stack. Untuk start symbol S , selalu berlaku:

$$\$ \in FOLLOW(S).$$

Aturan komputasi $FOLLOW$:

1. Masukkan $\$$ ke $FOLLOW(S)$ untuk start symbol S .
2. Untuk produksi $A \rightarrow \alpha B \beta$, tambahkan $FIRST(\beta) - \{\epsilon\}$ ke $FOLLOW(B)$.
3. Untuk produksi $A \rightarrow \alpha B$, tambahkan $FOLLOW(A)$ ke $FOLLOW(B)$.
4. Untuk produksi $A \rightarrow \alpha B \beta$ dengan $\epsilon \in FIRST(\beta)$, tambahkan $FOLLOW(A)$ ke $FOLLOW(B)$.
5. Ulangi sampai fixed point, yaitu tidak ada $FOLLOW$ yang berubah lagi.

$FOLLOW$ menghubungkan posisi nonterminal dalam produksi dengan konteks sintaksis di sekelilingnya. Tanpa $FOLLOW$, parser tidak dapat menangani produksi ϵ secara deterministik.

10.2.5 Tabel Parsing LL(1)

Inti Konsep.

Tabel parsing LL(1) $M[A, a]$ memetakan nonterminal A dan lookahead terminal a ke produksi yang harus dipakai. Sel kosong berarti error, sedangkan sel berisi lebih dari satu produksi berarti konflik.

Tabel parsing adalah pusat LL(1). Baris tabel adalah nonterminal, kolom tabel adalah terminal termasuk $\$$. Ketika puncak stack adalah nonterminal A dan lookahead input adalah a , parser melihat $M[A, a]$.

Aturan pengisian tabel untuk setiap produksi $A \rightarrow \alpha$:

1. Untuk setiap terminal $a \in FIRST(\alpha) - \{\epsilon\}$, masukkan $A \rightarrow \alpha$ ke $M[A, a]$.
2. Jika $\epsilon \in FIRST(\alpha)$, maka untuk setiap $b \in FOLLOW(A)$, masukkan $A \rightarrow \alpha$ ke $M[A, b]$.
3. Semua sel yang tidak terisi dianggap error.

Contoh grammar ekspresi yang sudah cocok untuk predictive parsing:

$$\begin{aligned} E &\rightarrow TX, \\ X &\rightarrow +E \mid \epsilon, \\ T &\rightarrow intY \mid (E), \\ Y &\rightarrow *T \mid \epsilon. \end{aligned}$$

Potongan tabelnya:

	int	+	*	(	)	\$
E	$E \rightarrow TX$			$E \rightarrow TX$		
X		$X \rightarrow +E$			$X \rightarrow \epsilon$	$X \rightarrow \epsilon$
T	$T \rightarrow intY$			$T \rightarrow (E)$		
Y		$Y \rightarrow \epsilon$	$Y \rightarrow *T$		$Y \rightarrow \epsilon$	$Y \rightarrow \epsilon$

Tabel 27: Contoh potongan tabel parsing LL(1)

Tabel ini menggantikan logika **if-else** panjang dalam predictive recursive descent. Keputusan parsing menjadi operasi lookup.

10.2.6 Kondisi Grammar LL(1)

Inti Konsep.

Grammar adalah LL(1) jika tabel parsingnya tidak memiliki sel yang berisi lebih dari satu produksi. Konflik tabel berarti satu token lookahead tidak cukup untuk memilih produksi secara unik.

Sumber menyatakan syarat praktis LL(1): setelah tabel dibangun, tidak boleh ada ****multiply defined entries****, yaitu sel $M[A, a]$ yang menerima lebih dari satu produksi. Jika konflik muncul, grammar tersebut bukan LL(1) dalam bentuknya saat ini.

Penyebab umum konflik:

Penyebab	Dampak pada LL(1)
Ambiguitas grammar	Satu string dapat memiliki lebih dari satu derivasi atau parse tree, sehingga beberapa produksi bersaing.
Left recursion	Parser top-down dapat kembali ke nonterminal yang sama sebelum mengonsumsi input; tabel sering menjadi konflik.
Belum left-factored	Dua alternatif berbagi prefix yang sama, sehingga satu lookahead belum cukup membedakan pilihan.

Tabel 28: Penyebab konflik pada tabel LL(1)

Konflik tidak selalu berarti bahasanya tidak dapat diparse secara deterministik oleh parser lain. Konflik hanya menyatakan bahwa grammar tersebut, dengan strategi LL(1), belum memenuhi syarat satu keputusan unik per pasangan nonterminal dan lookahead.

Komentar 10.1: Catatan: Tiga Syarat Formal LL(1)

Untuk membuktikan grammar G bersifat LL(1) tanpa membangun seluruh tabel, periksa tiga kondisi formal berikut untuk setiap nonterminal A dengan dua alternatif berbeda $A \rightarrow \alpha$ dan $A \rightarrow \beta$:

$$FIRST(\alpha) \cap FIRST(\beta) = \emptyset.$$

Artinya, dua alternatif tidak boleh dapat diawali terminal yang sama. Kedua, paling banyak satu alternatif $A \rightarrow \alpha$ yang nullable, yaitu dapat menurunkan ϵ . Ketiga, jika ada alternatif nullable, maka alternatif lain tidak boleh bersaing dengan FOLLOW nonterminal:

$$\epsilon \in FIRST(\alpha) \Rightarrow FIRST(\beta) \cap FOLLOW(A) = \emptyset \quad \text{untuk setiap alternatif lain } \beta.$$

Dalam soal “buktikan grammar ini LL(1)”, hitung FIRST dan FOLLOW, lalu verifikasi tiga kondisi ini. Jika satu kondisi gagal, tabel parsing akan memiliki multiply defined entry.

10.2.7 Algoritma Nonrecursive Predictive Parsing

Inti Konsep.

Nonrecursive predictive parser memakai stack, input buffer, dan tabel parsing. Puncak stack menunjukkan simbol grammar paling kiri yang harus dicocokkan atau diekspansi.

Parser dimulai dengan stack berisi start symbol di atas endmarker:

$$S \$$$

Input juga diakhiri dengan endmarker:

$$w\$$$

Pada setiap langkah, parser membandingkan puncak stack X dengan lookahead input a :

1. Jika X terminal dan $X = a$, lakukan match: pop X dan majukan input.
2. Jika X terminal tetapi $X \neq a$, parsing error.
3. Jika X nonterminal, baca tabel $M[X, a]$.
4. Jika $M[X, a]$ berisi $X \rightarrow Y_1 Y_2 \cdots Y_k$, pop X , lalu push $Y_k, \dots, Y_2, Y_1$ agar Y_1 berada di puncak stack.
5. Jika $M[X, a]$ kosong, parsing error.
6. Jika stack dan input sama-sama mencapai $\$$, accept.

Empat aksi utama parser adalah:

Aksi	Makna
Expand	Nonterminal di puncak stack diganti oleh body produksi dari tabel.
Match	Terminal di puncak stack cocok dengan lookahead, lalu token dikonsumsi.
Accept	Stack dan input mencapai endmarker secara bersamaan.
Error	Tidak ada produksi valid atau terminal puncak tidak cocok dengan lookahead.

Tabel 29: Aksi dalam nonrecursive predictive parser

Algoritma ini berhubungan dengan leftmost derivation karena simbol di puncak stack selalu merepresentasikan simbol paling kiri yang belum selesai diproses.

10.2.8 Transformasi Grammar untuk LL(1)

Inti Konsep.

Grammar sering perlu ditransformasi sebelum menjadi LL(1). Dua transformasi utama adalah eliminasi left recursion dan left factoring.

Left recursion harus dihilangkan karena top-down parser dapat memanggil atau mengekspansi nonterminal yang sama tanpa kemajuan input. Bentuk umum:

$$A \rightarrow A\alpha \mid \beta$$

diubah menjadi:

$$\begin{aligned} A &\rightarrow \beta A', \\ A' &\rightarrow \alpha A' \mid \epsilon. \end{aligned}$$

Left factoring dipakai ketika beberapa produksi memiliki prefix sama:

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$$

diubah menjadi:

$$\begin{aligned} A &\rightarrow \alpha A', \\ A' &\rightarrow \beta_1 \mid \beta_2. \end{aligned}$$

Contoh dari sumber:

$$T \rightarrow \text{int} \mid \text{int} * T$$

tidak mudah dipilih dengan satu lookahead karena keduanya diawali **int**. Setelah left factoring:

$$\begin{aligned} T &\rightarrow \text{int} Y, \\ Y &\rightarrow *T \mid \epsilon. \end{aligned}$$

Transformasi ini tidak dimaksudkan untuk mengubah bahasa, melainkan mengubah struktur grammar agar keputusan parsing menjadi eksplisit dan deterministik.

10.2.9 Error Handling dan Error Recovery

Inti Konsep.

Error pada LL(1) terjadi ketika sel tabel kosong atau terminal puncak stack tidak cocok dengan lookahead. Sumber baru menambahkan bahwa compiler dapat merespons error dengan recovery agar parsing berlanjut, atau repair yang mencoba memperbaiki token sumber.

LL(1) Parser dapat mendeteksi kesalahan sintaks dengan dua cara utama. Pertama, ketika $M[A, a]$ kosong untuk nonterminal A dan lookahead a , berarti tidak ada produksi yang valid. Kedua, ketika puncak stack adalah terminal t tetapi lookahead bukan t , input tidak sesuai dengan terminal yang diharapkan.

Sumber baru membedakan dua respons:

- ****Error recovery****: parser tidak langsung berhenti total, tetapi berusaha melewati bagian bermasalah agar dapat menemukan error lain.
- ****Error repair****: compiler mencoba memodifikasi token sumber secara konseptual, misalnya menyisipkan token yang hilang atau menghapus token berlebih.

Strategi yang disebut sumber:

Strategi	Makna
Panic mode	Melewati token sampai menemukan delimiter atau token sinkronisasi, misalnya titik koma.
Unit deletion	Membuang unit sintaks utuh, misalnya statement atau block yang bermasalah.
Terminal repair	Menyisipkan terminal yang terlewat atau menghapus terminal yang berlebih.
Context-sensitive repair	Menggunakan informasi semantik, misalnya tipe atau deklarasi, untuk memperkirakan perbaikan.
Spelling repair	Mengoreksi salah ketik token keyword atau identifier secara heuristik.

Tabel 30: Strategi penanganan kesalahan sintaks

Untuk LL(1), panic mode sering dikaitkan dengan FOLLOW(nonterminal) sebagai himpunan sinkronisasi: jika parser gagal pada nonterminal A , token dilewati sampai menemukan token yang masuk FOLLOW(A), lalu parser mencoba melanjutkan dari konteks setelah A .

10.2.10 synch Entries dan Phrase-Level Recovery

Inti Konsep.

synch entries adalah penanda pada tabel LL(1) yang membantu panic-mode recovery melakukan sinkronisasi ulang. Phrase-level recovery lebih lokal: sel error diisi dengan routine perbaikan kecil untuk frasa sintaksis tertentu.

Pada tabel LL(1) biasa, sel kosong berarti error. Untuk recovery, sebagian sel kosong dapat diisi dengan penanda *synch*. Umumnya, untuk setiap nonterminal A , semua terminal dalam $FOLLOW(A)$ ditempatkan sebagai kandidat sinkronisasi. Jika parser sedang memproses A dan lookahead berada pada $FOLLOW(A)$, parser dapat menganggap bagian A hilang atau salah, lalu mem-pop A agar parsing lanjut pada konteks setelah A .

Mekanisme panic mode dengan *synch*:

1. Jika $M[A, a]$ berisi produksi, parser tetap melakukan expand normal.
2. Jika $M[A, a]$ berisi *synch*, parser melaporkan error lalu mem-pop A dari stack.
3. Jika $M[A, a]$ kosong dan bukan *synch*, parser melewati token input a sampai menemukan token yang lebih masuk akal.
4. Jika puncak stack terminal tidak cocok dengan lookahead, parser dapat mem-pop terminal tersebut sebagai token yang dianggap hilang.

Phrase-level recovery berbeda karena parser tidak hanya melewati token. Sel error tertentu pada tabel diisi dengan routine yang mencoba perbaikan lokal, misalnya menyisipkan token yang sering hilang, menghapus token berlebih, atau mengganti token yang salah pada konteks tertentu.

Dummy identifier adalah teknik context-sensitive repair pada fase yang melibatkan symbol table. Jika parser atau semantic analyzer menemukan identifier yang belum dideklarasikan, compiler dapat memasukkan identifier tersebut ke symbol table dengan tipe dummy. Tujuannya bukan membenarkan program, melainkan mencegah satu kesalahan deklarasi menghasilkan rentetan error palsu pada penggunaan berikutnya.

Komentar 10.2: Catatan: Batas Recovery

Error recovery tidak bertujuan menebak program benar secara sempurna. Tujuannya adalah menjaga parser tetap berjalan agar compiler dapat melaporkan beberapa error sekaligus. Recovery yang terlalu agresif dapat menghasilkan pesan error lanjutan yang palsu, sehingga strategi sinkronisasi harus dipilih hati-hati.

Subtopik ini berhubungan langsung dengan tabel parsing: selain produksi dan error, tabel dapat diperkaya dengan informasi sinkronisasi. Ini membuat LL(1) tidak hanya menjadi recognizer, tetapi juga bagian dari compiler yang memberi diagnostik.

10.3 Algoritma dan Rumus Penting

10.3.1 Rumus FIRST

Makna Rumus.

Rumus FIRST menentukan terminal apa yang mungkin muncul pertama dari hasil derivasi suatu string grammar. Rumus ini dipakai saat mengisi tabel parsing untuk produksi non-nullable.

$$FIRST(\alpha) = \{a \in T \mid \alpha \Rightarrow^* aw\} \cup \{\epsilon \mid \alpha \Rightarrow^* \epsilon\} \quad (18)$$

dengan:

- α : string simbol grammar.
- a : terminal pertama yang mungkin muncul.
- w : sisa string terminal atau simbol hasil derivasi.
- ϵ : string kosong apabila α nullable.

10.3.2 Rumus FOLLOW

Makna Rumus.

Rumus FOLLOW menentukan terminal yang dapat muncul setelah suatu nonterminal. FOLLOW dipakai ketika produksi dapat menurunkan ϵ .

$$FOLLOW(A) = \{a \in T \mid S \Rightarrow^* \alpha A a \beta\} \quad (19)$$

dengan:

- A : nonterminal yang sedang dianalisis.
- S : start symbol grammar.
- a : terminal yang dapat muncul tepat setelah A .
- α, β : konteks sentential form di sekitar A .

10.3.3 Algoritma Konstruksi Tabel LL(1)

Tujuan Algoritma.

Algoritma ini mengubah grammar, FIRST, dan FOLLOW menjadi tabel keputusan $M[A, a]$. Grammar LL(1) valid jika tidak ada sel tabel yang menerima lebih dari satu produksi.

Listing 9: Pseudocode konstruksi tabel parsing LL(1)

```

1 Input: Grammar G, FIRST, FOLLOW
2 Output: Parsing table M
3
4 1. Inisialisasi semua sel  $M[A, a]$  sebagai error.
5 2. Untuk setiap produksi  $A \rightarrow \alpha$ :
6 3.   Untuk setiap terminal  $a$  dalam  $FIRST(\alpha) - \{\epsilon\}$ :
7 4.   Masukkan  $A \rightarrow \alpha$  ke  $M[A, a]$ .
8 5.   Jika  $\epsilon$  ada dalam  $FIRST(\alpha)$ :
9 6.   Untuk setiap terminal  $b$  dalam  $FOLLOW(A)$ :
10 7.   Masukkan  $A \rightarrow \alpha$  ke  $M[A, b]$ .
11 8. Jika suatu sel menerima lebih dari satu produksi:
12 9.   Grammar bukan LL(1).
```

Langkah 5–7 adalah bagian yang sering menimbulkan kesalahan. Produksi ϵ tidak ditempatkan pada kolom ϵ , melainkan pada kolom terminal yang dapat mengikuti nonterminal tersebut.

10.3.4 Algoritma Nonrecursive Predictive Parser

Tujuan Algoritma.

Algoritma ini menjalankan parser LL(1) memakai stack eksplisit. Outputnya adalah accept jika input sesuai grammar, atau error jika tidak ada langkah parsing yang valid.

Listing 10: Pseudocode nonrecursive predictive parser

```

1 Input: String token w$, parsing table M, start symbol S
2 Output: ACCEPT atau ERROR
3
4 1. Stack = [S, $] // S berada di puncak stack
5 2. a = token pertama input
6 3. while top(Stack) != $:
7 4.     X = top(Stack)
8 5.     if X adalah terminal:
9 6.         if X == a:
10 7.             pop Stack
11 8.             a = token berikutnya
12 9.         else:
13 10.            ERROR
14 11.     else:
15 12.         if M[X, a] berisi X -> Y1 Y2 ... Yk:
16 13.             pop Stack
17 14.             push Yk, ..., Y2, Y1
18 15.         else:
19 16.            ERROR
20 17. if a == $:
21 18.     ACCEPT
22 19. else:
23 20.     ERROR

```

Push dilakukan terbalik agar Y_1 , simbol paling kiri pada body produksi, berada di puncak stack dan diproses lebih dulu.

10.3.5 Trace Parsing int * int

Tujuan Trace.

Trace menunjukkan bagaimana stack, input, dan tabel bekerja bersama. Setiap baris adalah satu langkah match atau expand.

Untuk grammar:

$$\begin{aligned}
 E &\rightarrow TX, \\
 X &\rightarrow +E \mid \epsilon, \\
 T &\rightarrow intY \mid (E), \\
 Y &\rightarrow *T \mid \epsilon,
 \end{aligned}$$

trace untuk input int * int \$ adalah:

Stack	Input	Aksi
$E \$$	int * int \$	Expand $E \rightarrow TX$
$T X \$$	int * int \$	Expand $T \rightarrow intY$
$int Y X \$$	int * int \$	Match int
$Y X \$$	* int \$	Expand $Y \rightarrow *T$
$* T X \$$	* int \$	Match *
$T X \$$	int \$	Expand $T \rightarrow intY$
$int Y X \$$	int \$	Match int
$Y X \$$	\$	Expand $Y \rightarrow \epsilon$
$X \$$	\$	Expand $X \rightarrow \epsilon$
$\$$	\$	Accept

Tabel 31: Trace parsing LL(1) untuk int * int

Trace ini memperlihatkan dua penggunaan produksi nullable. Y dan X dikosongkan karena lookahead \$ berada pada FOLLOW masing-masing nonterminal sesuai tabel parsing.

11 Intermediate Code

Intermediate Code atau Intermediate Representation (IR) adalah representasi program di antara source code dan target code. Dalam compiler, IR menjadi jembatan antara front end yang memahami bahasa sumber dan back end yang memahami mesin target. Dengan IR, compiler tidak harus dibuat ulang penuh untuk setiap kombinasi bahasa dan arsitektur.

Fokus ujian pada materi ini adalah memahami bentuk-bentuk IR, terutama syntax tree, DAG, three-address code, quadruples, triples, dan indirect triples. Selain itu, mahasiswa perlu mampu menerjemahkan ekspresi dan control flow sederhana ke bentuk kode antara serta membaca representasi IR pada soal.

11.1 Outline Fundamental Konsep Materi

1. **WAJIB** Motivasi Intermediate Representation
 - (a) **WAJIB** Jembatan front end dan back end
 - (b) **WAJIB** Modularitas m bahasa dan n mesin target
 - (c) **WAJIB** Optimasi machine-independent
2. **WAJIB** Posisi IR dalam pipeline compiler
 - (a) **WAJIB** Setelah lexical, syntax, dan semantic analysis
 - (b) **WAJIB** Sebelum optimizer dan code generator
3. **WAJIB** Bentuk IR
 - (a) **WAJIB** Syntax tree atau AST
 - (b) **WAJIB** DAG
 - (c) **PAHAM** Value-number atau hash node pada DAG
 - (d) **WAJIB** Postfix notation
 - (e) **WAJIB** Three-address code
 - (f) **WAJIB** Quadruples
 - (g) **WAJIB** Triples
 - (h) **PAHAM** Indirect triples
4. **WAJIB** Komponen three-address code
 - (a) **WAJIB** Address
 - (b) **WAJIB** Temporary variables
 - (c) **WAJIB** Assignment, unary, binary, jump, dan call
5. **WAJIB** Translasi ekspresi
 - (a) **WAJIB** Ekspresi aritmetika
 - (b) **WAJIB** Assignment
 - (c) **PAHAM** Array
 - (d) **PAHAM** Pointer
6. **WAJIB** Boolean dan control flow
 - (a) **WAJIB** Jumping code
 - (b) **WAJIB** BZ dan BR

- (c) **WAJIB** Short-circuit evaluation
- (d) **WAJIB** If dan while
- 7. **WAJIB** Backpatching
 - (a) **WAJIB** truelist
 - (b) **WAJIB** falselist
 - (c) **WAJIB** nextlist
 - (d) **WAJIB** makelist, merge, dan backpatch
- 8. **PAHAM** Kaitan dengan contoh soal UAS
 - (a) **PAHAM** Uninitialized variable detection
 - (b) **PAHAM** Constant folding dan common subexpression elimination
 - (c) **PAHAM** Loop-invariant code motion dan frequency reduction
 - (d) **PAHAM** Strength reduction dan loop unrolling
 - (e) **PAHAM** Dead code dan unreachable code elimination

11.2 Penjelasan Konsep

11.2.1 Motivasi Intermediate Representation

Inti Konsep.

IR memisahkan detail bahasa sumber dari detail mesin target. Dengan IR, compiler menjadi modular: front end cukup menerjemahkan source program ke IR, sedangkan back end cukup menerjemahkan IR ke kode mesin target.

Tanpa IR, compiler untuk m bahasa dan n arsitektur harus dibuat sebanyak $m \times n$ kombinasi. Dengan IR yang stabil, cukup dibuat m front end dan n back end. Setiap front end menerjemahkan bahasa sumber ke IR yang sama, dan setiap back end menerjemahkan IR ke mesin target.

IR juga membuat optimasi lebih mudah. Optimasi pada source code terlalu terikat pada sintaks bahasa, sedangkan optimasi pada assembly terlalu terikat pada mesin. IR berada di tengah: cukup rendah untuk dianalisis secara mekanis, tetapi cukup tinggi untuk masih menyimpan struktur program.

Konsep ini menghubungkan materi compiler dengan teori bahasa formal. Front end memakai regular language dan CFG untuk menganalisis source program, lalu hasil analisis itu disusun menjadi representasi yang siap dioptimasi atau diterjemahkan.

11.2.2 Posisi IR dalam Pipeline Compiler

Inti Konsep.

IR biasanya dibangkitkan setelah lexical analysis, syntax analysis, dan semantic analysis selesai. Output IR kemudian menjadi input untuk optimizer atau langsung ke code generator.

Pipeline konseptualnya:

Source Code $\rightarrow$ Tokens $\rightarrow$ Syntax Tree $\rightarrow$ Checked Tree $\rightarrow$ IR $\rightarrow$ Target Code.

Lexical analyzer mengubah karakter menjadi token. Parser membentuk parse tree atau syntax tree. Semantic analyzer memeriksa tipe, scope, dan aturan makna lain. Setelah struktur program dianggap valid, intermediate code generator menghasilkan IR.

IR dapat langsung diberikan ke code generator, tetapi compiler modern biasanya menambahkan machine-independent optimizer. Optimizer ini bekerja pada IR tanpa perlu mengetahui bahasa sumber asli atau detail register mesin target.

11.2.3 Syntax Tree dan DAG

Inti Konsep.

Syntax tree merepresentasikan struktur ekspresi sebagai pohon, sedangkan DAG membagikan node untuk subekspresi yang sama. DAG membantu menemukan common subexpression agar komputasi tidak diulang.

Syntax tree atau **Abstract Syntax Tree (AST)** adalah bentuk IR tingkat tinggi. Node internal menyatakan operator, sedangkan leaf menyatakan operand seperti identifier atau konstanta. AST menghapus detail sintaks yang tidak penting, misalnya tanda kurung yang hanya mengatur prioritas.

Directed Acyclic Graph (DAG) mirip AST, tetapi satu node dapat memiliki lebih dari satu parent. Jika ekspresi yang sama muncul berulang, DAG menyimpan satu node yang dipakai bersama. Ini penting untuk optimasi common subexpression.

Representasi	Kelebihan	Keterbatasan
AST	Menyimpan struktur ekspresi secara natural dan mudah dipakai semantic analysis.	Kurang langsung untuk code generation linear.
DAG	Menghindari duplikasi subekspresi yang sama.	Lebih kompleks karena node dapat dipakai bersama.

Tabel 32: Perbandingan AST dan DAG

AST dan DAG biasanya lebih dekat ke front end, sedangkan three-address code lebih dekat ke back end.

11.2.4 Value-Number dan Hash Node pada DAG

Inti Konsep.

Value-number adalah identitas untuk nilai hasil ekspresi pada DAG. Dengan hash node berdasarkan operator dan operand, compiler dapat mengenali subekspresi yang sama tanpa membangun node duplikat.

DAG berguna karena satu hasil komputasi dapat dipakai ulang oleh beberapa parent. Agar reuse ini sistematis, compiler perlu cara menentukan bahwa dua ekspresi menghasilkan nilai yang sama. Salah satu tekniknya adalah memberi **value-number**, yaitu nomor atau identitas kanonik untuk nilai yang dihitung.

Saat membangun DAG untuk ekspresi, compiler dapat memakai tabel hash:

1. Untuk leaf berupa variabel atau konstanta, cari atau buat node leaf dengan value-number terkait.
2. Untuk ekspresi operator, hitung key berdasarkan operator dan value-number operand.
3. Jika key sudah ada di hash table, gunakan node lama.
4. Jika key belum ada, buat node baru dan beri value-number baru.

Contoh: jika ekspresi $b * (-c)$ muncul dua kali dan nilai b maupun c belum berubah, hash key untuk operasi perkalian dengan operand yang sama akan ditemukan kembali. DAG memakai node lama, sehingga optimizer dapat menghindari perhitungan ulang.

Komentar 11.1: Catatan: Batas Validitas

Value-number hanya sah selama operand yang menjadi dasar ekspresi belum didefinisikan ulang. Jika assignment baru mengubah b atau c , ekspresi $b * (-c)$ setelah assignment tersebut tidak boleh otomatis dianggap sama dengan ekspresi sebelumnya.

Teknik ini menghubungkan representasi DAG dengan optimasi common subexpression elimination. DAG bukan sekadar gambar ekspresi, tetapi struktur data yang membantu compiler menemukan nilai yang sama secara mekanis.

11.2.5 Postfix Notation sebagai Intermediate Code

Inti Konsep.

Postfix notation menuliskan operator setelah operand, sehingga ekspresi dapat dievaluasi dengan stack tanpa tanda kurung. Dalam sumber UAS, postfix juga dipakai untuk control flow dengan instruksi branch seperti BZ dan BR.

Pada notasi infix biasa, ekspresi $a + b * c$ membutuhkan aturan prioritas operator. Dalam postfix, ekspresi yang sama ditulis:

$$a \ b \ c \ * \ +.$$

Urutan ini eksplisit: hitung $b * c$ lebih dulu, lalu tambahkan a . Mesin stack atau interpreter sederhana dapat membaca token dari kiri ke kanan; operand di-push, operator mem-pop operand yang diperlukan, lalu hasilnya di-push kembali.

Instruksi	Makna
Operand	Push nilai operand ke stack evaluasi.
Operator aritmetika	Pop operand, hitung hasil, lalu push hasil.
BZ L	Branch if zero/false: lompat ke label L jika nilai kondisi bernilai salah.
BR L	Branch tanpa syarat ke label L .

Tabel 33: Elemen postfix dan branch pada intermediate code

Trace evaluasi stack untuk postfix $3 \ 4 \ + \ 2 \ *$:

Token	Stack Setelah Token	Aksi
3	[3]	Push operand.
4	[3, 4]	Push operand.
+	[7]	Pop 3 dan 4, push 7.
2	[7, 2]	Push operand.
*	[14]	Pop 7 dan 2, push 14.

Tabel 34: Trace stack untuk evaluasi postfix

Untuk control flow, BZ dan BR membuat postfix tidak hanya merepresentasikan ekspresi, tetapi juga aliran kendali. Misalnya pola IF cond THEN $S1$ ELSE $S2$ memakai BZ L_{else} setelah evaluasi kondisi, lalu BR L_{end} setelah bagian then agar eksekusi melompati bagian else.

Komentar 11.2: Catatan: Pola Postfix UAS

Sumber NotebookLM menampilkan pola postfix untuk IF $a > b$ THEN $c := d$ ELSE $c := e$. Struktur instruksinya:

$$a \ b \ > \ L_{else} \ BZ \ c \ d \ := \ L_{end} \ BR \ c \ e \ := \ .$$

Maknanya: evaluasi $a > b$; jika hasilnya false atau zero, BZ melompat ke blok `else`; setelah blok `then`, BR melompati blok `else`.

Untuk `WHILE A < B DO A := A + 1`, pola umumnya:

$$L_{start} : A \ B < \ L_{out} \ BZ \ A \ A + 1 := \ L_{start} \ BR.$$

Label awal loop harus disimpan agar BR dapat kembali mengulang evaluasi kondisi. Kesalahan umum adalah melompat kembali ke awal body, padahal kondisi harus diuji ulang setiap iterasi.

11.2.6 Three-Address Code

Inti Konsep.

Three-address code (3AC) adalah IR linear dengan instruksi sederhana yang umumnya memiliki paling banyak tiga address: dua operand dan satu hasil. Ekspresi kompleks dipecah menjadi urutan operasi kecil.

Instruksi 3AC berbentuk umum:

$$x = y \text{ op } z.$$

Setiap instruksi hanya memuat satu operator utama. Ekspresi panjang seperti

$$a = b * -c + b * -c$$

dipecah menjadi temporary variables:

$$\begin{aligned} t_1 &= -c, \\ t_2 &= b * t_1, \\ t_3 &= -c, \\ t_4 &= b * t_3, \\ a &= t_2 + t_4. \end{aligned}$$

****Address**** adalah operand dalam 3AC. Address dapat berupa nama variabel, konstanta, atau temporary variable seperti $t_1, t_2, \dots$. Temporary dibuat compiler untuk menyimpan hasil antara subekspresi.

Bentuk Instruksi	Makna
$x = y \text{ op } z$	Assignment biner, misalnya $x = y + z$.
$x = \text{op } y$	Assignment unari, misalnya $x = -y$.
$x = y$	Copy instruction.
<code>goto L</code>	Lompat tanpa syarat ke label L .
<code>if x rel op y goto L</code>	Lompat bersyarat.
<code>param x, call p,n</code>	Pengiriman argumen dan pemanggilan prosedur.

Tabel 35: Bentuk umum instruksi three-address code

3AC penting karena mudah dihasilkan dari syntax tree dan cukup dekat dengan instruksi mesin untuk code generation.

11.2.7 Quadruples, Triples, dan Indirect Triples

Inti Konsep.

Quadruples, triples, dan indirect triples adalah cara menyimpan three-address code. Perbedaan terletak pada cara menyimpan hasil instruksi dan cara instruksi dirujuk ulang.

Quadruple menyimpan empat field: **op**, **arg1**, **arg2**, dan **result**. Karena result eksplisit, instruksi mudah dipindahkan saat optimasi.

Triple menyimpan tiga field: **op**, **arg1**, dan **arg2**. Hasil operasi dirujuk melalui nomor instruksi, misalnya hasil baris 1 ditulis sebagai (1). Ini menghemat temporary eksplisit, tetapi menyulitkan pemindahan instruksi karena indeks berubah.

Indirect triple menambahkan daftar pointer ke triple. Instruksi asli tetap berada di daftar triple, sedangkan urutan eksekusi diatur oleh daftar pointer. Optimasi dapat menukar urutan pointer tanpa mengubah nomor triple asli.

Representasi	Struktur	Implikasi Optimasi
Quadruple	op, arg1, arg2, result	Mudah dipindah karena result eksplisit.
Triple	op, arg1, arg2; hasil adalah indeks instruksi.	Sulit dipindah karena referensi indeks dapat berubah.
Indirect triple	Triple ditambah list pointer urutan eksekusi.	Reordering cukup mengubah pointer.

Tabel 36: Representasi implementasi three-address code

Soal ujian sering menguji kemampuan membaca perbedaan ini, terutama bahwa triple memakai referensi seperti (1) dan tidak menyimpan variabel result eksplisit.

11.2.8 Translasi Ekspresi, Array, dan Pointer

Inti Konsep.

Translasi ekspresi memecah perhitungan menjadi instruksi 3AC berurutan. Array dan pointer membutuhkan operasi alamat karena nilai yang diakses bergantung pada lokasi memori runtime.

Untuk ekspresi aritmetika, compiler biasanya membuat temporary baru untuk setiap hasil antara. Misalnya:

$$x = a + b * c$$

diterjemahkan menjadi:

$$\begin{aligned} t_1 &= b * c, \\ x &= a + t_1. \end{aligned}$$

Untuk array, alamat elemen harus dihitung. Jika elemen array memiliki lebar w dan indeks bawah low , offset dasar indeks i adalah:

$$(i - low) * w.$$

Bentuk IR yang dipakai sumber mencakup:

- $x = y[i]$: ambil nilai dari alamat berindeks.
- $x[i] = y$: simpan nilai ke alamat berindeks.

Untuk pointer, IR menyediakan operasi:

- $x = \&y$: ambil alamat y .
- $x = *y$: baca nilai dari alamat yang disimpan di y .
- $*x = y$: tulis nilai y ke alamat yang disimpan di x .

Konsep kuncinya adalah membedakan **lvalue**, yaitu lokasi yang dapat diberi nilai, dan **rvalue**, yaitu nilai yang dihitung dari ekspresi.

11.2.9 Boolean dan Control Flow

Inti Konsep.

Ekspresi boolean dapat diterjemahkan sebagai jumping code, bukan selalu sebagai nilai 0 atau 1. Dengan jumping code, hasil boolean diwujudkan sebagai lompatan ke label benar atau salah.

Untuk ekspresi boolean seperti $a < b$, compiler dapat menghasilkan:

```
if a < b goto Ltrue
goto Lfalse.
```

Pendekatan ini mendukung **short-circuit evaluation**. Pada ekspresi $B_1 || B_2$, jika B_1 sudah true, B_2 tidak perlu dievaluasi. Pada ekspresi $B_1 \&\& B_2$, jika B_1 false, B_2 tidak perlu dievaluasi.

Control flow memakai label:

- $B.true$: tujuan jika kondisi benar.
- $B.false$: tujuan jika kondisi salah.
- $S.next$: instruksi setelah statement selesai.

Contoh postfix dari sumber untuk:

```
IF a > b THEN c := d ELSE c := e
```

memakai instruksi branch seperti **BZ** (*branch if zero*) dan **BR** (*branch*). Jika kondisi salah, BZ melompat ke bagian **else**; setelah bagian **then**, BR melompati bagian **else**.

11.2.10 Backpatching

Inti Konsep.

Backpatching adalah teknik mengisi target instruksi jump yang belum diketahui saat instruksi dibuat. Compiler menyimpan daftar instruksi bolong, lalu mengisinya ketika label tujuan sudah tersedia.

Backpatching diperlukan pada translasi one-pass, terutama untuk boolean dan control flow. Saat compiler menghasilkan instruksi **goto _**, label tujuan mungkin belum diketahui karena bagian program tujuan belum dibaca.

Tiga list utama:

List	Peran
$B.truelist$	Daftar instruksi jump yang harus menuju label ketika boolean B true.
$B.falselist$	Daftar instruksi jump yang harus menuju label ketika boolean B false.
$S.nextlist$	Daftar jump yang harus menuju instruksi setelah statement S .

Tabel 37: List dalam backpatching

Operasi pendukung:

- `makelist(i)`: membuat list berisi instruksi i .
- `merge(p1,p2)`: menggabungkan dua list.
- `backpatch(p,i)`: mengisi target semua instruksi dalam list p dengan label i .

Untuk produksi boolean:

$$B \rightarrow B_1 \parallel M B_2,$$

instruksi pada B_1 .*falselist* di-backpatch ke $M.instr$, yaitu posisi awal evaluasi B_2 . Truelist hasilnya adalah gabungan B_1 .*truelist* dan B_2 .*truelist*.

11.2.11 Kaitan dengan Contoh Soal UAS

Inti Konsep.

Sumber UAS menekankan kemampuan membaca dan mengubah representasi IR, terutama postfix, quadruple, triple, serta optimasi sederhana dari kode antara menuju assembly.

Berdasarkan inventaris sumber, UAS 2425 memuat beberapa pola aplikasi Intermediate Code:

- Membaca control flow seperti while loop dalam bentuk postfix, quadruple, dan triple.
- Memahami bahwa triple merujuk hasil instruksi sebelumnya dengan nomor baris seperti (1).
- Mengubah 3AC atau quadruple menjadi assembly single accumulator seperti LDA, ADD, dan STO.
- Mengenali optimasi sederhana seperti dead code elimination, unreachable code, dan constant folding.

Artinya, penguasaan materi tidak cukup berhenti pada definisi IR. Mahasiswa perlu dapat melakukan translasi kecil secara manual dan membaca tabel representasi kode antara.

11.2.12 Optimasi dan Deteksi pada Intermediate Code

Inti Konsep.

IR membuat compiler dapat melakukan deteksi kesalahan dan optimasi sebelum masuk ke detail mesin target. Contohnya adalah deteksi variabel belum diinisialisasi, loop-invariant code motion, frequency reduction, strength reduction, dan loop unrolling.

Karena IR menampilkan aliran data dan aliran kontrol secara eksplisit, compiler dapat menganalisis fakta program lebih mudah daripada pada source code mentah. Analisis ini dapat dipakai untuk memperingatkan kesalahan atau memperbaiki efisiensi kode.

Teknik	Makna pada IR
Uninitialized variable detection	Mendeteksi penggunaan variabel sebelum ada definisi yang pasti mencapai titik penggunaan. Berkaitan dengan reaching definitions.
Constant folding	Mengevaluasi ekspresi konstanta saat compile-time, misalnya $2 + 3$ menjadi 5 atau $x * 1$ menjadi x .
Common subexpression elimination	Menggunakan kembali hasil ekspresi yang sudah dihitung jika operannya belum berubah, misalnya menyimpan $a + b$ ke temporary lalu memakai temporary tersebut pada penggunaan berikutnya.
Loop-invariant code motion	Memindahkan ekspresi yang nilainya tidak berubah di dalam loop ke sebelum loop.
Frequency reduction / code motion	Mengurangi frekuensi eksekusi instruksi mahal dengan memindahkannya ke lokasi yang dieksekusi lebih jarang.
Strength reduction	Mengganti operasi mahal dengan operasi lebih murah, misalnya perkalian indeks loop menjadi penjumlahan berulang.
Loop unrolling	Menggandakan body loop beberapa kali untuk mengurangi overhead branch dan membuka peluang optimasi lain.
Dead code dan unreachable code elimination	Menghapus instruksi yang hasilnya tidak pernah dipakai atau block yang tidak mungkin dieksekusi, misalnya cabang <code>if false</code> .

Tabel 38: Optimasi dan deteksi berbasis intermediate code

Optimasi ini harus menjaga semantik program. Misalnya loop-invariant code motion hanya aman jika ekspresi benar-benar tidak berubah di seluruh iterasi dan pemindahannya tidak mengubah efek samping atau penanganan error.

Subtopik ini menghubungkan Intermediate Code dengan materi Compiler bagian data-flow analysis. IR adalah objek yang dianalisis, sedangkan reaching definitions, available expressions, dan live variables adalah teknik untuk membuktikan kapan transformasi aman.

11.3 Algoritma dan Rumus Penting

11.3.1 Rumus Offset Array

Makna Rumus.

Rumus offset array menghitung jarak memori elemen array dari base address. Rumus ini dipakai saat menerjemahkan akses array ke intermediate code.

$$offset(i) = (i - low)w \quad (20)$$

dengan:

- i : indeks elemen yang diakses.
- low : indeks bawah array.
- w : lebar satu elemen array dalam byte atau word.

Jika indeks array dimulai dari 0, rumus menyederhana menjadi $offset(i) = iw$.

11.3.2 Pseudocode Pembuatan Three-Address Code Ekspresi

Tujuan Algoritma.

Algoritma ini menerjemahkan expression tree menjadi urutan three-address code dengan temporary variables. Setiap operator menghasilkan satu instruksi 3AC.

Listing 11: Pseudocode translasi ekspresi ke three-address code

```
1 function genExpr(node):
2     if node adalah identifier atau konstanta:
3         return node.place
4
5     left = genExpr(node.left)
6     right = genExpr(node.right)
7     temp = newtemp()
8     emit(temp = left op(node) right)
9     return temp
```

Algoritma ini bersifat postorder: anak kiri dan kanan diterjemahkan lebih dulu, lalu operator parent menghasilkan temporary baru.

11.3.3 Pseudocode Konstruksi Tabel Quadruple

Tujuan Algoritma.

Pseudocode ini menunjukkan bagaimana instruksi 3AC disimpan sebagai quadruple. Setiap instruksi dipetakan ke field operator, argumen, dan hasil.

Listing 12: Pseudocode emit quadruple

```
1 function emitBinary(result, arg1, op, arg2):
2     quad.op = op
3     quad.arg1 = arg1
4     quad.arg2 = arg2
5     quad.result = result
6     append quad to instructionList
```

Untuk instruksi unari atau jump, sebagian field dapat kosong. Misalnya goto L dapat disimpan dengan op = goto dan result = L.

11.3.4 Pseudocode Backpatching

Tujuan Algoritma.

Pseudocode backpatching mengisi label tujuan pada instruksi jump yang sebelumnya belum lengkap. Teknik ini menjaga translasi control flow tetap bisa dilakukan satu pass.

Listing 13: Pseudocode operasi dasar backpatching

```
1 function makelist(i):
2     return list berisi i
3
4 function merge(p1, p2):
5     return konkatenasi p1 dan p2
6
7 function backpatch(p, target):
8     for setiap instruksi i dalam p:
9         instruction[i].target = target
```

Backpatching penting untuk boolean short-circuit dan statement bercabang karena target jump sering baru diketahui setelah bagian program berikutnya selesai diproses.

12 Interpreter dan Virtual Machine

Interpreter dan Virtual Machine (VM) membahas cara program dieksekusi setelah melewati analisis bahasa. Jika compiler tradisional menerjemahkan program sumber menjadi kode target sebelum dijalankan, interpreter dan VM mengeksekusi representasi program secara langsung atau melalui bytecode. Materi ini menutup rangkaian compiler: dari source code, token, syntax tree, intermediate code, sampai model eksekusi runtime.

Fokus belajar untuk ujian adalah membedakan compiler, interpreter, VM, bytecode, dan JIT; memahami arsitektur stack-based VM; membaca model memori runtime; serta menghubungkan bytecode dengan intermediate code dan code generation.

12.1 Outline Fundamental Konsep Materi

1. **WAJIB** Definisi dasar
 - (a) **WAJIB** Compiler
 - (b) **WAJIB** Interpreter
 - (c) **WAJIB** Virtual Machine
 - (d) **WAJIB** Bytecode
 - (e) **WAJIB** JIT compiler
 - (f) **PAHAM** HotSpot client dan server compiler
2. **WAJIB** Motivasi model hibrida compiler dan VM
 - (a) **WAJIB** Portabilitas bytecode
 - (b) **WAJIB** Penyederhanaan back end compiler
 - (c) **PAHAM** Verifikasi dan profiling runtime
3. **WAJIB** Posisi dalam language-processing system
 - (a) **WAJIB** Source code ke bytecode
 - (b) **WAJIB** VM sebagai mesin target abstrak
 - (c) **WAJIB** Eksekusi interpretif atau JIT
4. **WAJIB** Arsitektur VM
 - (a) **WAJIB** Stack-based VM
 - (b) **PAHAM** Register-based VM
 - (c) **WAJIB** Instruction set dan opcode
 - (d) **WAJIB** Bytecode strongly typed
5. **WAJIB** Runtime memory model
 - (a) **WAJIB** Code area
 - (b) **WAJIB** Stack
 - (c) **WAJIB** Activation record atau stack frame
 - (d) **WAJIB** Operand stack
 - (e) **WAJIB** Local variables
 - (f) **WAJIB** Heap
6. **WAJIB** Siklus fetch-decode-execute

- (a) **WAJIB** Program counter
 - (b) **WAJIB** Decode opcode
 - (c) **WAJIB** Execute dan update state
7. **PAHAM** Optimasi interpreter VM
- (a) **PAHAM** Threaded code
 - (b) **PAHAM** Top-of-stack caching
 - (c) **PAHAM** Super-instructions
8. **WAJIB** Garbage collection
- (a) **WAJIB** Root set
 - (b) **WAJIB** Reachability
 - (c) **PAHAM** Mark-and-sweep
 - (d) **PAHAM** Reference counting
9. **WAJIB** Relasi dengan intermediate code dan code generation
10. **PAHAM** Kaitan dengan contoh UAS

12.2 Penjelasan Konsep

12.2.1 Compiler, Interpreter, VM, Bytecode, dan JIT

Inti Konsep.

Compiler menerjemahkan program sebelum eksekusi, interpreter mengeksekusi program secara langsung, dan VM mengeksekusi bytecode sebagai mesin abstrak. JIT compiler mempercepat VM dengan mengompilasi bytecode menjadi native code saat runtime.

Materi ini perlu dimulai dari pemisahan istilah karena semuanya sama-sama berhubungan dengan penerjemahan program, tetapi titik kerjanya berbeda.

Istilah	Definisi atau Peran
Compiler	Membaca program sumber dan menerjemahkannya menjadi program target yang ekuivalen sebelum dijalankan.
Interpreter	Mengeksekusi operasi program secara langsung tanpa menghasilkan object code eksternal sebagai output utama.
Virtual Machine	Software interpreter khusus yang bertindak seperti CPU abstrak untuk mengeksekusi instruksi perantara.
Bytecode	Bentuk intermediate representation level rendah yang menjadi input VM; opcode sering berukuran satu byte.
JIT compiler	Compiler runtime yang menerjemahkan bytecode menjadi instruksi mesin native saat program berjalan.

Tabel 39: Perbandingan istilah dasar eksekusi program

Compiler tradisional unggul dalam performa karena kode target sudah dihasilkan sebelum runtime. Interpreter unggul dalam fleksibilitas dan diagnostik karena eksekusi mengikuti struktur program secara langsung. VM mengambil jalan tengah: program dikompilasi ke bytecode portabel, lalu bytecode dieksekusi oleh VM.

12.2.2 HotSpot, Client Compiler, dan Server Compiler

Inti Konsep.

HotSpot adalah pendekatan JIT yang mengamati bagian program yang sering dieksekusi lalu mengompilasinya menjadi native code. Client compiler menekankan waktu startup cepat, sedangkan server compiler menekankan optimasi lebih agresif untuk program berjalan lama.

Interpreter VM dapat langsung menjalankan bytecode, tetapi mengeksekusi semua instruksi secara interpretif sering lambat. JIT memperbaikinya dengan profiling runtime: VM menghitung method atau loop mana yang menjadi **hot spot**, yaitu bagian yang sering dijalankan dan layak dikompilasi.

Mode JIT	Karakteristik
Client compiler	Menghasilkan native code lebih cepat dengan optimasi lebih ringan; cocok untuk startup cepat dan aplikasi interaktif.
Server compiler	Menghabiskan waktu kompilasi lebih banyak untuk optimasi agresif; cocok untuk program yang berjalan lama.
HotSpot profiling	Menggunakan data runtime untuk memilih bagian kode yang paling menguntungkan jika dikompilasi.

Tabel 40: Model HotSpot JIT pada VM

JIT berbasis HotSpot berbeda dari ahead-of-time compiler karena keputusan optimasinya didasarkan pada perilaku aktual saat program berjalan. Misalnya VM dapat melihat cabang mana yang sering dipilih, method mana yang sering dipanggil, atau tipe objek apa yang dominan.

Komentar 12.1: Catatan: Tradeoff JIT

JIT tidak selalu langsung lebih cepat sejak awal. Ada biaya profiling dan kompilasi runtime. Keuntungannya muncul ketika kode yang dikompilasi dipakai cukup sering sehingga penghematan eksekusi lebih besar daripada biaya kompilasinya.

Konsep HotSpot menghubungkan VM dengan optimasi compiler. Perbedaananya, optimizer JIT bekerja dengan informasi runtime yang tidak tersedia pada compiler statis biasa.

12.2.3 Motivasi Model Hibrida Compiler dan VM

Inti Konsep.

Model hibrida compiler dan VM mengompilasi source code ke bytecode portabel, lalu mengeksekusinya di VM. Tujuannya adalah menggabungkan portabilitas interpreter dengan performa yang dapat ditingkatkan melalui JIT.

Bahasa seperti Java dan C# memakai model hibrida. Source code tidak langsung diterjemahkan ke instruksi mesin tertentu, melainkan ke bytecode. Bytecode yang sama dapat berjalan pada sistem operasi dan hardware berbeda selama tersedia VM yang kompatibel.

Keuntungan model ini:

1. Portabilitas: satu bytecode dapat dijalankan di banyak platform.
2. Back end compiler lebih sederhana: compiler menargetkan mesin abstrak, bukan banyak CPU nyata.
3. VM dapat memverifikasi bytecode sebelum eksekusi untuk meningkatkan keamanan.

4. VM dapat melakukan profiling runtime dan memilih bagian panas untuk dikompilasi JIT.

Tradeoff-nya adalah overhead runtime. Jika bytecode hanya diinterpretasi, performanya biasanya lebih rendah daripada native code. JIT mengurangi overhead ini dengan menerjemahkan bagian yang sering dieksekusi menjadi kode mesin.

12.2.4 Posisi Interpreter dan VM dalam Language-Processing System

Inti Konsep.

Dalam sistem pemrosesan bahasa, compiler front end menghasilkan bytecode atau IR level rendah, lalu VM berperan sebagai mesin target abstrak yang mengeksekusi representasi tersebut.

Alur model VM dapat ditulis:

Source Code → Compiler Front End → Bytecode → Virtual Machine → Output.

Front end tetap melakukan lexical analysis, syntax analysis, semantic analysis, dan intermediate code generation. Perbedaannya, code generator tidak harus menghasilkan assembly mesin nyata; ia dapat menghasilkan bytecode untuk VM.

VM berada pada sisi runtime. Ia membaca bytecode, mengelola memori, menjalankan instruksi, menangani call stack, dan berinteraksi dengan layanan sistem seperti I/O atau thread melalui abstraksi yang disediakan runtime.

12.2.5 Stack-Based VM

Inti Konsep.

Stack-based VM mengeksekusi instruksi dengan operand stack. Operand tidak ditulis eksplisit sebagai register pada setiap instruksi; instruksi mengambil nilai dari puncak stack dan mendorong hasil kembali ke stack.

Pada stack-based VM, komputasi seperti $a + b$ biasanya dilakukan dengan urutan:

```
load a, load b, add.
```

Instruksi `load` mendorong nilai ke operand stack. Instruksi `add` mengambil dua nilai teratas, menjumlahkannya, lalu mendorong hasil.

Keuntungan stack-based VM adalah instruction format sederhana. Compiler tidak perlu memilih register virtual untuk setiap operasi karena lokasi operand implisit berada di stack. JVM adalah contoh penting stack-based VM.

Kelemahannya, operasi stack sering menghasilkan lebih banyak instruksi dan akses memori dibanding register-based VM. Karena itu, VM modern memakai optimasi seperti top-of-stack caching untuk menyimpan beberapa elemen puncak stack pada register CPU nyata.

12.2.6 Register-Based VM

Inti Konsep.

Register-based VM memakai register virtual sebagai tempat operand dan hasil. Instruksinya lebih mirip CPU nyata karena operand ditulis eksplisit.

Dalam register-based VM, operasi penjumlahan dapat ditulis seperti:

```
add r3, r1, r2
```

yang berarti $r3 = r1 + r2$. Operand dan result terlihat langsung pada instruksi.

Perbandingan singkat:

Model	Kelebihan	Keterbatasan
Stack-based VM	Instruksi sederhana; compiler bytecode lebih mudah karena operand implisit.	Dapat menghasilkan lebih banyak push/pop dan akses stack.
Register-based VM	Lebih dekat ke CPU nyata; instruksi dapat lebih padat secara jumlah langkah.	Compiler harus mengelola register virtual dan format instruksi lebih kompleks.

Tabel 41: Perbandingan stack-based dan register-based VM

Sumber menyebut Dalvik VM sebagai contoh register-based VM, sedangkan JVM sebagai contoh stack-based VM.

12.2.7 Instruction Set dan Bytecode

Inti Konsep.

Instruction set VM adalah kumpulan opcode yang dipahami VM. Bytecode menyimpan opcode dan operand ringkas sebagai bahasa mesin untuk mesin virtual.

Opcode adalah kode operasi, misalnya load, add, branch, call, atau return. Pada bytecode, opcode sering direpresentasikan sebagai angka 1 byte. Karena itu ada batas praktis jumlah opcode dasar, tetapi VM dapat memakai operand tambahan setelah opcode.

Pada JVM, banyak instruksi bersifat typed. Contohnya:

Instruksi	Makna
<code>iload_0</code>	Memuat local variable integer indeks 0 ke operand stack.
<code>iconst_1</code>	Mendorong konstanta integer 1 ke operand stack.
<code>iadd</code>	Mengambil dua integer dari stack, menjumlahkan, lalu mendorong hasil.
<code>fadd</code>	Operasi penjumlahan untuk floating-point.
<code>imul</code>	Mengalikan dua integer pada puncak stack.
<code>if_icmpgt</code>	Branch jika perbandingan dua integer memenuhi kondisi greater-than.
<code>invokevirtual</code>	Memanggil method virtual.
<code>ireturn</code>	Mengembalikan nilai integer ke pemanggil.

Tabel 42: Contoh instruksi bytecode JVM

Typed instruction membantu verifikasi bytecode karena VM dapat memeriksa apakah operasi integer, floating-point, object reference, dan return value digunakan secara konsisten.

12.2.8 Runtime Memory Model

Inti Konsep.

VM membagi memori runtime menjadi code area, stack, heap, dan frame. Pembagian ini memungkinkan eksekusi fungsi, penyimpanan objek, dan evaluasi instruksi bytecode berjalan terstruktur.

Model memori runtime yang relevan:

Area	Peran
Code area	Menyimpan bytecode atau instruksi yang akan dieksekusi.
Stack	Menyimpan frame untuk pemanggilan fungsi atau method.
Activation record / frame	Unit data untuk satu pemanggilan method; berisi locals, operand stack, status mesin tersimpan, dan return value.
Operand stack	Stack internal dalam frame yang dipakai instruksi stack-based VM untuk operand sementara.
Local variables	Slot dalam frame untuk argumen dan variabel lokal method.
Heap	Area dinamis untuk objek yang umur hidupnya dapat melampaui satu pemanggilan fungsi.

Tabel 43: Komponen runtime memory model VM

Setiap pemanggilan method membuat frame baru. Ketika method selesai, frame di-pop dari stack dan kontrol kembali ke pemanggil. Namun objek yang dibuat di heap tidak otomatis hilang saat frame dihapus; objek hanya dapat dibebaskan jika tidak lagi reachable.

12.2.9 Siklus Fetch-Decode-Execute

Inti Konsep.

Interpreter VM menjalankan bytecode dengan siklus fetch-decode-execute. VM mengambil instruksi dari program counter, mendekode opcode, menjalankan aksi, lalu memperbarui state runtime.

Siklus dasarnya:

1. **Fetch:** ambil opcode pada posisi program counter (**pc**).
2. **Decode:** tentukan instruksi apa yang direpresentasikan opcode, sering memakai **switch/case**.
3. **Execute:** jalankan operasi, misalnya push, pop, add, branch, call, atau return.
4. **Update:** perbarui **pc**, operand stack, locals, heap, atau call stack sesuai instruksi.

Overhead utama interpreter berasal dari pengulangan decode dan dispatch instruksi. Setiap instruksi bytecode memerlukan cabang kontrol di interpreter sebelum kerja sebenarnya dilakukan. Pada CPU nyata, pola branch tidak langsung seperti ini dapat menyebabkan pipeline stalls.

12.2.10 Optimasi Interpreter VM

Inti Konsep.

Optimasi interpreter VM berusaha mengurangi overhead dispatch dan akses stack. Teknik yang disebut sumber mencakup threaded code, top-of-stack caching, dan super-instructions.

****Threaded code**** mengurangi biaya loop dispatch. Alih-alih selalu kembali ke loop utama untuk memilih instruksi berikutnya, implementasi dapat melompat langsung ke handler instruksi berikutnya.

****Top-of-stack caching**** menyimpan beberapa elemen puncak operand stack di register CPU nyata. Karena stack-based VM sering membaca dan menulis puncak stack, caching ini mengurangi akses memori.

****Super-instructions**** menggabungkan beberapa bytecode yang sering muncul berurutan menjadi satu instruksi gabungan. Misalnya pola multiply lalu add dapat digabung sebagai instruksi konseptual **madd**. Dengan begitu jumlah dispatch berkurang.

Super-instructions dapat dibedakan menjadi dua. Static super-instruction ditentukan sebelum eksekusi, misalnya VM menyediakan handler khusus untuk pola bytecode umum. Dynamic super-instruction dibentuk berdasarkan profil runtime atau basic block yang benar-benar sering muncul pada program yang sedang berjalan. Keduanya mengejar tujuan yang sama: mengurangi jumlah dispatch interpreter tanpa mengubah hasil eksekusi bytecode.

Optimasi ini tidak mengubah semantik bytecode. Tujuannya murni meningkatkan performa interpreter.

12.2.11 Garbage Collection

Inti Konsep.

Garbage collection membebaskan objek heap yang tidak lagi reachable. VM memulai dari root set, menelusuri referensi yang masih hidup, lalu menganggap objek lain sebagai sampah.

Stack frame dapat dihapus otomatis ketika fungsi selesai, tetapi heap berbeda. Objek heap dapat tetap dibutuhkan setelah fungsi pembuatnya selesai, selama masih ada referensi dari tempat lain. Karena itu VM perlu menentukan objek mana yang masih dapat dicapai.

****Root set**** mencakup referensi dari register, static/class data, dan stack frame aktif. Dari root set, GC menelusuri graf referensi ke objek heap. Objek yang tercapai disebut reachable dan harus dipertahankan. Objek yang tidak tercapai dapat dibebaskan.

Dua teknik yang disebut sumber:

- **Mark-and-sweep**: tandai objek reachable, lalu sweep objek yang tidak ditandai.
- **Reference counting**: simpan jumlah referensi ke setiap objek, lalu bebaskan ketika hitungan menjadi nol.

Garbage collection membuat manajemen memori lebih aman untuk programmer, tetapi menambah pekerjaan runtime pada VM.

12.2.12 Relasi VM dengan Intermediate Code dan Code Generation

Inti Konsep.

Bytecode VM adalah bentuk intermediate representation yang cukup rendah untuk dieksekusi. Code generator dapat menargetkan bytecode, sementara JIT kemudian menargetkan mesin fisik.

Pada compiler tradisional, code generation menghasilkan assembly atau machine code. Pada sistem berbasis VM, code generation menghasilkan bytecode untuk mesin abstrak. Jadi VM dapat dipahami sebagai target machine dari compiler.

Relasinya dengan Intermediate Code:

- AST dan 3AC adalah IR yang dipakai untuk analisis dan optimasi.
- Bytecode adalah IR linear level rendah yang dirancang untuk dieksekusi VM.
- Stack-based bytecode mirip evaluasi postfix: operand di-push sampai operator muncul, lalu operator melakukan pop dan push hasil.

JIT compiler berada setelah bytecode. Ia membaca bytecode, melakukan optimasi yang bergantung pada mesin dan profil runtime, lalu menghasilkan native code. Dengan kata lain, JIT memindahkan sebagian pekerjaan back end ke runtime.

12.2.13 Kaitan dengan Contoh UAS

Inti Konsep.

Sumber UAS mengaitkan VM dengan kemampuan membaca eksekusi stack, postfix, branch, dan assembly sederhana. Ini berarti materi VM sering muncul tidak sebagai definisi saja, tetapi sebagai trace eksekusi.

Pola yang relevan untuk UAS:

- Evaluasi postfix menyerupai stack-based VM: operand di-push, operator mem-pop operand dan mem-push hasil.
- Instruksi branch seperti BZ dan BR berhubungan dengan eksekusi control flow pada interpreter.
- Translasi ke single accumulator assembly seperti LDA, ADD, MUL, dan STO melatih pemetaan IR atau bytecode ke operasi mesin sederhana.

Karena itu, saat belajar VM, fokuskan perhatian pada perubahan state: isi operand stack, program counter, frame aktif, dan efek tiap instruksi.

12.3 Algoritma dan Rumus Penting

12.3.1 Pseudocode Interpreter Loop

Tujuan Algoritma.

Interpreter loop menunjukkan cara VM mengeksekusi bytecode. Setiap iterasi mengambil opcode, memilih handler, menjalankan aksi, dan memperbarui program counter.

Listing 14: Pseudocode fetch-decode-execute VM

```
1 pc = entryPoint
2 while true:
3     opcode = code[pc]
4     pc = pc + 1
5
6     switch opcode:
7         case ICONST:
8             value = readOperand(code, pc)
9             pc = pc + 1
10            operandStack.push(value)
11
12        case IADD:
13            b = operandStack.pop()
14            a = operandStack.pop()
15            operandStack.push(a + b)
16
17        case GOTO:
18            target = readOperand(code, pc)
19            pc = target
20
21        case RETURN:
22            return operandStack.pop()
```

Pseudocode ini menyederhanakan banyak detail, tetapi memuat ide utama VM: bytecode adalah data, dan interpreter adalah program yang membaca data tersebut sebagai instruksi.

12.3.2 Pseudocode Eksekusi Instruksi Stack

Tujuan Algoritma.

Pseudocode ini memperlihatkan pola umum stack-based VM: operand diambil dari puncak stack, operasi dijalankan, lalu hasil didorong kembali.

Listing 15: Pola instruksi aritmetika stack-based VM

```
1 function executeBinaryOp(op):
2     right = operandStack.pop()
3     left = operandStack.pop()
4     result = apply(op, left, right)
5     operandStack.push(result)
```

Instruksi seperti `iadd`, `imul`, dan `isub` mengikuti pola ini. Perbedaannya hanya pada operasi yang diberikan ke fungsi `apply`.

12.3.3 Pseudocode Mark-and-Sweep

Tujuan Algoritma.

Mark-and-sweep membebaskan objek heap yang tidak reachable. Algoritma terdiri dari fase mark untuk menandai objek hidup dan fase sweep untuk membebaskan objek mati.

Listing 16: Pseudocode garbage collection mark-and-sweep

```
1 function collectGarbage():
2     for each object in heap:
3         object.marked = false
4
5     for each root in rootSet:
6         mark(root)
7
8     for each object in heap:
9         if object.marked == false:
10            free(object)
11
12 function mark(object):
13     if object == null or object.marked:
14         return
15     object.marked = true
16     for each reference child in object.fields:
17         mark(child)
```

Root set biasanya mencakup referensi dari stack frame aktif, static data, dan register atau state runtime VM.